ExpositoTOP

1.0.0

Generado por Doxygen 1.9.1

1 ExpositoTOP — Referencia del código **1**
1.1 Componentes . . . 1
1.2 Flujo de ejecución . . . 1

2 Indice de namespaces **3**
2.1 Paquetes . . . 3

3 Indice jerárquico **5**
3.1 Jerarquía de la clase . . . 5

4 Índice de clases **7**
4.1 Lista de clases . . . 7

5 Indice de archivos **9**
5.1 Lista de archivos . . . 9

6 Documentación de namespaces **11**
6.1 Paquetes es.ull.esit.utilities . . . 11
6.2 Paquetes es.ull.esit.utils . . . 11
6.2.1 Documentación de las funciones . . . 11
6.2.1.1 Pair< F, S >() . . . 11
6.3 Paquetes top . . . 12

7 Documentación de las clases **13**
7.1 Referencia de la Clase es.ull.esit.utilities.BellmanFord . . . 13
7.1.1 Descripción detallada . . . 15
7.1.2 Documentación del constructor y destructor . . . 15
7.1.2.1 BellmanFord() . . . 15
7.1.3 Documentación de las funciones miembro . . . 16
7.1.3.1 calculateEdges() . . . 16
7.1.3.2 getDistances() . . . 17
7.1.3.3 getValue() . . . 17
7.1.3.4 solve() . . . 17
7.1.4 Documentación de los datos miembro . . . 17
7.1.4.1 INFINITY . . . 18
7.1.4.2 distanceMatrix . . . 18
7.1.4.3 edges1 . . . 18
7.1.4.4 edges2 . . . 18
7.1.4.5 nodes . . . 18
7.1.4.6 path . . . 19
7.1.4.7 distances . . . 19
7.1.4.8 value . . . 19
7.2 Referencia de la Clase es.ull.esit.utilities.ExpositoUtilities . . . 20
7.2.1 Descripción detallada . . . 21

7.2.2 Documentación del constructor y destructor . . . 22
7.2.2.1 ExpositoUtilities() . . . 22
7.2.3 Documentación de las funciones miembro . . . 22
7.2.3.1 printFile() . . . 22
7.2.3.2 simplifyString() . . . 22
7.2.3.3 multiplyMatrices() . . . 23
7.2.3.4 writeTextToFile() . . . 23
7.2.3.5 getFormat() [1/11] . . . 24
7.2.3.6 getFormat() [2/11] . . . 25
7.2.3.7 getFormat() [3/11] . . . 25
7.2.3.8 getFormat() [4/11] . . . 26
7.2.3.9 getFormat() [5/11] . . . 27
7.2.3.10 getFormat() [6/11] . . . 27
7.2.3.11 getFormatIntegers() . . . 28
7.2.3.12 getFormat() [7/11] . . . 28
7.2.3.13 getFormat() [8/11] . . . 29
7.2.3.14 getFormat() [9/11] . . . 30
7.2.3.15 getFormat() [10/11] . . . 30
7.2.3.16 getFormat() [11/11] . . . 31
7.2.3.17 isInteger() . . . 32
7.2.3.18 isDouble() . . . 32
7.2.3.19 isAcyclic() . . . 33
7.2.3.20 thereIsPath() . . . 33
7.2.3.21 hasArc() . . . 34
7.2.4 Documentación de los datos miembro . . . 35
7.2.4.1 DEFAULT_COLUMN_WIDTH . . . 35
7.2.4.2 ALIGNMENT_LEFT . . . 35
7.2.4.3 ALIGNMENT_RIGHT . . . 36
7.3 Referencia de la Clase top.MainTOPTW . . . 36
7.3.1 Descripción detallada . . . 38
7.3.2 Documentación del constructor y destructor . . . 38
7.3.2.1 MainTOPTW() . . . 38
7.3.3 Documentación de las funciones miembro . . . 38
7.3.3.1 parseArgs() . . . 38
7.3.3.2 optionValue() . . . 39
7.3.3.3 run() . . . 40
7.3.3.4 main() . . . 41
7.3.4 Documentación de los datos miembro . . . 42
7.3.4.1 DEFAULT_DIR . . . 42
7.3.4.2 DEFAULT_ITERATIONS . . . 42
7.3.4.3 RCL_SIZES . . . 43
7.3.4.4 DEFAULT_INSTANCES . . . 43

7.3.4.5 LOGGER . . . 43
7.4 Referencia de la Clase top.MainTOPTW.Options . . . 44
7.4.1 Descripción detallada . . . 44
7.4.2 Documentación de los datos miembro . . . 45
7.4.2.1 dir . . . 45
7.4.2.2 iterations . . . 45
7.4.2.3 seed . . . 45
7.4.2.4 instances . . . 45
7.5 Referencia de la plantilla de la Clase es.ull.esit.utilities.PowerSet< E > . . . 46
7.5.1 Descripción detallada . . . 47
7.5.2 Documentación del constructor y destructor . . . 47
7.5.2.1 PowerSet() . . . 47
7.5.3 Documentación de las funciones miembro . . . 48
7.5.3.1 iterator() . . . 48
7.5.4 Documentación de los datos miembro . . . 48
7.5.4.1 arr . . . 48
7.6 Referencia de la Clase es.ull.esit.utilities.PowerSet< E >.PowerSetIterator . . . 49
7.6.1 Descripción detallada . . . 50
7.6.2 Documentación de las funciones miembro . . . 50
7.6.2.1 hasNext() . . . 50
7.6.2.2 next() . . . 50
7.6.3 Documentación de los datos miembro . . . 51
7.6.3.1 bset . . . 51
7.7 Referencia del enum top.TOPTWGRASP.SelectionStrategy . . . 51
7.7.1 Descripción detallada . . . 52
7.7.2 Documentación de los datos miembro . . . 52
7.7.2.1 RANDOM . . . 52
7.7.2.2 FUZZY_BEST . . . 52
7.7.2.3 FUZZY_ALPHA_CUT . . . 52
7.8 Referencia de la Clase top.TOPTW . . . 53
7.8.1 Descripción detallada . . . 55
7.8.2 Documentación del constructor y destructor . . . 56
7.8.2.1 TOPTW() . . . 56
7.8.3 Documentación de las funciones miembro . . . 56
7.8.3.1 isDepot() . . . 56
7.8.3.2 dataIndex() . . . 57
7.8.3.3 getDistance() [1/3] . . . 58
7.8.3.4 getDistance() [2/3] . . . 58
7.8.3.5 getTotalDistance() . . . 59
7.8.3.6 calculateDistanceMatrix() . . . 60
7.8.3.7 getMaxTimePerRoute() . . . 60
7.8.3.8 setMaxTimePerRoute() . . . 61

7.8.3.9 getMaxRoutes() . . . 61
7.8.3.10 setMaxRoutes() . . . 61
7.8.3.11 getPOIs() . . . 62
7.8.3.12 getDistance() [3/3] . . . 62
7.8.3.13 getTime() . . . 63
7.8.3.14 getNodes() . . . 64
7.8.3.15 setNodes() . . . 64
7.8.3.16 getX() . . . 64
7.8.3.17 setX() . . . 66
7.8.3.18 getY() . . . 67
7.8.3.19 setY() . . . 68
7.8.3.20 getScore() [1/2] . . . 68
7.8.3.21 getScore() [2/2] . . . 69
7.8.3.22 setScore() . . . 69
7.8.3.23 getReadyTime() . . . 70
7.8.3.24 setReadyTime() . . . 71
7.8.3.25 getDueTime() . . . 71
7.8.3.26 setDueTime() . . . 72
7.8.3.27 getServiceTime() . . . 72
7.8.3.28 setServiceTime() . . . 73
7.8.3.29 getVehicles() . . . 74
7.8.3.30 getDepots() . . . 74
7.8.3.31 toString() . . . 75
7.8.3.32 addNode() . . . 75
7.8.3.33 addNodeDepot() . . . 76
7.8.4 Documentación de los datos miembro . . . 76
7.8.4.1 nodes . . . 76
7.8.4.2 x . . . 76
7.8.4.3 y . . . 77
7.8.4.4 score . . . 77
7.8.4.5 readyTime . . . 77
7.8.4.6 dueTime . . . 77
7.8.4.7 serviceTime . . . 78
7.8.4.8 vehicles . . . 78
7.8.4.9 depots . . . 78
7.8.4.10 maxTimePerRoute . . . 78
7.8.4.11 maxRoutes . . . 79
7.8.4.12 distanceMatrix . . . 79
7.9 Referencia de la Clase top.TOPTWEvaluator . . . 79
7.9.1 Descripción detallada . . . 80
7.9.2 Documentación de las funciones miembro . . . 80
7.9.2.1 evaluate() . . . 80

7.9.3 Documentación de los datos miembro . . . 81
7.9.3.1 NO_EVALUATED . . . 81
7.10 Referencia de la Clase top.TOPTWGRASP . . . 81
7.10.1 Descripción detallada . . . 85
7.10.2 Documentación del constructor y destructor . . . 85
7.10.2.1 TOPTWGRASP() [1/2] . . . 85
7.10.2.2 TOPTWGRASP() [2/2] . . . 86
7.10.3 Documentación de las funciones miembro . . . 86
7.10.3.1 grasp() . . . 86
7.10.3.2 print() . . . 87
7.10.3.3 aleatorySelectionRCL() . . . 88
7.10.3.4 membership() . . . 88
7.10.3.5 fuzzySelectionBestFDRCL() . . . 89
7.10.3.6 fuzzySelectionAlphaCutRCL() . . . 90
7.10.3.7 select() . . . 91
7.10.3.8 newDepartureTimes() . . . 91
7.10.3.9 sortedCandidates() . . . 92
7.10.3.10 computeGreedySolution() . . . 93
7.10.3.11 updateSolution() . . . 94
7.10.3.12 departureTime() . . . 95
7.10.3.13 insertionCost() . . . 96
7.10.3.14 comprehensiveEvaluation() . . . 97
7.10.3.15 getSolution() . . . 98
7.10.3.16 setSolution() . . . 98
7.10.3.17 getSolutionTime() . . . 99
7.10.3.18 setSolutionTime() . . . 99
7.10.3.19 getStrategy() . . . 99
7.10.3.20 setStrategy() . . . 100
7.10.3.21 getAlpha() . . . 100
7.10.3.22 setAlpha() . . . 100
7.10.3.23 setOutput() . . . 101
7.10.3.24 getLastAverageFitness() . . . 101
7.10.3.25 getLastBestFitness() . . . 101
7.10.3.26 getMaxScore() . . . 102
7.10.4 Documentación de los datos miembro . . . 102
7.10.4.1 CUSTOMER . . . 102
7.10.4.2 ROUTE . . . 103
7.10.4.3 PREDECESSOR . . . 103
7.10.4.4 COST . . . 103
7.10.4.5 SCORE . . . 103
7.10.4.6 CANDIDATE_INFO_SIZE . . . 104
7.10.4.7 DEFAULT_ALPHA . . . 104

7.10.4.8 solution . . . 104
7.10.4.9 solutionTime . . . 104
7.10.4.10 random . . . 105
7.10.4.11 strategy . . . 105
7.10.4.12 alpha . . . 105
7.10.4.13 out . . . 105
7.10.4.14 lastAverageFitness . . . 106
7.10.4.15 lastBestFitness . . . 106
7.11 Referencia de la Clase top.TOPTWReader . . . 106
7.11.1 Descripción detallada . . . 107
7.11.2 Documentación del constructor y destructor . . . 107
7.11.2.1 TOPTWReader() . . . 108
7.11.3 Documentación de las funciones miembro . . . 108
7.11.3.1 readProblem() . . . 108
7.11.3.2 nextFields() . . . 109
7.11.4 Documentación de los datos miembro . . . 110
7.11.4.1 X_FIELD . . . 110
7.11.4.2 Y_FIELD . . . 110
7.11.4.3 SERVICE_FIELD . . . 111
7.11.4.4 SCORE_FIELD . . . 111
7.12 Referencia de la Clase top.TOPTWRoute . . . 111
7.12.1 Descripción detallada . . . 112
7.12.2 Documentación del constructor y destructor . . . 112
7.12.2.1 TOPTWRoute() [1/2] . . . 113
7.12.2.2 TOPTWRoute() [2/2] . . . 113
7.12.3 Documentación de las funciones miembro . . . 113
7.12.3.1 getPredecessor() . . . 113
7.12.3.2 getSuccessor() . . . 114
7.12.3.3 getId() . . . 114
7.12.3.4 setPredecessor() . . . 114
7.12.3.5 setSuccessor() . . . 114
7.12.3.6 setId() . . . 115
7.12.4 Documentación de los datos miembro . . . 115
7.12.4.1 predecessor . . . 115
7.12.4.2 successor . . . 115
7.12.4.3 id . . . 116
7.13 Referencia de la Clase top.TOPTWSolution . . . 116
7.13.1 Descripción detallada . . . 119
7.13.2 Documentación del constructor y destructor . . . 119
7.13.2.1 TOPTWSolution() . . . 119
7.13.3 Documentación de las funciones miembro . . . 120
7.13.3.1 initSolution() . . . 120

7.13.3.2 isDepot() . . . 121
7.13.3.3 equals() . . . 122
7.13.3.4 hashCode() . . . 123
7.13.3.5 getAvailableVehicles() . . . 123
7.13.3.6 getCreatedRoutes() . . . 123
7.13.3.7 getDistance() . . . 124
7.13.3.8 setAvailableVehicles() . . . 125
7.13.3.9 getPredecessor() . . . 125
7.13.3.10 getPredecessors() . . . 126
7.13.3.11 getProblem() . . . 126
7.13.3.12 getObjectiveFunctionValue() . . . 127
7.13.3.13 getPositionInRoute() . . . 127
7.13.3.14 getSuccessor() . . . 127
7.13.3.15 getSuccessors() . . . 128
7.13.3.16 getIndexRoute() . . . 128
7.13.3.17 getWaitingTime() . . . 129
7.13.3.18 setObjectiveFunctionValue() . . . 129
7.13.3.19 setPositionInRoute() . . . 130
7.13.3.20 setPredecessor() . . . 130
7.13.3.21 setSuccessor() . . . 131
7.13.3.22 setWaitingTime() . . . 131
7.13.3.23 getInfoSolution() . . . 132
7.13.3.24 evaluateFitness() . . . 133
7.13.3.25 addRoute() . . . 134
7.13.3.26 printSolution() . . . 134
7.13.4 Documentación de los datos miembro . . . 135
7.13.4.1 NO_INITIALIZED . . . 135
7.13.4.2 problem . . . 135
7.13.4.3 predecessors . . . 136
7.13.4.4 successors . . . 136
7.13.4.5 waitingTime . . . 136
7.13.4.6 positionInRoute . . . 136
7.13.4.7 routes . . . 136
7.13.4.8 availableVehicles . . . 137
7.13.4.9 objectiveFunctionValue . . . 137

8 Documentación de archivos **139**

8.1 Referencia del Archivo /home/andrea/Descargas/ExpositoTOP/doxygen/mainpage.md . . . 139
8.2 Referencia del Archivo es/ull/esit/utilities/BellmanFord.java . . . 139
8.3 Referencia del Archivo es/ull/esit/utilities/ExpositoUtilities.java . . . 139
8.4 Referencia del Archivo es/ull/esit/utilities/PowerSet.java . . . 139
8.5 Referencia del Archivo es/ull/esit/utils/Pair.java . . . 140

8.6 Referencia del Archivo top/MainTOPTW.java 140
8.7 Referencia del Archivo top/TOPTW.java 140
8.8 Referencia del Archivo top/TOPTWEvaluator.java 140
8.9 Referencia del Archivo top/TOPTWGRASP.java 141
8.10 Referencia del Archivo top/TOPTWReader.java 141
8.11 Referencia del Archivo top/TOPTWRoute.java 141
8.12 Referencia del Archivo top/TOPTWSolution.java 141

Índice alfabético **143**

Capítulo 1

ExpositoTOP — Referencia del código

RTVRPTW: Recomendador Turístico con Ventanas de Tiempo.

El sistema construye rutas turísticas de varios días: cada día es una ruta que sale del hotel (depósito), visita puntos de interés (POI) dentro de su horario de apertura y vuelve antes de una hora límite. El objetivo es maximizar el interés total (puntuación) de los POI visitados. Formalmente es el *Team Orienteering Problem with Time Windows* (TOPTW), que se resuelve con la metaheurística **GRASP**.

1.1. Componentes

Paquete	Clase	Responsabilidad
`top`	top::TOPTW	Modelo de la instancia: POI, ventanas temporales y distancias
`top`	top::TOPTWReader	Lectura de instancias en formato Cordeau/Righini
`top`	top::TOPTWSolution	Rutas como listas doblemente enlazadas; fitness
`top`	top::TOPTWEvaluator	Función objetivo
`top`	top::TOPTWGRASP	Construcción voraz aleatorizada con RCL y selección borrosa
`top`	top::MainTOPTW	Programa de línea de órdenes
`es.ull.esit.utilities`	es::ull::esit::utilities::ExpositoUtilities	Formato de tablas, ficheros, grafos
`es.ull.esit.utilities`	es::ull::esit::utilities::PowerSet, es::ull::esit::utilities::BellmanFord	Algoritmos auxiliares
`es.ull.esit.utils`	es::ull::esit::utils::Pair	Par inmutable

1.2. Flujo de ejecución

1. top::MainTOPTW lee cada instancia con top::TOPTWReader::readProblem.
2. Crea una top::TOPTWSolution y un top::TOPTWGRASP.
3. top::TOPTWGRASP::grasp repite top::TOPTWGRASP::computeGreedySolution e informa de la media y de la mejor puntuación.

El portal completo (manual de uso, arquitectura, calidad y decisiones de diseño) se genera con MkDocs.

Capítulo 2

Indice de namespaces

2.1. Paquetes

Aquí van los paquetes con una breve descripción (si etá disponible):

es.ull.esit.utilities 11
es.ull.esit.utils 11
top 12

Capítulo 3

Indice jerárquico

3.1. Jerarquía de la clase

Esta lista de herencias esta ordenada aproximadamente por orden alfabético:

es.ull.esit.utilities.BellmanFord . . . 13
es.ull.esit.utilities.ExpositoUtilities . . . 20
Iterable
    es.ull.esit.utilities.PowerSet< E > . . . 46
top.MainTOPTW . . . 36
top.MainTOPTW.Options . . . 44
top.TOPTWGRASP.SelectionStrategy . . . 51
top.TOPTW . . . 53
top.TOPTWEvaluator . . . 79
top.TOPTWGRASP . . . 81
top.TOPTWReader . . . 106
top.TOPTWRoute . . . 111
top.TOPTWSolution . . . 116
Iterator
    es.ull.esit.utilities.PowerSet< E >.PowerSetIterator . . . 49

Capítulo 4

Índice de clases

4.1. Lista de clases

Lista de las clases, estructuras, uniones e interfaces con una breve descripción:

es.ull.esit.utilities.BellmanFord
: Algoritmo de Bellman-Ford sobre una matriz de distancias 13

es.ull.esit.utilities.ExpositoUtilities
: Funciones de utilidad: formato de texto tabular, ficheros, matrices y grafos 20

top.MainTOPTW
: Punto de entrada del recomendador turístico RTVRPTW / TOPTW 36

top.MainTOPTW.Options
: Opciones de ejecución obtenidas de la línea de órdenes 44

es.ull.esit.utilities.PowerSet< E >
: Conjunto potencia (todos los subconjuntos) de un conjunto, recorrible con for-each 46

es.ull.esit.utilities.PowerSet< E >.PowerSetIterator
: Iterador sobre los subconjuntos, basado en un contador binario 49

top.TOPTWGRASP.SelectionStrategy
: Estrategias de selección de un candidato dentro de la RCL 51

top.TOPTW
: Modelo de una instancia del problema TOPTW (Team Orienteering Problem with Time Windows) 53

top.TOPTWEvaluator
: Evaluador de la función objetivo del TOPTW 79

top.TOPTWGRASP
: Metaheurística GRASP (Greedy Randomized Adaptive Search Procedure) para el TOPTW . . 81

top.TOPTWReader
: Lector de instancias TOPTW en el formato de Cordeau / Righini 106

top.TOPTWRoute
: Descriptor ligero de una ruta: identificador y nodos extremos 111

top.TOPTWSolution
: Solución del problema TOPTW representada mediante listas doblemente enlazadas 116

Capítulo 5

Indice de archivos

5.1. Lista de archivos

Lista de todos los archivos con descripciones breves:

es/ull/esit/utilities/BellmanFord.java . . . 139
es/ull/esit/utilities/ExpositoUtilities.java . . . 139
es/ull/esit/utilities/PowerSet.java . . . 139
es/ull/esit/utils/Pair.java . . . 140
top/MainTOPTW.java . . . 140
top/TOPTW.java . . . 140
top/TOPTWEvaluator.java . . . 140
top/TOPTWGRASP.java . . . 141
top/TOPTWReader.java . . . 141
top/TOPTWRoute.java . . . 141
top/TOPTWSolution.java . . . 141

Capítulo 6

Documentación de namespaces

6.1. Paquetes es.ull.esit.utilities

Clases

- class BellmanFord

  Algoritmo de Bellman-Ford sobre una matriz de distancias.
- class ExpositoUtilities

  Funciones de utilidad: formato de texto tabular, ficheros, matrices y grafos.
- class PowerSet

  Conjunto potencia (todos los subconjuntos) de un conjunto, recorrible con for-each.

6.2. Paquetes es.ull.esit.utils

Funciones

- record Pair< F, S > (F first, S second)

  Par inmutable de valores.

6.2.1. Documentación de las funciones

6.2.1.1. Pair< F, S >()

```
record es.ull.esit.utils.Pair< F, S > (
            F first,
            S second )
```

Par inmutable de valores.

Se declara como record: Java genera automáticamente el constructor, los accesores first() y second(), equals(), hashCode() y toString().

Parámetros

<F>	tipo del primer elemento.
<S>	tipo del segundo elemento.
first	primer elemento.
second	segundo elemento.

Método factoría.

Parámetros

<A>	tipo del primer elemento.
<B>	tipo del segundo elemento.
a	primer elemento.
b	segundo elemento.

Devuelve

nuevo par.

Definición en la línea 14 del archivo Pair.java.

6.3. Paquetes top

Clases

- class MainTOPTW

  Punto de entrada del recomendador turístico RTVRPTW / TOPTW.
- class TOPTW

  Modelo de una instancia del problema TOPTW (Team Orienteering Problem with Time Windows).
- class TOPTWEvaluator

  Evaluador de la función objetivo del TOPTW.
- class TOPTWGRASP

  Metaheurística GRASP (Greedy Randomized Adaptive Search Procedure) para el TOPTW.
- class TOPTWReader

  Lector de instancias TOPTW en el formato de Cordeau / Righini.
- class TOPTWRoute

  Descriptor ligero de una ruta: identificador y nodos extremos.
- class TOPTWSolution

  Solución del problema TOPTW representada mediante listas doblemente enlazadas.

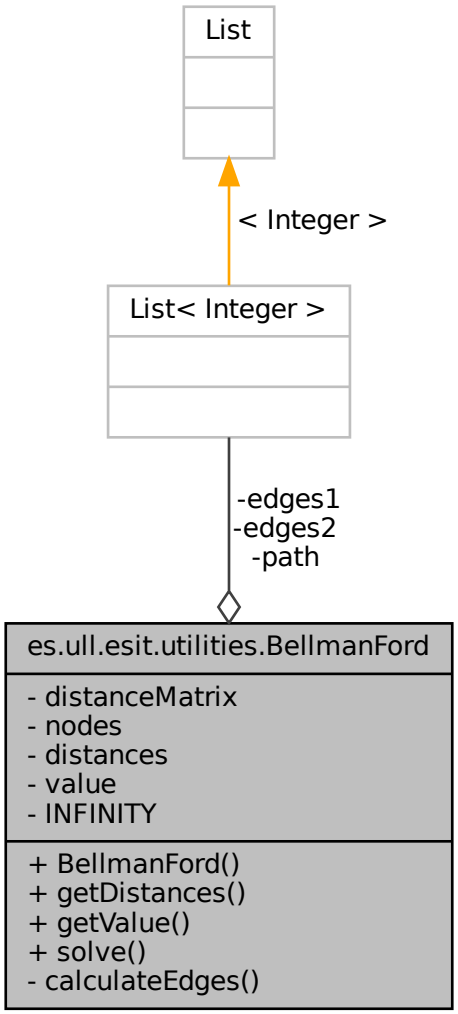
Capítulo 7

Documentación de las clases

7.1. Referencia de la Clase es.ull.esit.utilities.BellmanFord

Algoritmo de Bellman-Ford sobre una matriz de distancias.

Diagrama de colaboración para es.ull.esit.utilities.BellmanFord:



Métodos públicos

- BellmanFord (int[ ][ ] distanceMatrix, int nodes, List< Integer > path)

  Prepara el algoritmo.
- int[ ] getDistances ()

  Distancias mínimas desde el nodo 0.
- int getValue ()

  Valor del camino hasta el último nodo, cambiado de signo.
- void solve ()

  Ejecuta Bellman-Ford y reconstruye el camino hasta el último nodo.

Métodos privados

- void calculateEdges ()

  Construye la lista de arcos a partir de la matriz.

Atributos privados

- final int[ ][ ] distanceMatrix

  Matriz de distancias (Integer.MAX_VALUE = sin arco).
- List< Integer > edges1 = null

  Nodos origen de cada arco.
- List< Integer > edges2 = null

  Nodos destino de cada arco.
- final int nodes

  Número de nodos.
- final List< Integer > path

  Camino encontrado hasta el último nodo (en orden inverso).
- int[ ] distances = null

  Distancias mínimas desde el nodo 0.
- int value

  Valor del camino hasta el último nodo, cambiado de signo.

Atributos privados estáticos

- static final int INFINITY = 999999

  Valor usado como distancia infinita.

7.1.1. Descripción detallada

Algoritmo de Bellman-Ford sobre una matriz de distancias.

Calcula los caminos mínimos desde el nodo 0 a todos los demás. Los arcos se obtienen de la matriz: Integer.↩ MAX_VALUE significa que no hay arco. Admite pesos negativos, por lo que con los pesos cambiados de signo sirve para obtener el camino máximo en un grafo acíclico (de ahí que getValue() devuelva la distancia cambiada de signo). Tras solve(), la lista path contiene el camino desde el último nodo hasta el 0 en orden inverso.

Definición en la línea 16 del archivo BellmanFord.java.

7.1.2. Documentación del constructor y destructor

7.1.2.1. BellmanFord()

```
es.ull.esit.utilities.BellmanFord.BellmanFord (
            int distanceMatrix[][],
            int nodes,
            List< Integer > path )
```

Prepara el algoritmo.

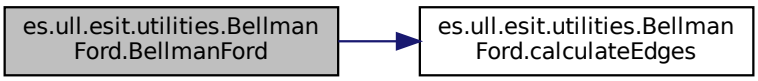
Parámetros

distanceMatrix	matriz de distancias (Integer.MAX_VALUE = sin arco).
nodes	número de nodos.
path	lista donde se escribirá el camino resultante.

Definición en la línea 41 del archivo BellmanFord.java.

Hace referencia a es.ull.esit.utilities.BellmanFord.calculateEdges(), es.ull.esit.utilities.BellmanFord.distanceMatrix, es.ull.esit.utilities.BellmanFord.INFINITY, es.ull.esit.utilities.BellmanFord.nodes y es.ull.esit.utilities.BellmanFord.↩ path.

Gráfico de llamadas para esta función:



7.1.3. Documentación de las funciones miembro

7.1.3.1. calculateEdges()

```
void es.ull.esit.utilities.BellmanFord.calculateEdges ( )  [private]
```

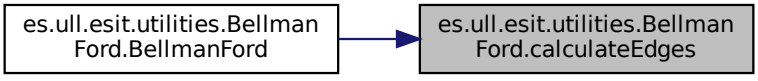
Construye la lista de arcos a partir de la matriz.

Definición en la línea 52 del archivo BellmanFord.java.

Hace referencia a es.ull.esit.utilities.BellmanFord.nodes.

Referenciado por es.ull.esit.utilities.BellmanFord.BellmanFord().

Gráfico de llamadas a esta función:



7.1.3.2. getDistances()

```
int [] es.ull.esit.utilities.BellmanFord.getDistances ( )
```

Distancias mínimas desde el nodo 0.

Devuelve

copia del vector de distancias, o null si no se ha ejecutado solve().

Definición en la línea 69 del archivo BellmanFord.java.

7.1.3.3. getValue()

```
int es.ull.esit.utilities.BellmanFord.getValue ( )
```

Valor del camino hasta el último nodo, cambiado de signo.

Devuelve

-distancia(0, n-1).

Definición en la línea 77 del archivo BellmanFord.java.

Hace referencia a es.ull.esit.utilities.BellmanFord.value.

7.1.3.4. solve()

```
void es.ull.esit.utilities.BellmanFord.solve ( )
```

Ejecuta Bellman-Ford y reconstruye el camino hasta el último nodo.

Definición en la línea 84 del archivo BellmanFord.java.

Hace referencia a es.ull.esit.utilities.BellmanFord.INFINITY y es.ull.esit.utilities.BellmanFord.nodes.

7.1.4. Documentación de los datos miembro

7.1.4.1. INFINITY

```
final int es.ull.esit.utilities.BellmanFord.INFINITY = 999999  [static], [private]
```

Valor usado como distancia infinita.

Definición en la línea 19 del archivo BellmanFord.java.

Referenciado por es.ull.esit.utilities.BellmanFord.BellmanFord() y es.ull.esit.utilities.BellmanFord.solve().

7.1.4.2. distanceMatrix

```
final int [][] es.ull.esit.utilities.BellmanFord.distanceMatrix  [private]
```

Matriz de distancias (Integer.MAX_VALUE = sin arco).

Definición en la línea 21 del archivo BellmanFord.java.

Referenciado por es.ull.esit.utilities.BellmanFord.BellmanFord().

7.1.4.3. edges1

```
List<Integer> es.ull.esit.utilities.BellmanFord.edges1 = null  [private]
```

Nodos origen de cada arco.

Definición en la línea 23 del archivo BellmanFord.java.

7.1.4.4. edges2

```
List<Integer> es.ull.esit.utilities.BellmanFord.edges2 = null  [private]
```

Nodos destino de cada arco.

Definición en la línea 25 del archivo BellmanFord.java.

7.1.4.5. nodes

```
final int es.ull.esit.utilities.BellmanFord.nodes  [private]
```

Número de nodos.

Definición en la línea 27 del archivo BellmanFord.java.

Referenciado por es.ull.esit.utilities.BellmanFord.BellmanFord(), es.ull.esit.utilities.BellmanFord.calculateEdges() y es.ull.esit.utilities.BellmanFord.solve().

7.1.4.6. path

```
final List<Integer> es.ull.esit.utilities.BellmanFord.path  [private]
```

Camino encontrado hasta el último nodo (en orden inverso).

Definición en la línea 29 del archivo BellmanFord.java.

Referenciado por es.ull.esit.utilities.BellmanFord.BellmanFord().

7.1.4.7. distances

```
int [] es.ull.esit.utilities.BellmanFord.distances = null  [private]
```

Distancias mínimas desde el nodo 0.

Definición en la línea 31 del archivo BellmanFord.java.

7.1.4.8. value

```
int es.ull.esit.utilities.BellmanFord.value  [private]
```

Valor del camino hasta el último nodo, cambiado de signo.

Definición en la línea 33 del archivo BellmanFord.java.

Referenciado por es.ull.esit.utilities.BellmanFord.getValue().

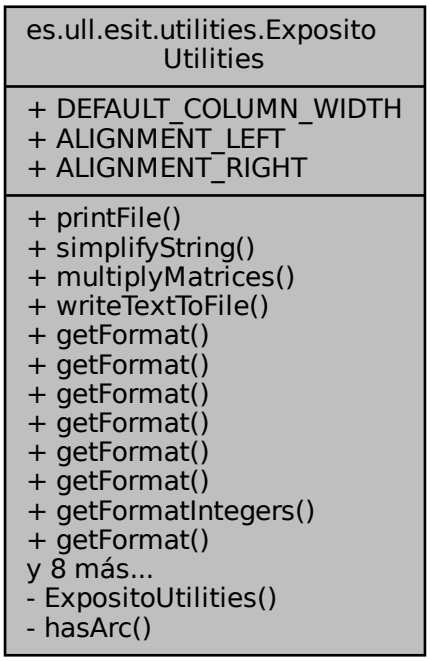
La documentación para esta clase fue generada a partir del siguiente fichero:

- es/ull/esit/utilities/BellmanFord.java

7.2. Referencia de la Clase es.ull.esit.utilities.ExpositoUtilities

Funciones de utilidad: formato de texto tabular, ficheros, matrices y grafos.

Diagrama de colaboración para es.ull.esit.utilities.ExpositoUtilities:



Métodos públicos estáticos

- static void printFile (String file, PrintStream out) throws IOException

  Copia el contenido de un fichero de texto en un flujo de salida.
- static String simplifyString (String string)

  Normaliza los espacios de una cadena.
- static double[ ][ ] multiplyMatrices (double[ ][ ] a, double[ ][ ] b)

  Producto de matrices.
- static void writeTextToFile (String file, String text) throws IOException

  Escribe un texto en un fichero, sobrescribiéndolo.
- static String getFormat (String string)

  Formatea una cadena numérica: los reales no enteros pasan a tres decimales.
- static String getFormat (double value)

  Formatea un real con tres decimales y punto decimal.
- static String getFormat (double value, int zeros)

  Formatea un real con el número de decimales indicado y punto decimal.
- static String getFormat (String string, int width)

  Alinea una cadena a la derecha en una columna.
- static String getFormat (String string, int width, int alignment)

Alinea una cadena en una columna.

- static String getFormat (List< String > strings, int width)

  Formatea una lista de cadenas en columnas del mismo ancho.
- static String getFormatIntegers (List< Integer > values)

  Formatea una lista de enteros en columnas de ancho DEFAULT_COLUMN_WIDTH.
- static String getFormat (String[ ] strings, int width)

  Formatea un vector de cadenas en columnas del mismo ancho, alineadas a la derecha.
- static String getFormat (String[ ][ ] matrixStrings, int width)

  Formatea una matriz de cadenas: una fila por línea.
- static String getFormat (String[ ] strings)

  Formatea un vector de cadenas con el ancho por defecto.
- static String getFormat (String[ ] strings, int[ ] width)

  Formatea un vector de cadenas con un ancho por columna, alineadas a la derecha.
- static String getFormat (String[ ] strings, int[ ] width, int[ ] alignment)

  Formatea un vector de cadenas con ancho y alineación por columna.
- static boolean isInteger (String str)

  Indica si una cadena representa un entero.
- static boolean isDouble (String str)

  Indica si una cadena representa un real.
- static boolean isAcyclic (int[ ][ ] distanceMatrix)

  Comprueba si un grafo de tareas es acíclico.
- static boolean thereIsPath (int[ ][ ] distanceMatrix, int node)

  Comprueba si existe un camino de un nodo a sí mismo (ciclo).

Atributos públicos estáticos

- static final int DEFAULT_COLUMN_WIDTH = 10

  Ancho de columna por defecto.
- static final int ALIGNMENT_LEFT = 1

  Alineación a la izquierda.
- static final int ALIGNMENT_RIGHT = 2

  Alineación a la derecha.

Métodos privados

- ExpositoUtilities ()

  Clase de utilidad: no instanciable.

Métodos privados estáticos

- static boolean hasArc (int[ ][ ] distanceMatrix, int from, int to)

  Indica si existe un arco entre dos nodos distintos.

7.2.1. Descripción detallada

Funciones de utilidad: formato de texto tabular, ficheros, matrices y grafos.

Clase estática usada principalmente para imprimir tablas alineadas en columnas (los números reales se formatean con tres decimales y punto decimal).

Definición en la línea 24 del archivo ExpositoUtilities.java.

7.2.2. Documentación del constructor y destructor

7.2.2.1. ExpositoUtilities()

```
es.ull.esit.utilities.ExpositoUtilities.ExpositoUtilities ( )  [private]
```

Clase de utilidad: no instanciable.

Definición en la línea 36 del archivo ExpositoUtilities.java.

7.2.3. Documentación de las funciones miembro

7.2.3.1. printFile()

```
static void es.ull.esit.utilities.ExpositoUtilities.printFile (
            String file,
            PrintStream out ) throws IOException  [static]
```

Copia el contenido de un fichero de texto en un flujo de salida.

Parámetros

file	ruta del fichero.
out	flujo de salida.

Excepciones

IOException	si el fichero no se puede leer.

Definición en la línea 45 del archivo ExpositoUtilities.java.

7.2.3.2. simplifyString()

```
static String es.ull.esit.utilities.ExpositoUtilities.simplifyString (
            String string )  [static]
```

Normaliza los espacios de una cadena.

Sustituye tabuladores por espacios, colapsa espacios repetidos y recorta.

Parámetros

string	cadena original.

Devuelve

cadena con los campos separados por un único espacio.

Definición en la línea 60 del archivo ExpositoUtilities.java.

Referenciado por top.TOPTWReader.nextFields().

Gráfico de llamadas a esta función:



7.2.3.3. multiplyMatrices()

```
static double [][] es.ull.esit.utilities.ExpositoUtilities.multiplyMatrices (
            double a[][],
            double b[][] )  [static]
```

Producto de matrices.

Parámetros

a	matriz m x n.
b	matriz n x p.

Devuelve

matriz producto m x p (vacía si a está vacía).

Excepciones

IllegalArgumentException	si las dimensiones no son compatibles.

Definición en la línea 71 del archivo ExpositoUtilities.java.

7.2.3.4. writeTextToFile()

```
static void es.ull.esit.utilities.ExpositoUtilities.writeTextToFile (
```

```
            String file,
            String text ) throws IOException  [static]
```

Escribe un texto en un fichero, sobrescribiéndolo.

Parámetros

file	ruta del fichero.
text	texto a escribir.

Excepciones

IOException	si no se puede escribir.

Definición en la línea 99 del archivo ExpositoUtilities.java.

7.2.3.5. getFormat() [1/11]

```
static String es.ull.esit.utilities.ExpositoUtilities.getFormat (
            String string )  [static]
```

Formatea una cadena numérica: los reales no enteros pasan a tres decimales.

Parámetros

string	cadena a formatear.

Devuelve

cadena formateada (sin cambios si es entera o no numérica).

Definición en la línea 110 del archivo ExpositoUtilities.java.

Hace referencia a es.ull.esit.utilities.ExpositoUtilities.isDouble() y es.ull.esit.utilities.ExpositoUtilities.isInteger().

Referenciado por es.ull.esit.utilities.ExpositoUtilities.getFormat(), top.TOPTWSolution.getInfoSolution() y top.↩ TOPTW.toString().

Gráfico de llamadas para esta función:

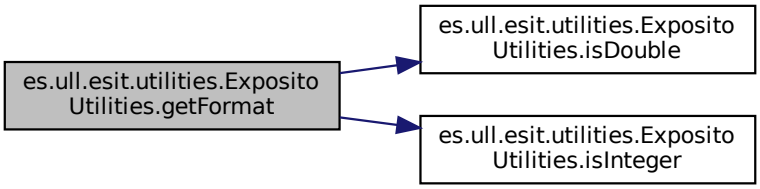
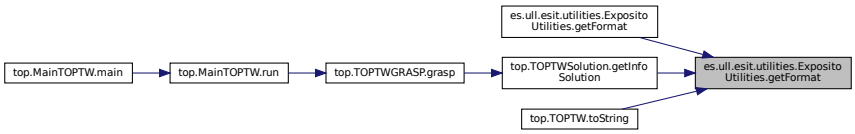


Gráfico de llamadas a esta función:



7.2.3.6. getFormat() [2/11]

```
static String es.ull.esit.utilities.ExpositoUtilities.getFormat (
            double value )  [static]
```

Formatea un real con tres decimales y punto decimal.

Parámetros

value	valor.

Devuelve

texto formateado.

Definición en la línea 122 del archivo ExpositoUtilities.java.

Hace referencia a es.ull.esit.utilities.ExpositoUtilities.getFormat().

Gráfico de llamadas para esta función:



7.2.3.7. getFormat() [3/11]

```
static String es.ull.esit.utilities.ExpositoUtilities.getFormat (
            double value,
            int zeros )  [static]
```

Formatea un real con el número de decimales indicado y punto decimal.

Parámetros

value	valor.
zeros	número de decimales.

Devuelve

texto formateado.

Definición en la línea 132 del archivo ExpositoUtilities.java.

7.2.3.8. getFormat() [4/11]

```
static String es.ull.esit.utilities.ExpositoUtilities.getFormat (
            String string,
            int width )  [static]
```

Alinea una cadena a la derecha en una columna.

Parámetros

string	cadena.
width	ancho de la columna.

Devuelve

cadena alineada.

Definición en la línea 150 del archivo ExpositoUtilities.java.

Hace referencia a es.ull.esit.utilities.ExpositoUtilities.ALIGNMENT_RIGHT y es.ull.esit.utilities.ExpositoUtilities.↩ getFormat().

Gráfico de llamadas para esta función:



7.2.3.9. getFormat() [5/11]

```
static String es.ull.esit.utilities.ExpositoUtilities.getFormat (
            String string,
            int width,
            int alignment )  [static]
```

Alinea una cadena en una columna.

Parámetros

string	cadena.
width	ancho de la columna.
alignment	ALIGNMENT_LEFT o ALIGNMENT_RIGHT.

Devuelve

cadena alineada.

Definición en la línea 161 del archivo ExpositoUtilities.java.

Hace referencia a es.ull.esit.utilities.ExpositoUtilities.ALIGNMENT_LEFT.

7.2.3.10. getFormat() [6/11]

```
static String es.ull.esit.utilities.ExpositoUtilities.getFormat (
            List< String > strings,
            int width )  [static]
```

Formatea una lista de cadenas en columnas del mismo ancho.

Parámetros

strings	cadenas.
width	ancho de cada columna.

Devuelve

fila formateada.

Definición en la línea 172 del archivo ExpositoUtilities.java.

Hace referencia a es.ull.esit.utilities.ExpositoUtilities.getFormat().

Gráfico de llamadas para esta función:



7.2.3.11. getFormatIntegers()

```
static String es.ull.esit.utilities.ExpositoUtilities.getFormatIntegers (
            List< Integer > values )  [static]
```

Formatea una lista de enteros en columnas de ancho DEFAULT_COLUMN_WIDTH.

Parámetros

values	enteros.

Devuelve

fila formateada.

Definición en la línea 181 del archivo ExpositoUtilities.java.

Hace referencia a es.ull.esit.utilities.ExpositoUtilities.DEFAULT_COLUMN_WIDTH.

7.2.3.12. getFormat() [7/11]

```
static String es.ull.esit.utilities.ExpositoUtilities.getFormat (
            String[] strings,
            int width )  [static]
```

Formatea un vector de cadenas en columnas del mismo ancho, alineadas a la derecha.

Parámetros

strings	cadenas.
width	ancho de cada columna.

Devuelve

fila formateada.

Definición en la línea 195 del archivo ExpositoUtilities.java.

Hace referencia a es.ull.esit.utilities.ExpositoUtilities.getFormat().

Gráfico de llamadas para esta función:



7.2.3.13. getFormat() [8/11]

```
static String es.ull.esit.utilities.ExpositoUtilities.getFormat (
            String matrixStrings[][],
            int width )  [static]
```

Formatea una matriz de cadenas: una fila por línea.

Parámetros

matrixStrings	matriz de cadenas.
width	ancho de cada columna.

Devuelve

texto formateado.

Definición en la línea 207 del archivo ExpositoUtilities.java.

Hace referencia a es.ull.esit.utilities.ExpositoUtilities.getFormat().

Gráfico de llamadas para esta función:



7.2.3.14. getFormat() [9/11]

```
static String es.ull.esit.utilities.ExpositoUtilities.getFormat (
            String[] strings )  [static]
```

Formatea un vector de cadenas con el ancho por defecto.

Parámetros

strings	cadenas.

Devuelve

fila formateada.

Definición en la línea 223 del archivo ExpositoUtilities.java.

Hace referencia a es.ull.esit.utilities.ExpositoUtilities.DEFAULT_COLUMN_WIDTH y es.ull.esit.utilities.Exposito↩ Utilities.getFormat().

Gráfico de llamadas para esta función:



7.2.3.15. getFormat() [10/11]

```
static String es.ull.esit.utilities.ExpositoUtilities.getFormat (
            String[] strings,
            int[] width )  [static]
```

Formatea un vector de cadenas con un ancho por columna, alineadas a la derecha.

Parámetros

strings	cadenas.
width	ancho de cada columna.

Devuelve

fila formateada.

Definición en la línea 233 del archivo ExpositoUtilities.java.

Hace referencia a es.ull.esit.utilities.ExpositoUtilities.ALIGNMENT_RIGHT y es.ull.esit.utilities.ExpositoUtilities.↩ getFormat().

Gráfico de llamadas para esta función:



7.2.3.16. getFormat() [11/11]

```
static String es.ull.esit.utilities.ExpositoUtilities.getFormat (
            String[] strings,
            int[] width,
            int[] alignment )  [static]
```

Formatea un vector de cadenas con ancho y alineación por columna.

Cada valor numérico real se escribe con tres decimales.

Parámetros

strings	cadenas.
width	ancho de cada columna.
alignment	alineación de cada columna.

Devuelve

fila formateada.

Definición en la línea 247 del archivo ExpositoUtilities.java.

Hace referencia a es.ull.esit.utilities.ExpositoUtilities.getFormat().

Gráfico de llamadas para esta función:



7.2.3.17. isInteger()

```
static boolean es.ull.esit.utilities.ExpositoUtilities.isInteger (
            String str )  [static]
```

Indica si una cadena representa un entero.

Parámetros

str	cadena.

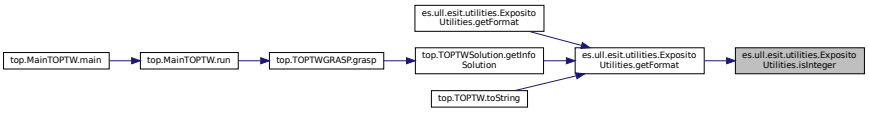
Devuelve

true si Integer.parseInt la acepta.

Definición en la línea 260 del archivo ExpositoUtilities.java.

Referenciado por es.ull.esit.utilities.ExpositoUtilities.getFormat().

Gráfico de llamadas a esta función:



7.2.3.18. isDouble()

```
static boolean es.ull.esit.utilities.ExpositoUtilities.isDouble (
            String str )  [static]
```

Indica si una cadena representa un real.

Parámetros

str	cadena.

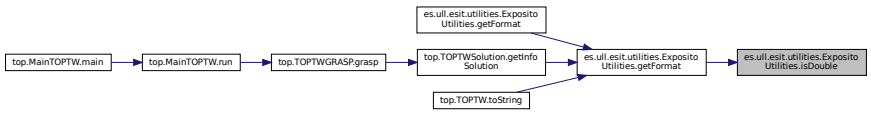
Devuelve

true si Double.parseDouble la acepta.

Definición en la línea 277 del archivo ExpositoUtilities.java.

Referenciado por es.ull.esit.utilities.ExpositoUtilities.getFormat().

Gráfico de llamadas a esta función:



7.2.3.19. isAcyclic()

```
static boolean es.ull.esit.utilities.ExpositoUtilities.isAcyclic (
            int distanceMatrix[][] )  [static]
```

Comprueba si un grafo de tareas es acíclico.

El grafo se da como matriz de distancias en la que Integer.MAX_VALUE indica ausencia de arco. Los nodos 0 y n-1 son el inicio y el fin ficticios, por lo que solo se comprueban los nodos 1..n-2.

Parámetros

distanceMatrix	matriz de adyacencia ponderada.

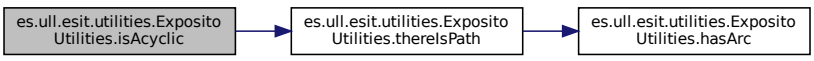
Devuelve

true si ninguna tarea real pertenece a un ciclo.

Definición en la línea 299 del archivo ExpositoUtilities.java.

Hace referencia a es.ull.esit.utilities.ExpositoUtilities.thereIsPath().

Gráfico de llamadas para esta función:



7.2.3.20. thereIsPath()

```
static boolean es.ull.esit.utilities.ExpositoUtilities.thereIsPath (
            int distanceMatrix[][],
            int node )  [static]
```

Comprueba si existe un camino de un nodo a sí mismo (ciclo).

Parámetros

distanceMatrix	matriz de adyacencia ponderada (Integer.MAX_VALUE = sin arco).
node	nodo de partida.

Devuelve

true si el nodo es alcanzable desde sí mismo.

Definición en la línea 315 del archivo ExpositoUtilities.java.

Hace referencia a es.ull.esit.utilities.ExpositoUtilities.hasArc().

Referenciado por es.ull.esit.utilities.ExpositoUtilities.isAcyclic().

Gráfico de llamadas para esta función:

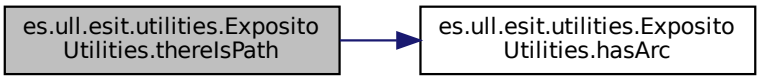
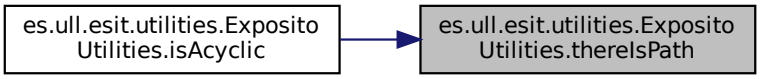


Gráfico de llamadas a esta función:



7.2.3.21. hasArc()

```
static boolean es.ull.esit.utilities.ExpositoUtilities.hasArc (
            int distanceMatrix[][],
            int from,
            int to )  [static], [private]
```

Indica si existe un arco entre dos nodos distintos.

Parámetros

distanceMatrix	matriz de adyacencia ponderada (Integer.MAX_VALUE = sin arco).
from	nodo origen.
to	nodo destino.

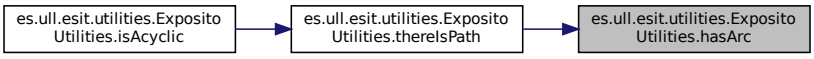
Devuelve

true si from y to son distintos y hay arco entre ellos.

Definición en la línea 350 del archivo ExpositoUtilities.java.

Referenciado por es.ull.esit.utilities.ExpositoUtilities.thereIsPath().

Gráfico de llamadas a esta función:



7.2.4. Documentación de los datos miembro

7.2.4.1. DEFAULT_COLUMN_WIDTH

```
final int es.ull.esit.utilities.ExpositoUtilities.DEFAULT_COLUMN_WIDTH = 10  [static]
```

Ancho de columna por defecto.

Definición en la línea 27 del archivo ExpositoUtilities.java.

Referenciado por es.ull.esit.utilities.ExpositoUtilities.getFormat() y es.ull.esit.utilities.ExpositoUtilities.getFormat↩ Integers().

7.2.4.2. ALIGNMENT_LEFT

```
final int es.ull.esit.utilities.ExpositoUtilities.ALIGNMENT_LEFT = 1  [static]
```

Alineación a la izquierda.

Definición en la línea 29 del archivo ExpositoUtilities.java.

Referenciado por es.ull.esit.utilities.ExpositoUtilities.getFormat().

7.2.4.3. ALIGNMENT_RIGHT

```
final int es.ull.esit.utilities.ExpositoUtilities.ALIGNMENT_RIGHT = 2  [static]
```

Alineación a la derecha.

Definición en la línea 31 del archivo ExpositoUtilities.java.

Referenciado por es.ull.esit.utilities.ExpositoUtilities.getFormat().

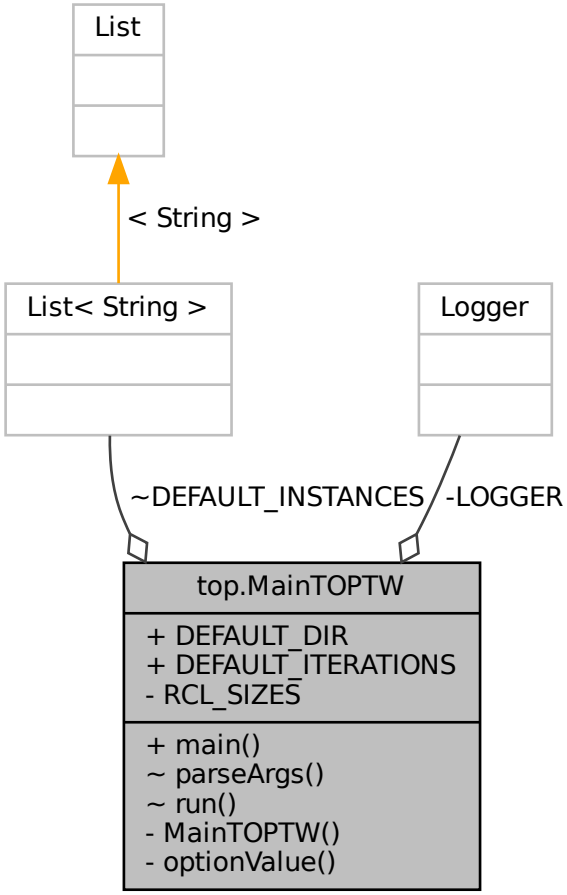
La documentación para esta clase fue generada a partir del siguiente fichero:

- es/ull/esit/utilities/ExpositoUtilities.java

7.3. Referencia de la Clase top.MainTOPTW

Punto de entrada del recomendador turístico RTVRPTW / TOPTW.

Diagrama de colaboración para top.MainTOPTW:



Clases

- class Options

  Opciones de ejecución obtenidas de la línea de órdenes.

Métodos públicos estáticos

- static void main (String[ ] args)

  Programa principal.

Atributos públicos estáticos

- static final String DEFAULT_DIR = "Instances/TOPTW"

  Directorio de instancias por defecto.
- static final int DEFAULT_ITERATIONS = 10000

  Número de iteraciones por defecto del experimento original.

Funciones estáticas del 'package'

- static Options parseArgs (String[ ] args)

  Interpreta los argumentos de la línea de órdenes.
- static void run (Options options, PrintStream out) throws IOException

  Ejecuta el experimento para las opciones dadas.

Atributos Estáticos del 'package'

- static final List< String > DEFAULT_INSTANCES

  Instancias del experimento original.

Métodos privados

- MainTOPTW ()

  Clase de arranque: no instanciable.

Métodos privados estáticos

- static String optionValue (Iterator< String > it, String option)

  Obtiene el valor que sigue a una opción.

Atributos privados estáticos

- static final int[ ] RCL_SIZES = {3, 5, 7}

  Tamaños de RCL evaluados.
- static final Logger LOGGER = Logger.getLogger(MainTOPTW.class.getName())

  Registro de errores.

7.3.1. Descripción detallada

Punto de entrada del recomendador turístico RTVRPTW / TOPTW.

Ejecuta el GRASP sobre un conjunto de instancias con tres tamaños de lista restringida de candidatos (3, 5 y 7) y escribe en la salida estándar el informe de cada solución construida, la media y la mejor puntuación.

Uso:
```
java -jar ExpositoTOP.jar [--dir DIR] [--iterations N] [--seed S] [instancia ...]
```

Sin argumentos se reproduce el experimento original: las 29 instancias de Solomon (c1, r1, rc1) del directorio Instances/TOPTW con 10000 iteraciones.

Definición en la línea 28 del archivo MainTOPTW.java.

7.3.2. Documentación del constructor y destructor

7.3.2.1. MainTOPTW()

```
top.MainTOPTW.MainTOPTW ( )  [private]
```

Clase de arranque: no instanciable.

Definición en la línea 50 del archivo MainTOPTW.java.

7.3.3. Documentación de las funciones miembro

7.3.3.1. parseArgs()

```
static Options top.MainTOPTW.parseArgs (
            String[] args )  [static], [package]
```

Interpreta los argumentos de la línea de órdenes.

Parámetros

args	argumentos.

Devuelve

opciones de ejecución.

Excepciones

IllegalArgumentException	si falta el valor de una opción o no es numérico.

Definición en la línea 73 del archivo MainTOPTW.java.

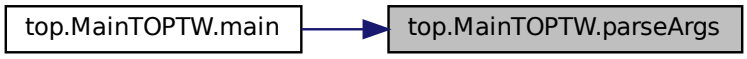
Hace referencia a top.MainTOPTW.DEFAULT_INSTANCES, top.MainTOPTW.Options.dir, top.MainTOPTW.↩ Options.instances, top.MainTOPTW.Options.iterations, top.MainTOPTW.optionValue() y top.MainTOPTW.↩ Options.seed.

Referenciado por top.MainTOPTW.main().

Gráfico de llamadas para esta función:



Gráfico de llamadas a esta función:



7.3.3.2. optionValue()

```
static String top.MainTOPTW.optionValue (
            Iterator< String > it,
            String option )  [static], [private]
```

Obtiene el valor que sigue a una opción.

Parámetros

it	iterador sobre los argumentos, situado tras la opción.
option	nombre de la opción (para el mensaje de error).

Devuelve

valor de la opción.

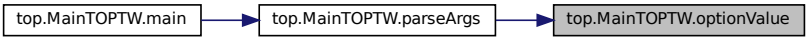
Excepciones

IllegalArgumentException	si la opción no tiene valor.

Definición en la línea 105 del archivo MainTOPTW.java.

Referenciado por top.MainTOPTW.parseArgs().

Gráfico de llamadas a esta función:



7.3.3.3. run()

```
static void top.MainTOPTW.run (
            Options options,
            PrintStream out ) throws IOException  [static], [package]
```

Ejecuta el experimento para las opciones dadas.

Parámetros

options	opciones de ejecución.
out	flujo de salida de los resultados.

Excepciones

IOException	si alguna instancia no se puede leer.

Definición en la línea 118 del archivo MainTOPTW.java.

Hace referencia a top.TOPTWGRASP.grasp(), top.MainTOPTW.RCL_SIZES y top.TOPTWReader.readProblem().

Referenciado por top.MainTOPTW.main().

Gráfico de llamadas para esta función:

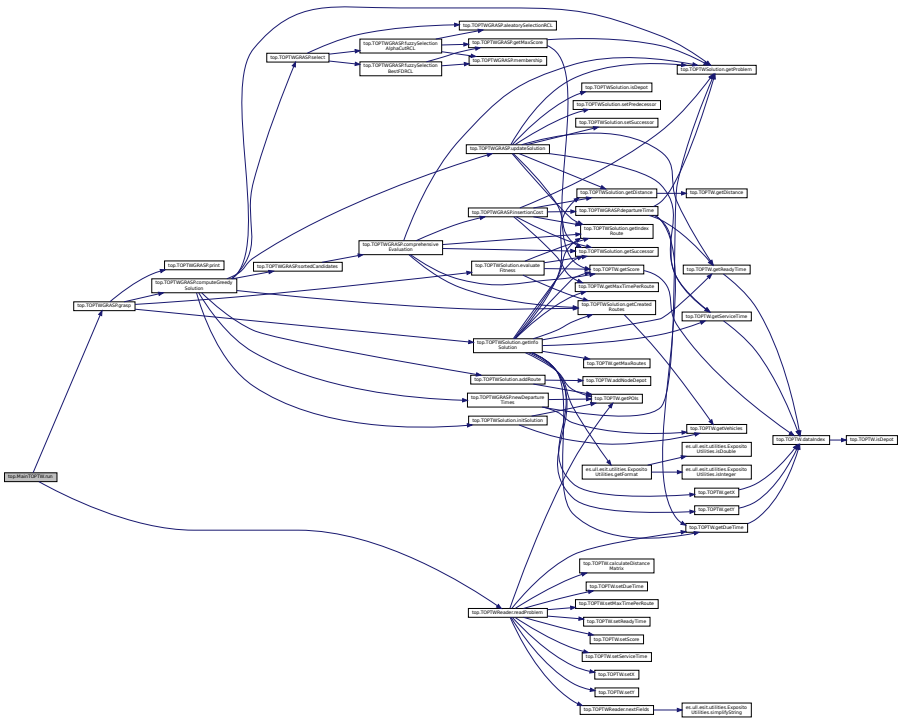


Gráfico de llamadas a esta función:



7.3.3.4. main()

```
static void top.MainTOPTW.main (
            String[] args )  [static]
```

Programa principal.

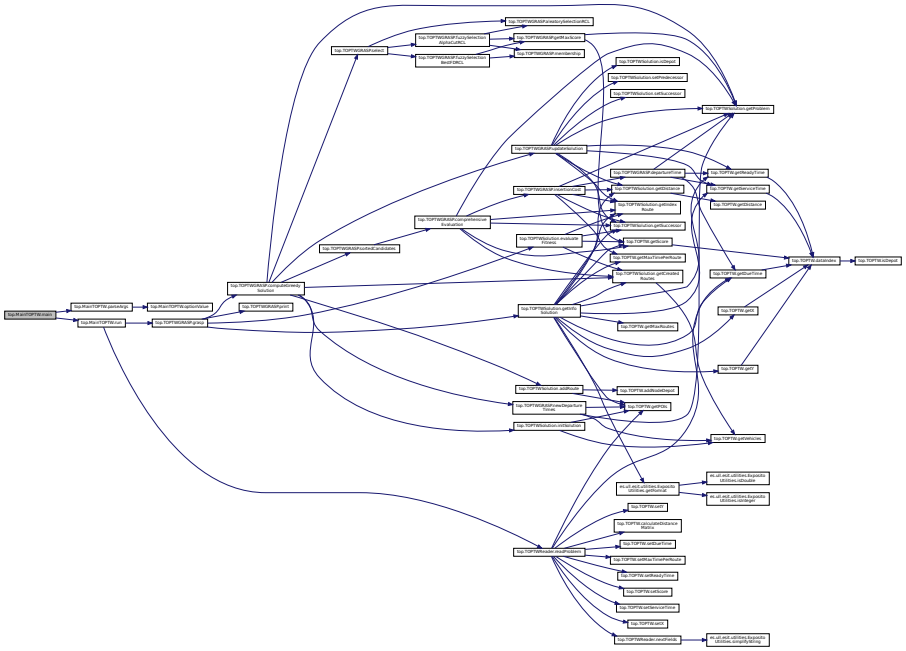
Parámetros

args	[–dir DIR] [–iterations N] [–seed S] [instancia ...]

Definición en la línea 135 del archivo MainTOPTW.java.

Hace referencia a top.MainTOPTW.LOGGER, top.MainTOPTW.parseArgs() y top.MainTOPTW.run().

Gráfico de llamadas para esta función:



7.3.4. Documentación de los datos miembro

7.3.4.1. DEFAULT_DIR

```
final String top.MainTOPTW.DEFAULT_DIR = "Instances/TOPTW"  [static]
```

Directorio de instancias por defecto.

Definición en la línea 31 del archivo MainTOPTW.java.

7.3.4.2. DEFAULT_ITERATIONS

```
final int top.MainTOPTW.DEFAULT_ITERATIONS = 10000  [static]
```

Número de iteraciones por defecto del experimento original.

Definición en la línea 33 del archivo MainTOPTW.java.

7.3.4.3. RCL_SIZES

```
final int [] top.MainTOPTW.RCL_SIZES = {3, 5, 7}  [static], [private]
```

Tamaños de RCL evaluados.

Definición en la línea 35 del archivo MainTOPTW.java.

Referenciado por top.MainTOPTW.run().

7.3.4.4. DEFAULT_INSTANCES

```
final List<String> top.MainTOPTW.DEFAULT_INSTANCES  [static], [package]
```

Valor inicial:

```
= List.of(
            "c101.txt", "c102.txt", "c103.txt", "c104.txt", "c105.txt", "c106.txt", "c107.txt", "c108.txt",
            "c109.txt",
            "r101.txt", "r102.txt", "r103.txt", "r104.txt", "r105.txt", "r106.txt", "r107.txt", "r108.txt",
            "r109.txt", "r110.txt", "r111.txt", "r112.txt",
            "rc101.txt", "rc102.txt", "rc103.txt", "rc104.txt", "rc105.txt", "rc106.txt", "rc107.txt",
            "rc108.txt")
```

Instancias del experimento original.

Definición en la línea 37 del archivo MainTOPTW.java.

Referenciado por top.MainTOPTW.parseArgs().

7.3.4.5. LOGGER

```
final Logger top.MainTOPTW.LOGGER = Logger.getLogger(MainTOPTW.class.getName())  [static],
[private]
```

Registro de errores.

Definición en la línea 45 del archivo MainTOPTW.java.

Referenciado por top.MainTOPTW.main().

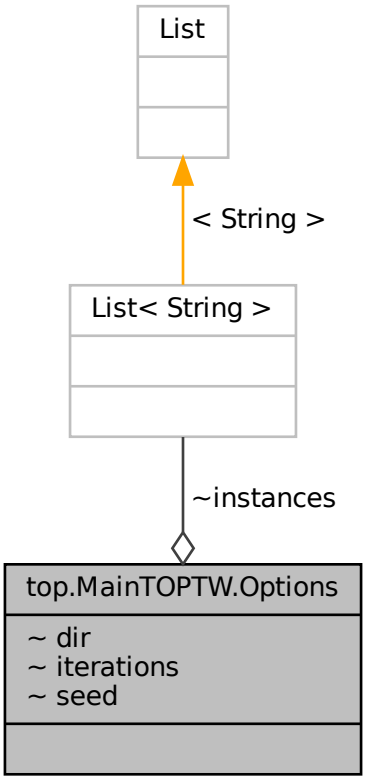
La documentación para esta clase fue generada a partir del siguiente fichero:

- top/MainTOPTW.java

7.4. Referencia de la Clase top.MainTOPTW.Options

Opciones de ejecución obtenidas de la línea de órdenes.

Diagrama de colaboración para top.MainTOPTW.Options:



Atributos del 'package'

- String dir = DEFAULT_DIR

  Directorio de instancias.
- int iterations = DEFAULT_ITERATIONS

  Iteraciones del GRASP por tamaño de RCL.
- Long seed

  Semilla aleatoria, o null para no fijarla.
- List< String > instances = new ArrayList<>()

  Instancias a resolver.

7.4.1. Descripción detallada

Opciones de ejecución obtenidas de la línea de órdenes.

Definición en la línea 56 del archivo MainTOPTW.java.

7.4.2. Documentación de los datos miembro

7.4.2.1. dir

```
String top.MainTOPTW.Options.dir = DEFAULT_DIR  [package]
```

Directorio de instancias.

Definición en la línea 58 del archivo MainTOPTW.java.

Referenciado por top.MainTOPTW.parseArgs().

7.4.2.2. iterations

```
int top.MainTOPTW.Options.iterations = DEFAULT_ITERATIONS  [package]
```

Iteraciones del GRASP por tamaño de RCL.

Definición en la línea 60 del archivo MainTOPTW.java.

Referenciado por top.MainTOPTW.parseArgs().

7.4.2.3. seed

```
Long top.MainTOPTW.Options.seed  [package]
```

Semilla aleatoria, o null para no fijarla.

Definición en la línea 62 del archivo MainTOPTW.java.

Referenciado por top.MainTOPTW.parseArgs().

7.4.2.4. instances

```
List<String> top.MainTOPTW.Options.instances = new ArrayList<>()  [package]
```

Instancias a resolver.

Definición en la línea 64 del archivo MainTOPTW.java.

Referenciado por top.MainTOPTW.parseArgs().

La documentación para esta clase fue generada a partir del siguiente fichero:

- top/MainTOPTW.java

7.5. Referencia de la plantilla de la Clase es.ull.esit.utilities.PowerSet< E >

Conjunto potencia (todos los subconjuntos) de un conjunto, recorrible con for-each.

Diagrama de herencias de es.ull.esit.utilities.PowerSet< E >

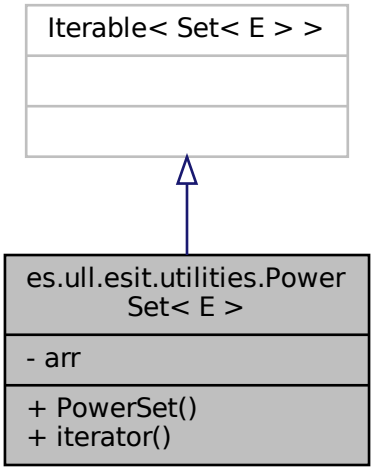
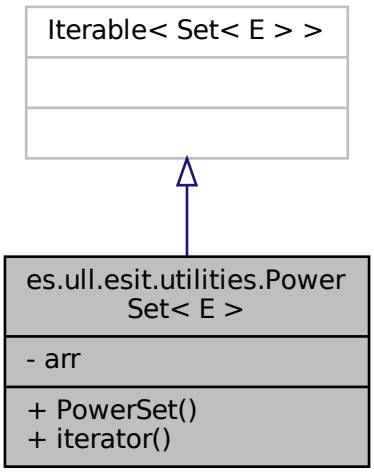


Diagrama de colaboración para es.ull.esit.utilities.PowerSet< E >:



Clases

- class PowerSetIterator

  Iterador sobre los subconjuntos, basado en un contador binario.

Métodos públicos

- PowerSet (Set< E > set)

  Crea el conjunto potencia de un conjunto.
- Iterator< Set< E > > iterator ()

  Devuelve un iterador nuevo sobre los 2^n subconjuntos.

Atributos privados

- final Object[ ] arr

  Elementos del conjunto original.

7.5.1. Descripción detallada

Conjunto potencia (todos los subconjuntos) de un conjunto, recorrible con for-each.

Cada llamada a iterator() devuelve un iterador nuevo e independiente, por lo que el conjunto potencia puede recorrerse tantas veces como se quiera. El iterador usa un BitSet como contador binario: el bit i indica si el elemento i pertenece al subconjunto actual. Genera 2^n subconjuntos empezando por el vacío. Los elementos deben ser comparables, ya que cada subconjunto se devuelve como TreeSet.

Parámetros

<E>	tipo de los elementos.

Definición en la línea 20 del archivo PowerSet.java.

7.5.2. Documentación del constructor y destructor

7.5.2.1. PowerSet()

```
es.ull.esit.utilities.PowerSet< E >.PowerSet (
            Set< E > set )
```

Crea el conjunto potencia de un conjunto.

Parámetros

set	conjunto original.

Definición en la línea 29 del archivo PowerSet.java.

7.5.3. Documentación de las funciones miembro

7.5.3.1. iterator()

```
Iterator<Set<E> > es.ull.esit.utilities.PowerSet< E >.iterator ( )
```

Devuelve un iterador nuevo sobre los 2^n subconjuntos.

Devuelve

iterador independiente de los anteriores.

Definición en la línea 38 del archivo PowerSet.java.

7.5.4. Documentación de los datos miembro

7.5.4.1. arr

```
final Object [] es.ull.esit.utilities.PowerSet< E >.arr  [private]
```

Elementos del conjunto original.

Definición en la línea 23 del archivo PowerSet.java.

Referenciado por es.ull.esit.utilities.PowerSet< E >.PowerSetIterator.hasNext() y es.ull.esit.utilities.PowerSet< E >.PowerSetIterator.next().

La documentación para esta clase fue generada a partir del siguiente fichero:

- es/ull/esit/utilities/PowerSet.java

7.6. Referencia de la Clase es.ull.esit.utilities.PowerSet< E >.PowerSetIterator

Iterador sobre los subconjuntos, basado en un contador binario.

Diagrama de herencias de es.ull.esit.utilities.PowerSet< E >.PowerSetIterator

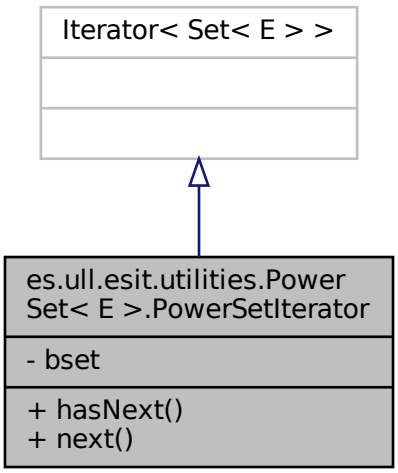
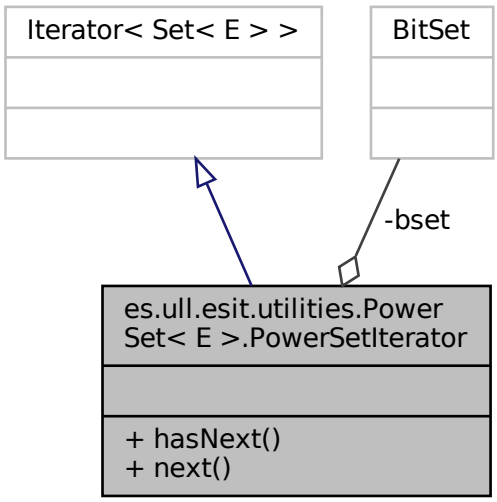


Diagrama de colaboración para es.ull.esit.utilities.PowerSet< E >.PowerSetIterator:



Métodos públicos

- boolean hasNext ()

  Indica si quedan subconjuntos.
- Set< E > next ()

  Devuelve el siguiente subconjunto.

Atributos privados

- final BitSet bset = new BitSet(arr.length + 1)

  Contador binario del subconjunto actual; el bit n marca el final.

7.6.1. Descripción detallada

Iterador sobre los subconjuntos, basado en un contador binario.

Definición en la línea 45 del archivo PowerSet.java.

7.6.2. Documentación de las funciones miembro

7.6.2.1. hasNext()

`boolean es.ull.esit.utilities.PowerSet< E >.PowerSetIterator.hasNext ( )`

Indica si quedan subconjuntos.

Devuelve

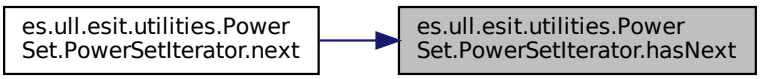
true mientras no se hayan generado los 2^n subconjuntos.

Definición en la línea 54 del archivo PowerSet.java.

Hace referencia a es.ull.esit.utilities.PowerSet< E >.arr.

Referenciado por es.ull.esit.utilities.PowerSet< E >.PowerSetIterator.next().

Gráfico de llamadas a esta función:



7.6.2.2. next()

`Set<E> es.ull.esit.utilities.PowerSet< E >.PowerSetIterator.next ( )`

Devuelve el siguiente subconjunto.

Devuelve

subconjunto actual.

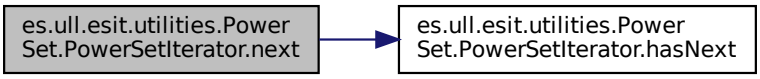
Excepciones

NoSuchElementException	si ya se han generado todos.

Definición en la línea 65 del archivo PowerSet.java.

Hace referencia a es.ull.esit.utilities.PowerSet< E >.arr y es.ull.esit.utilities.PowerSet< E >.PowerSetIterator.↩ hasNext().

Gráfico de llamadas para esta función:



7.6.3. Documentación de los datos miembro

7.6.3.1. bset

```
final BitSet es.ull.esit.utilities.PowerSet< E >.PowerSetIterator.bset = new BitSet(arr.length
+ 1)  [private]
```

Contador binario del subconjunto actual; el bit n marca el final.

Definición en la línea 47 del archivo PowerSet.java.

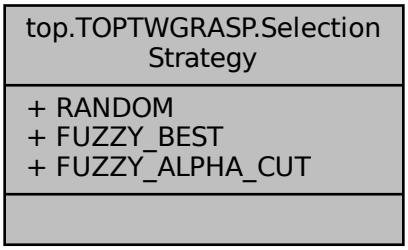
La documentación para esta clase fue generada a partir del siguiente fichero:

- es/ull/esit/utilities/PowerSet.java

7.7. Referencia del enum top.TOPTWGRASP.SelectionStrategy

Estrategias de selección de un candidato dentro de la RCL.

Diagrama de colaboración para top.TOPTWGRASP.SelectionStrategy:



Atributos públicos

- RANDOM

  Selección uniforme al azar.
- FUZZY_BEST

  Selección del candidato con mejor grado de pertenencia borroso.
- FUZZY_ALPHA_CUT

  Selección al azar entre los candidatos que superan un corte alfa borroso.

7.7.1. Descripción detallada

Estrategias de selección de un candidato dentro de la RCL.

Definición en la línea 38 del archivo TOPTWGRASP.java.

7.7.2. Documentación de los datos miembro

7.7.2.1. RANDOM

`top.TOPTWGRASP.SelectionStrategy.RANDOM`

Selección uniforme al azar.

Definición en la línea 40 del archivo TOPTWGRASP.java.

7.7.2.2. FUZZY_BEST

`top.TOPTWGRASP.SelectionStrategy.FUZZY_BEST`

Selección del candidato con mejor grado de pertenencia borroso.

Definición en la línea 42 del archivo TOPTWGRASP.java.

7.7.2.3. FUZZY_ALPHA_CUT

`top.TOPTWGRASP.SelectionStrategy.FUZZY_ALPHA_CUT`

Selección al azar entre los candidatos que superan un corte alfa borroso.

Definición en la línea 45 del archivo TOPTWGRASP.java.

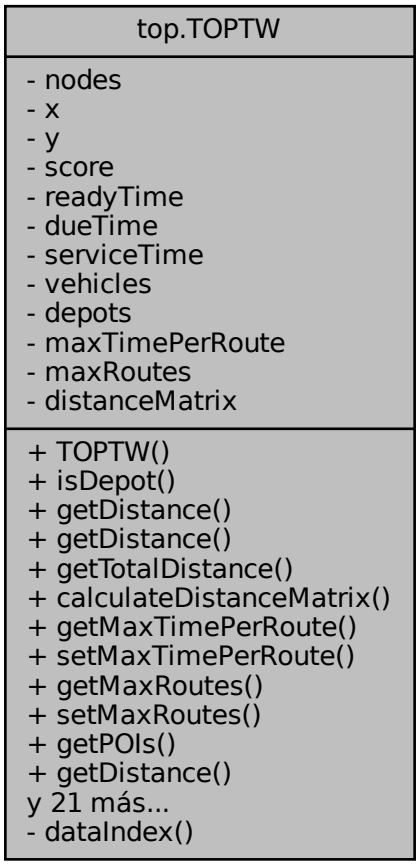
La documentación para este enum ha sido generada a partir del siguiente fichero:

- top/TOPTWGRASP.java

7.8. Referencia de la Clase top.TOPTW

Modelo de una instancia del problema TOPTW (Team Orienteering Problem with Time Windows).

Diagrama de colaboración para top.TOPTW:



Métodos públicos

- TOPTW (int nodes, int routes)

  Crea una instancia vacía con el número de POI y rutas indicados.
- boolean isDepot (int a)

  Indica si un índice corresponde a un depósito virtual.
- double getDistance (int[ ] route)

  Calcula la longitud total de una ruta dada como vector de nodos.
- double getDistance (List< Integer > route)

  Calcula la longitud total de una ruta dada como lista de nodos.
- double getTotalDistance (List< List< Integer >> routes)

  Calcula la longitud acumulada de un conjunto de rutas.
- void calculateDistanceMatrix ()

Rellena la matriz de distancias euclídeas a partir de las coordenadas.

- double getMaxTimePerRoute ()

  Devuelve la duración máxima de cada ruta.
- void setMaxTimePerRoute (double maxTimePerRoute)

  Establece la duración máxima de cada ruta.
- double getMaxRoutes ()

  Devuelve el número máximo de rutas.
- void setMaxRoutes (double maxRoutes)

  Establece el número máximo de rutas.
- int getPOIs ()

  Devuelve el número de puntos de interés (sin el depósito).
- double getDistance (int i, int j)

  Distancia entre dos nodos.
- double getTime (int i, int j)

  Tiempo de viaje entre dos nodos.
- int getNodes ()

  Devuelve el número de POI.
- void setNodes (int nodes)

  Modifica el número de POI.
- double getX (int index)

  Coordenada X de un nodo.
- void setX (int index, double x)

  Establece la coordenada X de un nodo.
- double getY (int index)

  Coordenada Y de un nodo.
- void setY (int index, double y)

  Establece la coordenada Y de un nodo.
- double getScore (int index)

  Puntuación de un nodo.
- double[ ] getScore ()

  Copia del vector de puntuaciones.
- void setScore (int index, double score)

  Establece la puntuación de un nodo.
- double getReadyTime (int index)

  Inicio de la ventana temporal de un nodo.
- void setReadyTime (int index, double readyTime)

  Establece el inicio de la ventana temporal de un nodo.
- double getDueTime (int index)

  Fin de la ventana temporal de un nodo.
- void setDueTime (int index, double dueTime)

  Establece el fin de la ventana temporal de un nodo.
- double getServiceTime (int index)

  Tiempo de servicio (duración de la visita) de un nodo.
- void setServiceTime (int index, double serviceTime)

  Establece el tiempo de servicio de un nodo.
- int getVehicles ()

  Número de vehículos (rutas) disponibles.
- int getDepots ()

  Número de depósitos virtuales registrados.
- String toString ()

  Representación tabular de la instancia.

- int addNode ()

  Añade un nuevo POI incrementando el contador de nodos.
- int addNodeDepot ()

  Registra un nuevo depósito virtual.

Métodos privados

- int dataIndex (int index)

  Traduce un índice de nodo al índice real en los vectores de datos.

Atributos privados

- int nodes

  Número de POI (sin contar el depósito).
- final double[ ] x

  Coordenada X de cada nodo (índice 0 = depósito).
- final double[ ] y

  Coordenada Y de cada nodo (índice 0 = depósito).
- final double[ ] score

  Puntuación o beneficio obtenido al visitar cada nodo.
- final double[ ] readyTime

  Instante más temprano en que puede empezar la visita de cada nodo.
- final double[ ] dueTime

  Instante más tardío en que puede empezar la visita de cada nodo.
- final double[ ] serviceTime

  Duración de la visita de cada nodo.
- final int vehicles

  Número de vehículos (rutas / días de visita) disponibles.
- int depots

  Número de depósitos virtuales añadidos por las soluciones.
- double maxTimePerRoute

  Duración máxima permitida para cada ruta.
- double maxRoutes

  Número máximo de rutas permitidas.
- final double[ ][ ] distanceMatrix

  Matriz simétrica de distancias euclídeas entre nodos.

7.8.1. Descripción detallada

Modelo de una instancia del problema TOPTW (Team Orienteering Problem with Time Windows).

Representa el conjunto de puntos de interés (POI) que un recomendador turístico puede incluir en las rutas de los visitantes. Cada POI tiene coordenadas, una puntuación (beneficio o interés turístico), una ventana temporal [readyTime, dueTime] en la que puede comenzar la visita y un tiempo de servicio (duración de la visita).

El índice 0 corresponde al depósito (punto de salida y llegada de todas las rutas, típicamente el hotel). Los índices mayores que el número de POI se reservan para los depósitos "virtuales" que la solución crea al abrir rutas adicionales; todos ellos se tratan como el depósito real (índice 0) al consultar datos.

La matriz de distancias es euclídea y simétrica; el tiempo de viaje entre dos nodos se considera igual a su distancia.

Definición en la línea 24 del archivo TOPTW.java.

7.8.2. Documentación del constructor y destructor

7.8.2.1. TOPTW()

```
top.TOPTW.TOPTW (
            int nodes,
            int routes )
```

Crea una instancia vacía con el número de POI y rutas indicados.

Reserva espacio para nodes + 1 nodos (los POI más el depósito) e inicializa la matriz de distancias a cero. Las coordenadas y demás datos deben cargarse después mediante los métodos set (normalmente desde TOPTWReader).

Parámetros

nodes	número de puntos de interés, sin contar el depósito.
routes	número de rutas (vehículos) disponibles.

Definición en la línea 60 del archivo TOPTW.java.

Hace referencia a top.TOPTW.nodes.

7.8.3. Documentación de las funciones miembro

7.8.3.1. isDepot()

```
boolean top.TOPTW.isDepot (
            int a )
```

Indica si un índice corresponde a un depósito virtual.

Parámetros

a	índice de nodo.

Devuelve

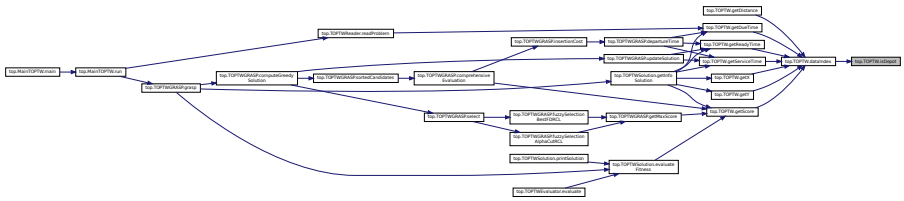
true si el índice es mayor que el número de POI (depósito virtual).

Definición en la línea 80 del archivo TOPTW.java.

Hace referencia a top.TOPTW.nodes.

Referenciado por top.TOPTW.dataIndex().

Gráfico de llamadas a esta función:



7.8.3.2. dataIndex()

```
int top.TOPTW.dataIndex (
            int index )  [private]
```

Traduce un índice de nodo al índice real en los vectores de datos.

Los depósitos virtuales comparten los datos del depósito real (índice 0).

Parámetros

index	índice de nodo, posiblemente de un depósito virtual.

Devuelve

el propio índice, o 0 si es un depósito virtual.

Definición en la línea 92 del archivo TOPTW.java.

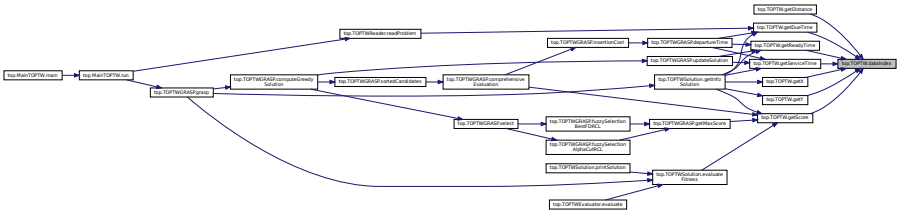
Hace referencia a top.TOPTW.isDepot().

Referenciado por top.TOPTW.getDistance(), top.TOPTW.getDueTime(), top.TOPTW.getReadyTime(), top.↩ TOPTW.getScore(), top.TOPTW.getServiceTime(), top.TOPTW.getX() y top.TOPTW.getY().

Gráfico de llamadas para esta función:



Gráfico de llamadas a esta función:



7.8.3.3. getDistance() [1/3]

```
double top.TOPTW.getDistance (
            int[] route )
```

Calcula la longitud total de una ruta dada como vector de nodos.

Parámetros

route	secuencia de nodos visitados.

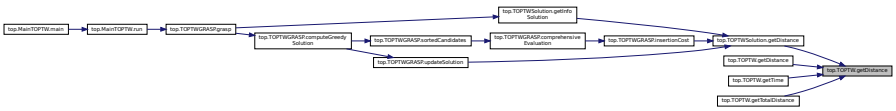
Devuelve

suma de las distancias entre nodos consecutivos.

Definición en la línea 102 del archivo TOPTW.java.

Referenciado por top.TOPTWSolution.getDistance(), top.TOPTW.getDistance(), top.TOPTW.getTime() y top.↩ TOPTW.getTotalDistance().

Gráfico de llamadas a esta función:



7.8.3.4. getDistance() [2/3]

```
double top.TOPTW.getDistance (
            List< Integer > route )
```

Calcula la longitud total de una ruta dada como lista de nodos.

Parámetros

route	secuencia de nodos visitados.

Devuelve

suma de las distancias entre nodos consecutivos.

Definición en la línea 116 del archivo TOPTW.java.

Hace referencia a top.TOPTW.getDistance().

Gráfico de llamadas para esta función:



7.8.3.5. getTotalDistance()

```
double top.TOPTW.getTotalDistance (
            List< List< Integer >> routes )
```

Calcula la longitud acumulada de un conjunto de rutas.

Parámetros

routes	rutas a evaluar.

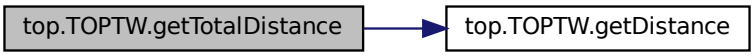
Devuelve

suma de las longitudes de todas las rutas.

Definición en la línea 130 del archivo TOPTW.java.

Hace referencia a top.TOPTW.getDistance().

Gráfico de llamadas para esta función:



7.8.3.6. calculateDistanceMatrix()

```
void top.TOPTW.calculateDistanceMatrix ( )
```

Rellena la matriz de distancias euclídeas a partir de las coordenadas.

Debe invocarse una vez cargadas las coordenadas de todos los nodos.

Definición en la línea 143 del archivo TOPTW.java.

Referenciado por top.TOPTWReader.readProblem().

Gráfico de llamadas a esta función:



7.8.3.7. getMaxTimePerRoute()

```
double top.TOPTW.getMaxTimePerRoute ( )
```

Devuelve la duración máxima de cada ruta.

Devuelve

tiempo máximo por ruta.

Definición en la línea 162 del archivo TOPTW.java.

Hace referencia a top.TOPTW.maxTimePerRoute.

Referenciado por top.TOPTWSolution.getInfoSolution() y top.TOPTWGRASP.insertionCost().

Gráfico de llamadas a esta función:



7.8.3.8. setMaxTimePerRoute()

```
void top.TOPTW.setMaxTimePerRoute (
            double maxTimePerRoute )
```

Establece la duración máxima de cada ruta.

Parámetros

maxTimePerRoute	tiempo máximo por ruta.

Definición en la línea 170 del archivo TOPTW.java.

Hace referencia a top.TOPTW.maxTimePerRoute.

Referenciado por top.TOPTWReader.readProblem().

Gráfico de llamadas a esta función:



7.8.3.9. getMaxRoutes()

```
double top.TOPTW.getMaxRoutes ( )
```

Devuelve el número máximo de rutas.

Devuelve

número máximo de rutas.

Definición en la línea 178 del archivo TOPTW.java.

Hace referencia a top.TOPTW.maxRoutes.

Referenciado por top.TOPTWSolution.getInfoSolution().

Gráfico de llamadas a esta función:



7.8.3.10. setMaxRoutes()

```
void top.TOPTW.setMaxRoutes (
            double maxRoutes )
```

Establece el número máximo de rutas.

Parámetros

maxRoutes	número máximo de rutas.

Definición en la línea 186 del archivo TOPTW.java.

Hace referencia a top.TOPTW.maxRoutes.

7.8.3.11. getPOIs()

```
int top.TOPTW.getPOIs ( )
```

Devuelve el número de puntos de interés (sin el depósito).

Devuelve

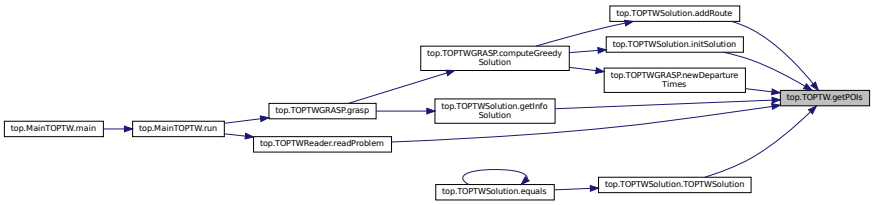
número de POI.

Definición en la línea 194 del archivo TOPTW.java.

Hace referencia a top.TOPTW.nodes.

Referenciado por top.TOPTWSolution.addRoute(), top.TOPTWSolution.getInfoSolution(), top.TOPTWSolution.↩ initSolution(), top.TOPTWGRASP.newDepartureTimes(), top.TOPTWReader.readProblem() y top.TOPTWSolution.↩ TOPTWSolution().

Gráfico de llamadas a esta función:



7.8.3.12. getDistance() [3/3]

```
double top.TOPTW.getDistance (
            int i,
            int j )
```

Distancia entre dos nodos.

Parámetros

i	nodo origen (los depósitos virtuales se tratan como el 0).
j	nodo destino (los depósitos virtuales se tratan como el 0).

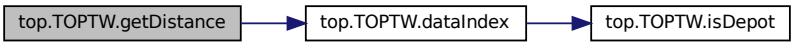
Devuelve

distancia euclídea entre ambos.

Definición en la línea 205 del archivo TOPTW.java.

Hace referencia a top.TOPTW.dataIndex().

Gráfico de llamadas para esta función:



7.8.3.13. getTime()

```
double top.TOPTW.getTime (
            int i,
            int j )
```

Tiempo de viaje entre dos nodos.

En este modelo el tiempo de viaje coincide con la distancia.

Parámetros

i	nodo origen.
j	nodo destino.

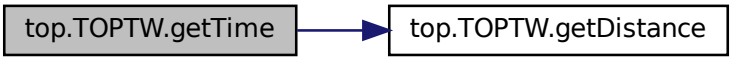
Devuelve

tiempo de viaje entre ambos.

Definición en la línea 218 del archivo TOPTW.java.

Hace referencia a top.TOPTW.getDistance().

Gráfico de llamadas para esta función:



7.8.3.14. getNodes()

```
int top.TOPTW.getNodes ( )
```

Devuelve el número de POI.

Devuelve

número de POI.

Definición en la línea 226 del archivo TOPTW.java.

Hace referencia a top.TOPTW.nodes.

7.8.3.15. setNodes()

```
void top.TOPTW.setNodes (
             int nodes )
```

Modifica el número de POI.

Parámetros

nodes	nuevo número de POI.

Definición en la línea 234 del archivo TOPTW.java.

Hace referencia a top.TOPTW.nodes.

7.8.3.16. getX()

```
double top.TOPTW.getX (
             int index )
```

Coordenada X de un nodo.

Parámetros

index	índice del nodo.

Devuelve

coordenada X.

Definición en la línea 243 del archivo TOPTW.java.

Hace referencia a top.TOPTW.dataIndex().

Referenciado por top.TOPTWSolution.getInfoSolution().

Gráfico de llamadas para esta función:

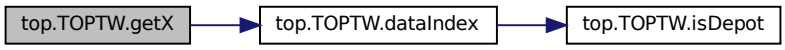
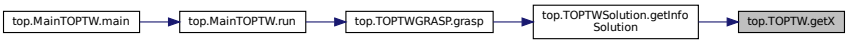


Gráfico de llamadas a esta función:



7.8.3.17. setX()

```
void top.TOPTW.setX (
            int index,
            double x )
```

Establece la coordenada X de un nodo.

Parámetros

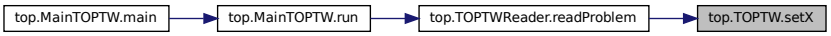
index	índice del nodo.
x	coordenada X.

Definición en la línea 252 del archivo TOPTW.java.

Hace referencia a top.TOPTW.x.

Referenciado por top.TOPTWReader.readProblem().

Gráfico de llamadas a esta función:



7.8.3.18. getY()

```
double top.TOPTW.getY (
            int index )
```

Coordenada Y de un nodo.

Parámetros

index	índice del nodo.

Devuelve

coordenada Y.

Definición en la línea 261 del archivo TOPTW.java.

Hace referencia a top.TOPTW.dataIndex().

Referenciado por top.TOPTWSolution.getInfoSolution().

Gráfico de llamadas para esta función:

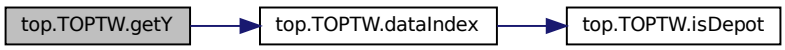


Gráfico de llamadas a esta función:



7.8.3.19. setY()

```
void top.TOPTW.setY (
            int index,
            double y )
```

Establece la coordenada Y de un nodo.

Parámetros

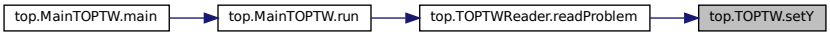
index	índice del nodo.
y	coordenada Y.

Definición en la línea 270 del archivo TOPTW.java.

Hace referencia a top.TOPTW.y.

Referenciado por top.TOPTWReader.readProblem().

Gráfico de llamadas a esta función:



7.8.3.20. getScore() [1/2]

```
double top.TOPTW.getScore (
            int index )
```

Puntuación de un nodo.

Parámetros

index	índice del nodo.

Devuelve

puntuación (beneficio) del nodo.

Definición en la línea 279 del archivo TOPTW.java.

Hace referencia a top.TOPTW.dataIndex().

Referenciado por top.TOPTWGRASP.comprehensiveEvaluation(), top.TOPTWSolution.evaluateFitness(), top.↩ TOPTWSolution.getInfoSolution() y top.TOPTWGRASP.getMaxScore().

Gráfico de llamadas para esta función:

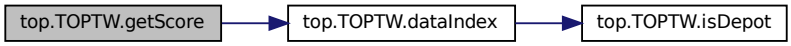
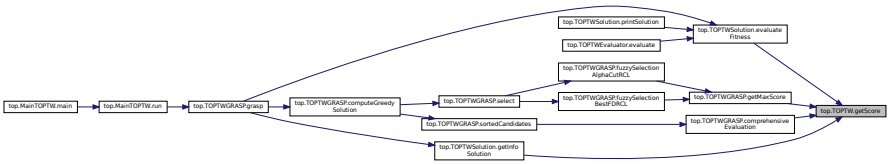


Gráfico de llamadas a esta función:



7.8.3.21. getScore() [2/2]

```
double [] top.TOPTW.getScore ( )
```

Copia del vector de puntuaciones.

Se devuelve una copia para no exponer la representación interna.

Devuelve

vector de puntuaciones indexado por nodo.

Definición en la línea 290 del archivo TOPTW.java.

7.8.3.22. setScore()

```
void top.TOPTW.setScore (
            int index,
            double score )
```

Establece la puntuación de un nodo.

Parámetros

index	índice del nodo.
score	puntuación.

Definición en la línea 299 del archivo TOPTW.java.

Hace referencia a top.TOPTW.score.

Referenciado por top.TOPTWReader.readProblem().

Gráfico de llamadas a esta función:



7.8.3.23. getReadyTime()

```
double top.TOPTW.getReadyTime (
            int index )
```

Inicio de la ventana temporal de un nodo.

Parámetros

index	índice del nodo.

Devuelve

instante más temprano de comienzo de la visita.

Definición en la línea 308 del archivo TOPTW.java.

Hace referencia a top.TOPTW.dataIndex().

Referenciado por top.TOPTWGRASP.departureTime(), top.TOPTWSolution.getInfoSolution() y top.TOPTWGRASP.↩ updateSolution().

Gráfico de llamadas para esta función:

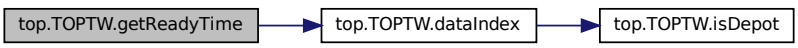
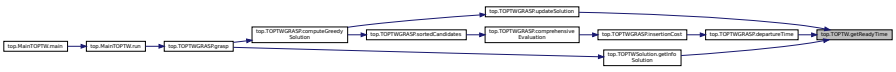


Gráfico de llamadas a esta función:



7.8.3.24. setReadyTime()

```
void top.TOPTW.setReadyTime (
             int index,
             double readyTime )
```

Establece el inicio de la ventana temporal de un nodo.

Parámetros

index	índice del nodo.
readyTime	instante más temprano de comienzo de la visita.

Definición en la línea 317 del archivo TOPTW.java.

Hace referencia a top.TOPTW.readyTime.

Referenciado por top.TOPTWReader.readProblem().

Gráfico de llamadas a esta función:



7.8.3.25. getDueTime()

```
double top.TOPTW.getDueTime (
             int index )
```

Fin de la ventana temporal de un nodo.

Parámetros

index	índice del nodo.

Devuelve

instante más tardío de comienzo de la visita.

Definición en la línea 326 del archivo TOPTW.java.

Hace referencia a top.TOPTW.dataIndex().

Referenciado por top.TOPTWGRASP.departureTime(), top.TOPTWSolution.getInfoSolution() y top.TOPTWReader.↩ readProblem().

Gráfico de llamadas para esta función:

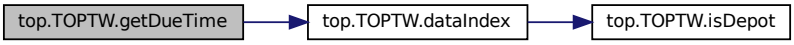


Gráfico de llamadas a esta función:



7.8.3.26. setDueTime()

```
void top.TOPTW.setDueTime (
            int index,
            double dueTime )
```

Establece el fin de la ventana temporal de un nodo.

Parámetros

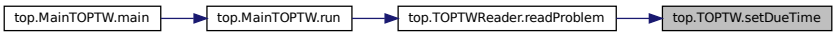
index	índice del nodo.
dueTime	instante más tardío de comienzo de la visita.

Definición en la línea 335 del archivo TOPTW.java.

Hace referencia a top.TOPTW.dueTime.

Referenciado por top.TOPTWReader.readProblem().

Gráfico de llamadas a esta función:



7.8.3.27. getServiceTime()

```
double top.TOPTW.getServiceTime (
            int index )
```

Tiempo de servicio (duración de la visita) de un nodo.

Parámetros

index	índice del nodo.

Devuelve

duración de la visita.

Definición en la línea 344 del archivo TOPTW.java.

Hace referencia a top.TOPTW.dataIndex().

Referenciado por top.TOPTWGRASP.departureTime(), top.TOPTWSolution.getInfoSolution() y top.TOPTWGRASP.↩ updateSolution().

Gráfico de llamadas para esta función:

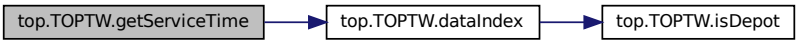
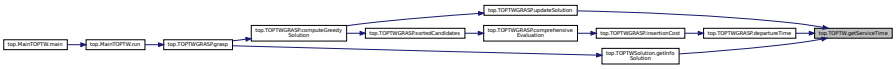


Gráfico de llamadas a esta función:



7.8.3.28. setServiceTime()

```
void top.TOPTW.setServiceTime (
             int index,
             double serviceTime )
```

Establece el tiempo de servicio de un nodo.

Parámetros

index	índice del nodo.
serviceTime	duración de la visita.

Definición en la línea 353 del archivo TOPTW.java.

Hace referencia a top.TOPTW.serviceTime.

Referenciado por top.TOPTWReader.readProblem().

Gráfico de llamadas a esta función:



7.8.3.29. getVehicles()

```
int top.TOPTW.getVehicles ( )
```

Número de vehículos (rutas) disponibles.

Devuelve

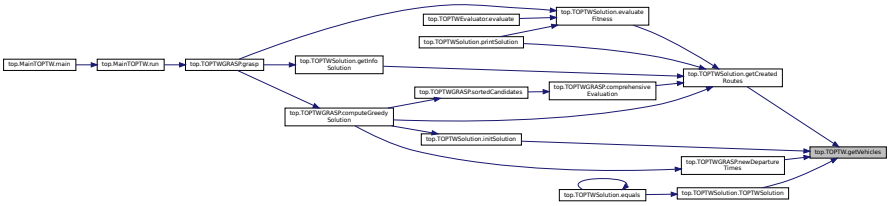
número de vehículos.

Definición en la línea 361 del archivo TOPTW.java.

Hace referencia a top.TOPTW.vehicles.

Referenciado por top.TOPTWSolution.getCreatedRoutes(), top.TOPTWSolution.initSolution(), top.TOPTWGRASP.↩ newDepartureTimes() y top.TOPTWSolution.TOPTWSolution().

Gráfico de llamadas a esta función:



7.8.3.30. getDepots()

```
int top.TOPTW.getDepots ( )
```

Número de depósitos virtuales registrados.

Devuelve

número de depósitos añadidos con addNodeDepot().

Definición en la línea 369 del archivo TOPTW.java.

Hace referencia a top.TOPTW.depots.

7.8.3.31. toString()

```
String top.TOPTW.toString ( )
```

Representación tabular de la instancia.

Devuelve

texto con los datos de cada POI y el número de vehículos.

Definición en la línea 378 del archivo TOPTW.java.

Hace referencia a es.ull.esit.utilities.ExpositoUtilities.getFormat() y top.TOPTW.nodes.

Gráfico de llamadas para esta función:



7.8.3.32. addNode()

```
int top.TOPTW.addNode ( )
```

Añade un nuevo POI incrementando el contador de nodos.

Devuelve

nuevo número de POI.

Definición en la línea 409 del archivo TOPTW.java.

Hace referencia a top.TOPTW.nodes.

7.8.3.33. addNodeDepot()

```
int top.TOPTW.addNodeDepot ( )
```

Registra un nuevo depósito virtual.

Devuelve

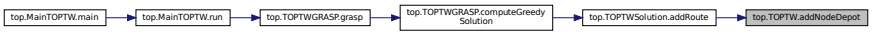
número de depósitos virtuales tras el registro.

Definición en la línea 418 del archivo TOPTW.java.

Hace referencia a top.TOPTW.depots.

Referenciado por top.TOPTWSolution.addRoute().

Gráfico de llamadas a esta función:



7.8.4. Documentación de los datos miembro

7.8.4.1. nodes

```
int top.TOPTW.nodes  [private]
```

Número de POI (sin contar el depósito).

Definición en la línea 26 del archivo TOPTW.java.

Referenciado por top.TOPTW.addNode(), top.TOPTW.getNodes(), top.TOPTW.getPOIs(), top.TOPTW.isDepot(), top.TOPTW.setNodes(), top.TOPTW.TOPTW() y top.TOPTW.toString().

7.8.4.2. x

```
final double [] top.TOPTW.x  [private]
```

Coordenada X de cada nodo (índice 0 = depósito).

Definición en la línea 28 del archivo TOPTW.java.

Referenciado por top.TOPTW.setX().

7.8.4.3. y

```
final double [] top.TOPTW.y  [private]
```

Coordenada Y de cada nodo (índice 0 = depósito).

Definición en la línea 30 del archivo TOPTW.java.

Referenciado por top.TOPTW.setY().

7.8.4.4. score

```
final double [] top.TOPTW.score  [private]
```

Puntuación o beneficio obtenido al visitar cada nodo.

Definición en la línea 32 del archivo TOPTW.java.

Referenciado por top.TOPTW.setScore().

7.8.4.5. readyTime

```
final double [] top.TOPTW.readyTime  [private]
```

Instante más temprano en que puede empezar la visita de cada nodo.

Definición en la línea 34 del archivo TOPTW.java.

Referenciado por top.TOPTW.setReadyTime().

7.8.4.6. dueTime

```
final double [] top.TOPTW.dueTime  [private]
```

Instante más tardío en que puede empezar la visita de cada nodo.

Definición en la línea 36 del archivo TOPTW.java.

Referenciado por top.TOPTW.setDueTime().

7.8.4.7. serviceTime

```
final double [] top.TOPTW.serviceTime  [private]
```

Duración de la visita de cada nodo.

Definición en la línea 38 del archivo TOPTW.java.

Referenciado por top.TOPTW.setServiceTime().

7.8.4.8. vehicles

```
final int top.TOPTW.vehicles  [private]
```

Número de vehículos (rutas / días de visita) disponibles.

Definición en la línea 40 del archivo TOPTW.java.

Referenciado por top.TOPTW.getVehicles().

7.8.4.9. depots

```
int top.TOPTW.depots  [private]
```

Número de depósitos virtuales añadidos por las soluciones.

Definición en la línea 42 del archivo TOPTW.java.

Referenciado por top.TOPTW.addNodeDepot() y top.TOPTW.getDepots().

7.8.4.10. maxTimePerRoute

```
double top.TOPTW.maxTimePerRoute  [private]
```

Duración máxima permitida para cada ruta.

Definición en la línea 44 del archivo TOPTW.java.

Referenciado por top.TOPTW.getMaxTimePerRoute() y top.TOPTW.setMaxTimePerRoute().

7.8.4.11. maxRoutes

```
double top.TOPTW.maxRoutes  [private]
```

Número máximo de rutas permitidas.

Definición en la línea 46 del archivo TOPTW.java.

Referenciado por top.TOPTW.getMaxRoutes() y top.TOPTW.setMaxRoutes().

7.8.4.12. distanceMatrix

```
final double [][] top.TOPTW.distanceMatrix  [private]
```

Matriz simétrica de distancias euclídeas entre nodos.

Definición en la línea 48 del archivo TOPTW.java.

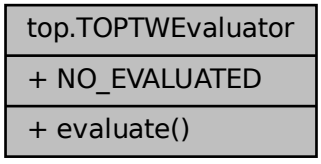
La documentación para esta clase fue generada a partir del siguiente fichero:

- top/TOPTW.java

7.9. Referencia de la Clase top.TOPTWEvaluator

Evaluador de la función objetivo del TOPTW.

Diagrama de colaboración para top.TOPTWEvaluator:



Métodos públicos

- double evaluate (TOPTWSolution solution)

  Evalúa una solución y guarda el resultado en ella.

Atributos públicos estáticos

- static final double NO_EVALUATED = -1.0

  Valor que indica que una solución todavía no ha sido evaluada.

7.9.1. Descripción detallada

Evaluador de la función objetivo del TOPTW.

La función objetivo del TOPTW es maximizar la suma de las puntuaciones de los puntos de interés visitados por todas las rutas. El evaluador calcula ese valor y lo almacena en la propia solución.

Definición en la línea 10 del archivo TOPTWEvaluator.java.

7.9.2. Documentación de las funciones miembro

7.9.2.1. evaluate()

```
double top.TOPTWEvaluator.evaluate (
            TOPTWSolution solution )
```

Evalúa una solución y guarda el resultado en ella.

Parámetros

solution	solución a evaluar.

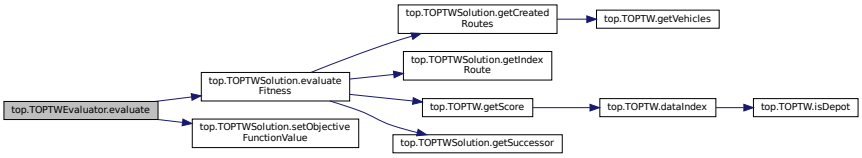
Devuelve

valor de la función objetivo (puntuación total).

Definición en la línea 19 del archivo TOPTWEvaluator.java.

Hace referencia a top.TOPTWSolution.evaluateFitness() y top.TOPTWSolution.setObjectiveFunctionValue().

Gráfico de llamadas para esta función:



7.9.3. Documentación de los datos miembro

7.9.3.1. NO_EVALUATED

```
final double top.TOPTWEvaluator.NO_EVALUATED = -1.0  [static]
```

Valor que indica que una solución todavía no ha sido evaluada.

Definición en la línea 12 del archivo TOPTWEvaluator.java.

Referenciado por top.TOPTWSolution.TOPTWSolution().

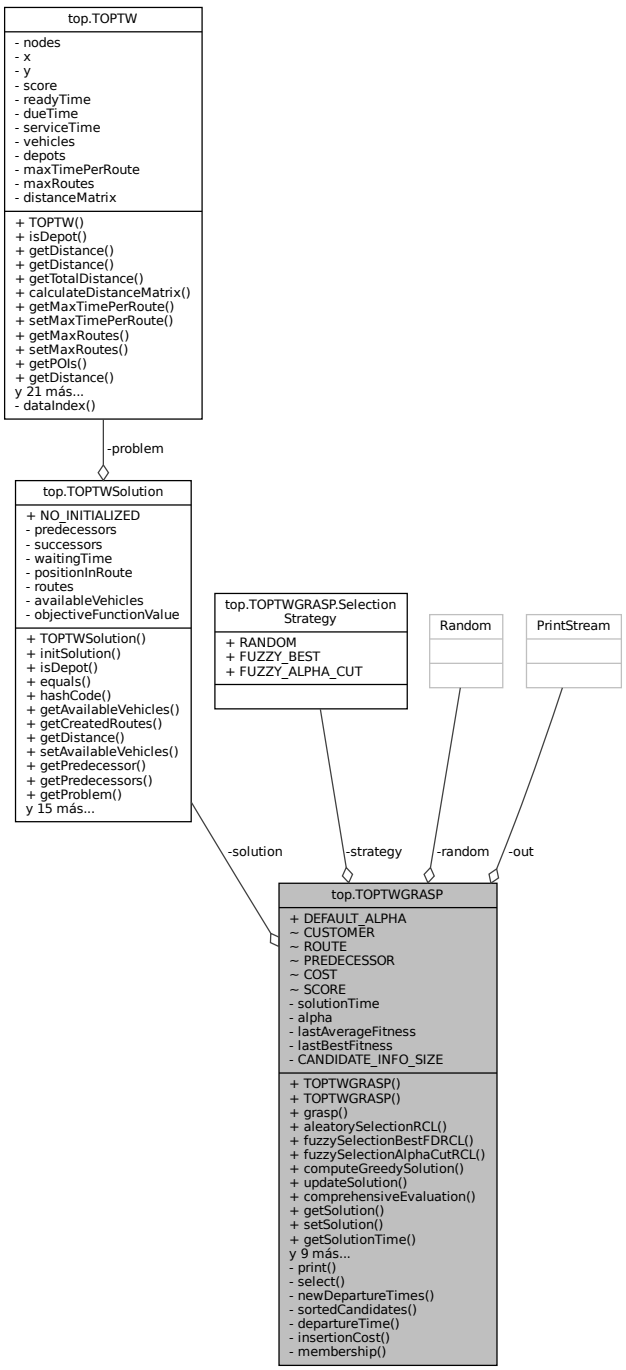
La documentación para esta clase fue generada a partir del siguiente fichero:

- top/TOPTWEvaluator.java

7.10. Referencia de la Clase top.TOPTWGRASP

Metaheurística GRASP (Greedy Randomized Adaptive Search Procedure) para el TOPTW.

Diagrama de colaboración para top.TOPTWGRASP:



Clases

- enum SelectionStrategy

  Estrategias de selección de un candidato dentro de la RCL.

Métodos públicos

- TOPTWGRASP (TOPTWSolution sol)

  Crea el GRASP con un generador aleatorio no determinista y salida silenciosa.
- TOPTWGRASP (TOPTWSolution sol, Random random, PrintStream out)

  Crea el GRASP con un generador y un flujo de salida concretos.
- double grasp (int maxIterations, int maxSizeRCL)

  Ejecuta el GRASP.
- int aleatorySelectionRCL (int maxTRCL)

  Selecciona uniformemente una posición de la RCL.
- int fuzzySelectionBestFDRCL (List< double[ ]> rcl)

  Selección borrosa: candidato con menor grado de pertenencia (mayor puntuación).
- int fuzzySelectionAlphaCutRCL (List< double[ ]> rcl, double alpha)

  Selección borrosa con corte alfa.
- void computeGreedySolution (int maxSizeRCL)

  Construye una solución voraz aleatorizada (fase constructiva del GRASP).
- void updateSolution (double[ ] candidateSelected, List< List< Double >> departureTimes)

  Inserta un candidato en la solución y propaga los tiempos de salida.
- List< double[ ]> comprehensiveEvaluation (List< Integer > customers, List< List< Double >> departure↩ Times)

  Evalúa la mejor inserción factible de cada POI pendiente.
- TOPTWSolution getSolution ()

  Solución de trabajo.
- void setSolution (TOPTWSolution solution)

  Sustituye la solución de trabajo.
- int getSolutionTime ()

  Tiempo de cómputo registrado.
- void setSolutionTime (int solutionTime)

  Registra el tiempo de cómputo.
- SelectionStrategy getStrategy ()

  Estrategia de selección actual.
- void setStrategy (SelectionStrategy strategy)

  Cambia la estrategia de selección dentro de la RCL.
- double getAlpha ()

  Valor alfa del corte borroso.
- void setAlpha (double alpha)

  Cambia el valor alfa del corte borroso.
- void setOutput (PrintStream out)

  Cambia el flujo de salida de los informes.
- double getLastAverageFitness ()

  Puntuación media de la última ejecución.
- double getLastBestFitness ()

  Mejor puntuación de la última ejecución.
- double getMaxScore ()

  Puntuación máxima entre todos los nodos de la instancia.

Atributos públicos estáticos

- static final double DEFAULT_ALPHA = 0.8

  Valor alfa por defecto para el corte borroso.

Atributos Estáticos del 'package'

- static final int CUSTOMER = 0

  Posición del cliente en el vector de información de un candidato.
- static final int ROUTE = 1

  Posición de la ruta en el vector de información de un candidato.
- static final int PREDECESSOR = 2

  Posición del predecesor en el vector de información de un candidato.
- static final int COST = 3

  Posición del coste temporal en el vector de información de un candidato.
- static final int SCORE = 4

  Posición de la puntuación en el vector de información de un candidato.

Métodos privados

- void print (String text)

  Escribe una línea en el flujo de salida, si existe.
- int select (List< double[ ]> rcl)

  Aplica la estrategia de selección configurada.
- List< Double > newDepartureTimes ()

  Crea un vector de tiempos de salida inicializado a cero.
- List< double[ ]> sortedCandidates (List< Integer > customers, List< List< Double >> departureTimes)

  Evalúa los candidatos y los ordena por coste de inserción creciente.
- double departureTime (double arrival, int node)

  Tiempo de salida de un nodo dado su instante de llegada.
- double insertionCost (int routeIndex, int pre, int candidate, List< List< Double >> departureTimes)

  Coste de insertar un candidato justo después de un nodo de una ruta.

Métodos privados estáticos

- static double membership (double[ ] candidate, double maxScore)

  Grado de pertenencia borroso de un candidato.

Atributos privados

- TOPTWSolution solution

  Solución sobre la que se construye.
- int solutionTime

  Tiempo de cómputo de la solución (reservado).
- final Random random

  Generador de números aleatorios (reutilizado en todas las selecciones).
- SelectionStrategy strategy = SelectionStrategy.FUZZY_ALPHA_CUT

  Estrategia de selección dentro de la RCL.
- double alpha = DEFAULT_ALPHA

  Valor alfa para la estrategia FUZZY_ALPHA_CUT.
- PrintStream out

  Flujo donde se escriben los informes; null para no escribir nada.
- double lastAverageFitness

  Puntuación media de la última ejecución de grasp().
- double lastBestFitness

  Mejor puntuación de la última ejecución de grasp().

Atributos privados estáticos

- static final int CANDIDATE_INFO_SIZE = 5

  Tamaño del vector de información de un candidato.

7.10.1. Descripción detallada

Metaheurística GRASP (Greedy Randomized Adaptive Search Procedure) para el TOPTW.

Cada iteración construye una solución de forma voraz y aleatorizada:

1. Se evalúa, para cada POI pendiente, la mejor posición de inserción factible (respetando ventanas temporales y la duración máxima de ruta) en las rutas abiertas.
2. Los candidatos se ordenan por coste temporal de inserción y los mejores (hasta maxSizeRCL) forman la lista restringida de candidatos (RCL).
3. Se elige un candidato de la RCL según la estrategia configurada (aleatoria, borrosa con el mejor grado o borrosa con corte alfa) y se inserta.
4. Si no queda ningún candidato factible se abre una nueva ruta, mientras haya vehículos.

El método grasp() repite la construcción el número de iteraciones indicado y registra la puntuación media y la mejor obtenida.

Pseudocódigo de referencia:

```
procedure GRASP(MaxIterations, Seed)
  for k = 1..MaxIterations do
    Solution = GreedyRandomizedConstruction(Seed)
    Solution = LocalSearch(Solution)          // pendiente de implementar
    UpdateSolution(Solution, BestSolution)
  return BestSolution
```

Definición en la línea 35 del archivo TOPTWGRASP.java.

7.10.2. Documentación del constructor y destructor

7.10.2.1. TOPTWGRASP() [1/2]

```
top.TOPTWGRASP.TOPTWGRASP (
              TOPTWSolution sol )
```

Crea el GRASP con un generador aleatorio no determinista y salida silenciosa.

Parámetros

sol	solución de trabajo.

Definición en la línea 83 del archivo TOPTWGRASP.java.

7.10.2.2. TOPTWGRASP() [2/2]

```
top.TOPTWGRASP.TOPTWGRASP (
            TOPTWSolution sol,
            Random random,
            PrintStream out )
```

Crea el GRASP con un generador y un flujo de salida concretos.

Permite ejecuciones reproducibles (pasando un Random con semilla) y desacopla el algoritmo de la consola.

Parámetros

sol	solución de trabajo.
random	generador de números aleatorios.
out	flujo para los informes de cada iteración, o null para no escribir.

Definición en la línea 97 del archivo TOPTWGRASP.java.

Hace referencia a top.TOPTWGRASP.out y top.TOPTWGRASP.random.

7.10.3. Documentación de las funciones miembro

7.10.3.1. grasp()

```
double top.TOPTWGRASP.grasp (
            int maxIterations,
            int maxSizeRCL )
```

Ejecuta el GRASP.

Construye maxIterations soluciones voraces aleatorizadas, escribe el informe de cada una (si hay flujo de salida) y al final la media y la mejor puntuación. Ambos valores quedan disponibles en getLastAverageFitness() y getLastBestFitness().

Parámetros

maxIterations	número de construcciones.
maxSizeRCL	tamaño máximo de la lista restringida de candidatos.

Devuelve

mejor puntuación obtenida.

Definición en la línea 116 del archivo TOPTWGRASP.java.

Hace referencia a top.TOPTWGRASP.computeGreedySolution(), top.TOPTWSolution.evaluateFitness(), top.↩ TOPTWSolution.getInfoSolution() y top.TOPTWGRASP.print().

Referenciado por top.MainTOPTW.run().

Gráfico de llamadas para esta función:

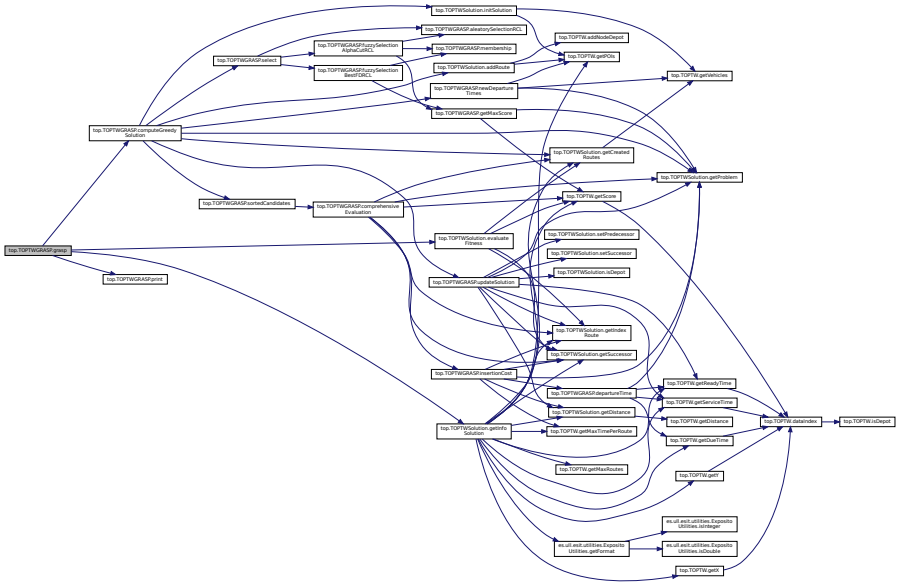
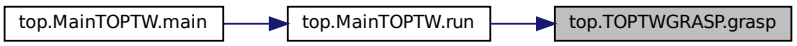


Gráfico de llamadas a esta función:



7.10.3.2. print()

```
void top.TOPTWGRASP.print (
            String text )  [private]
```

Escribe una línea en el flujo de salida, si existe.

Parámetros

text	texto a escribir.

Definición en la línea 141 del archivo TOPTWGRASP.java.

Referenciado por top.TOPTWGRASP.grasp().

Gráfico de llamadas a esta función:



7.10.3.3. aleatorySelectionRCL()

```
int top.TOPTWGRASP.aleatorySelectionRCL (
            int maxTRCL )
```

Selecciona uniformemente una posición de la RCL.

Parámetros

maxTRCL	tamaño de la RCL.

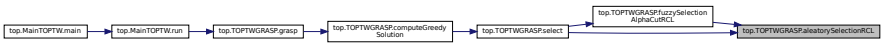
Devuelve

posición seleccionada en [0, maxTRCL).

Definición en la línea 152 del archivo TOPTWGRASP.java.

Referenciado por top.TOPTWGRASP.fuzzySelectionAlphaCutRCL() y top.TOPTWGRASP.select().

Gráfico de llamadas a esta función:



7.10.3.4. membership()

```
static double top.TOPTWGRASP.membership (
            double[] candidate,
            double maxScore )  [static], [private]
```

Grado de pertenencia borroso de un candidato.

Vale 1 - puntuación/puntuación máxima: cuanto menor, más interesante el POI.

Parámetros

candidate	vector de información del candidato.
maxScore	puntuación máxima de la instancia.

Devuelve

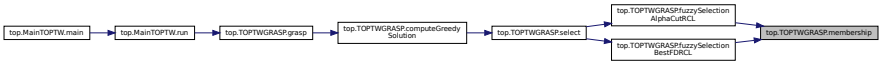
grado de pertenencia.

Definición en la línea 163 del archivo TOPTWGRASP.java.

Hace referencia a top.TOPTWGRASP.SCORE.

Referenciado por top.TOPTWGRASP.fuzzySelectionAlphaCutRCL() y top.TOPTWGRASP.fuzzySelectionBest↩FDRCL().

Gráfico de llamadas a esta función:



7.10.3.5. fuzzySelectionBestFDRCL()

```
int top.TOPTWGRASP.fuzzySelectionBestFDRCL (
            List< double[]> rcl )
```

Selección borrosa: candidato con menor grado de pertenencia (mayor puntuación).

Parámetros

rcl	lista restringida de candidatos.

Devuelve

posición seleccionada, o -1 si la lista está vacía.

Definición en la línea 172 del archivo TOPTWGRASP.java.

Hace referencia a top.TOPTWGRASP.getMaxScore() y top.TOPTWGRASP.membership().

Referenciado por top.TOPTWGRASP.select().

Gráfico de llamadas para esta función:



Gráfico de llamadas a esta función:



7.10.3.6. fuzzySelectionAlphaCutRCL()

```
int top.TOPTWGRASP.fuzzySelectionAlphaCutRCL (
            List< double[]> rcl,
            double alpha )
```

Selección borrosa con corte alfa.

Elige al azar entre los candidatos cuyo grado de pertenencia es menor o igual que alpha; si no hay ninguno, elige al azar entre toda la RCL.

Parámetros

rcl	lista restringida de candidatos.
alpha	umbral del corte.

Devuelve

posición seleccionada.

Definición en la línea 196 del archivo TOPTWGRASP.java.

Hace referencia a top.TOPTWGRASP.aleatorySelectionRCL(), top.TOPTWGRASP.alpha, top.TOPTWGRASP.get↩ MaxScore() y top.TOPTWGRASP.membership().

Referenciado por top.TOPTWGRASP.select().

Gráfico de llamadas para esta función:

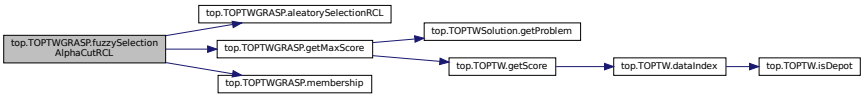


Gráfico de llamadas a esta función:



7.10.3.7. select()

```
int top.TOPTWGRASP.select (
            List< double[]> rcl )  [private]
```

Aplica la estrategia de selección configurada.

Parámetros

rcl	lista restringida de candidatos.

Devuelve

posición seleccionada.

Definición en la línea 215 del archivo TOPTWGRASP.java.

Hace referencia a top.TOPTWGRASP.aleatorySelectionRCL(), top.TOPTWGRASP.fuzzySelectionAlphaCutRCL() y top.TOPTWGRASP.fuzzySelectionBestFDRCL().

Referenciado por top.TOPTWGRASP.computeGreedySolution().

Gráfico de llamadas para esta función:

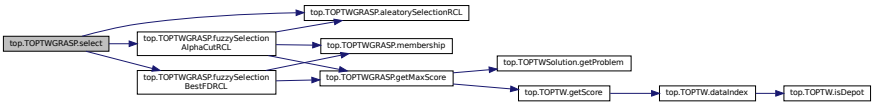
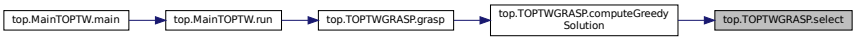


Gráfico de llamadas a esta función:



7.10.3.8. newDepartureTimes()

```
List<Double> top.TOPTWGRASP.newDepartureTimes ( )  [private]
```

Crea un vector de tiempos de salida inicializado a cero.

Devuelve

lista con un tiempo por nodo (POI y depósitos).

Definición en la línea 231 del archivo TOPTWGRASP.java.

Hace referencia a top.TOPTW.getPOIs(), top.TOPTWSolution.getProblem() y top.TOPTW.getVehicles().

Referenciado por top.TOPTWGRASP.computeGreedySolution().

Gráfico de llamadas para esta función:

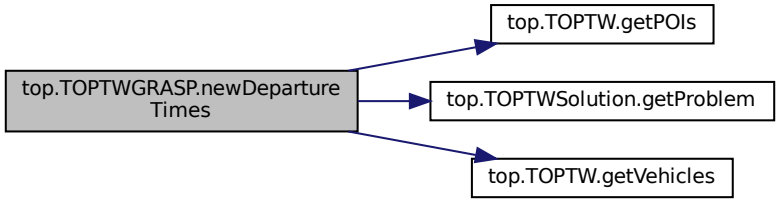


Gráfico de llamadas a esta función:



7.10.3.9. sortedCandidates()

```
List<double[]> top.TOPTWGRASP.sortedCandidates (
            List< Integer > customers,
            List< List< Double >> departureTimes )  [private]
```

Evalúa los candidatos y los ordena por coste de inserción creciente.

Parámetros

customers	POI pendientes.
departureTimes	tiempos de salida por ruta y nodo.

Devuelve

candidatos ordenados.

Definición en la línea 246 del archivo TOPTWGRASP.java.

Hace referencia a top.TOPTWGRASP.comprehensiveEvaluation() y top.TOPTWGRASP.COST.

Referenciado por top.TOPTWGRASP.computeGreedySolution().

Gráfico de llamadas para esta función:

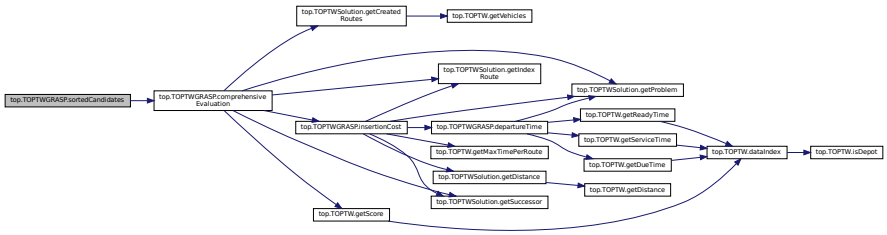


Gráfico de llamadas a esta función:



7.10.3.10. computeGreedySolution()

```
void top.TOPTWGRASP.computeGreedySolution (
             int maxSizeRCL )
```

Construye una solución voraz aleatorizada (fase constructiva del GRASP).

Parámetros

maxSizeRCL	tamaño máximo de la lista restringida de candidatos.

Definición en la línea 256 del archivo TOPTWGRASP.java.

Hace referencia a top.TOPTWSolution.addRoute(), top.TOPTWGRASP.CUSTOMER, top.TOPTWSolution.get↩ CreatedRoutes(), top.TOPTWSolution.getProblem(), top.TOPTWSolution.initSolution(), top.TOPTWGRASP.new↩ DepartureTimes(), top.TOPTWGRASP.select(), top.TOPTWGRASP.sortedCandidates() y top.TOPTWGRASP.↩ updateSolution().

Referenciado por top.TOPTWGRASP.grasp().

Gráfico de llamadas para esta función:

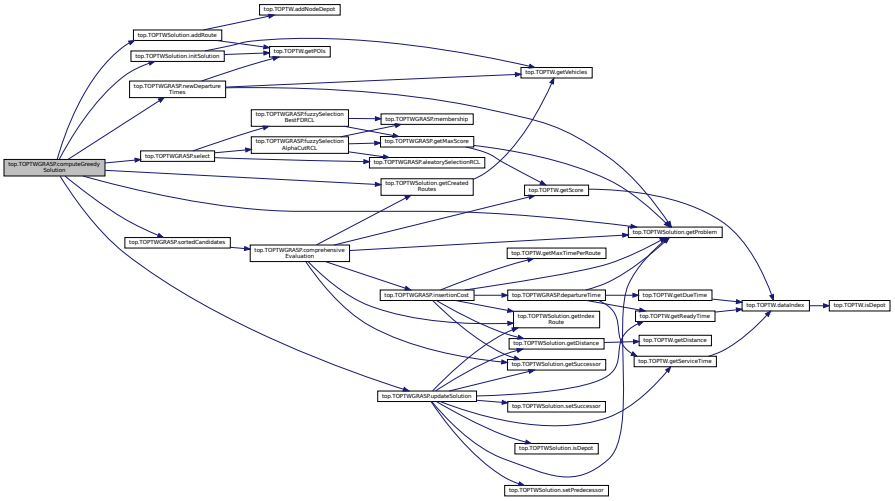


Gráfico de llamadas a esta función:



7.10.3.11. updateSolution()

```
void top.TOPTWGRASP.updateSolution (
            double[] candidateSelected,
            List< List< Double >> departureTimes )
```

Inserta un candidato en la solución y propaga los tiempos de salida.

Enlaza el cliente tras su predecesor y recalcula los tiempos de salida de todos los nodos posteriores de la ruta hasta volver al depósito.

Parámetros

candidateSelected	vector {cliente, ruta, predecesor, coste, puntuación}.
departureTimes	tiempos de salida por ruta y nodo (se actualizan).

Definición en la línea 297 del archivo TOPTWGRASP.java.

Hace referencia a top.TOPTWGRASP.CUSTOMER, top.TOPTWSolution.getDistance(), top.TOPTWSolution.↩ getIndexRoute(), top.TOPTWSolution.getProblem(), top.TOPTW.getReadyTime(), top.TOPTW.getServiceTime(), top.TOPTWSolution.getSuccessor(), top.TOPTWSolution.isDepot(), top.TOPTWGRASP.PREDECESSOR, top.↩ TOPTWGRASP.ROUTE, top.TOPTWSolution.setPredecessor() y top.TOPTWSolution.setSuccessor().

Referenciado por top.TOPTWGRASP.computeGreedySolution().

Gráfico de llamadas para esta función:

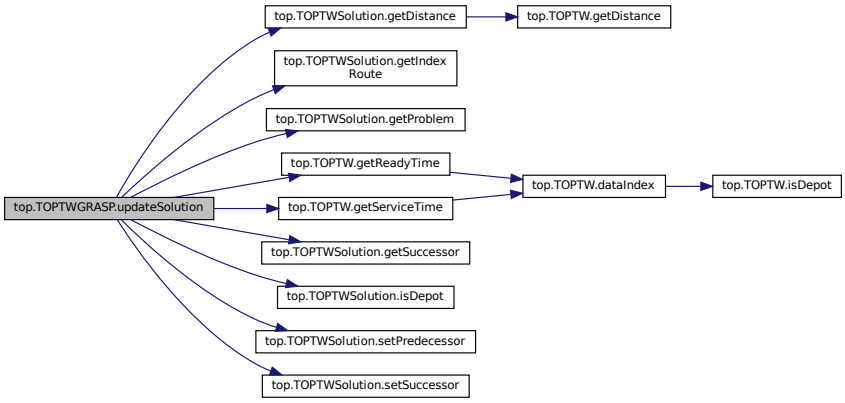


Gráfico de llamadas a esta función:



7.10.3.12. departureTime()

```
double top.TOPTWGRASP.departureTime (
            double arrival,
            int node )  [private]
```

Tiempo de salida de un nodo dado su instante de llegada.

Si se llega antes de la apertura de la ventana se espera hasta ella; después se suma la duración de la visita. Si la llegada no es estrictamente anterior al cierre de la ventana, la visita no es factible.

Parámetros

arrival	instante de llegada.
node	nodo visitado.

Devuelve

instante de salida, o NaN si se incumple la ventana temporal.

Definición en la línea 336 del archivo TOPTWGRASP.java.

Hace referencia a top.TOPTW.getDueTime(), top.TOPTWSolution.getProblem(), top.TOPTW.getReadyTime() y top.TOPTW.getServiceTime().

Referenciado por top.TOPTWGRASP.insertionCost().

Gráfico de llamadas para esta función:

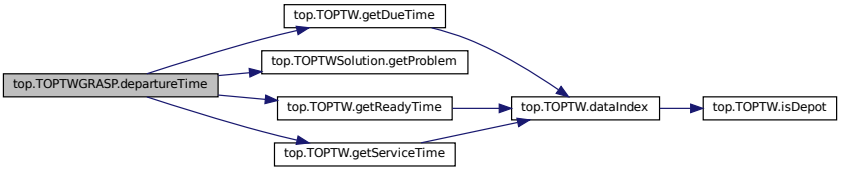


Gráfico de llamadas a esta función:



7.10.3.13. insertionCost()

```
double top.TOPTWGRASP.insertionCost (
            int routeIndex,
            int pre,
            int candidate,
            List< List< Double >> departureTimes )  [private]
```

Coste de insertar un candidato justo después de un nodo de una ruta.

Comprueba la ventana temporal del candidato, la del sucesor y la de todos los nodos siguientes hasta volver al depósito, así como la duración máxima de ruta.

Parámetros

routeIndex	ruta donde se inserta.
pre	nodo tras el que se inserta.
candidate	POI candidato.
departureTimes	tiempos de salida por ruta y nodo.

Devuelve

tiempo de llegada al depósito tras la inserción, o NaN si no es factible.

Definición en la línea 356 del archivo TOPTWGRASP.java.

Hace referencia a top.TOPTWGRASP.departureTime(), top.TOPTWSolution.getDistance(), top.TOPTWSolution.↩ getIndexRoute(), top.TOPTW.getMaxTimePerRoute(), top.TOPTWSolution.getProblem() y top.TOPTWSolution.↩ getSuccessor().

Referenciado por top.TOPTWGRASP.comprehensiveEvaluation().

Gráfico de llamadas para esta función:

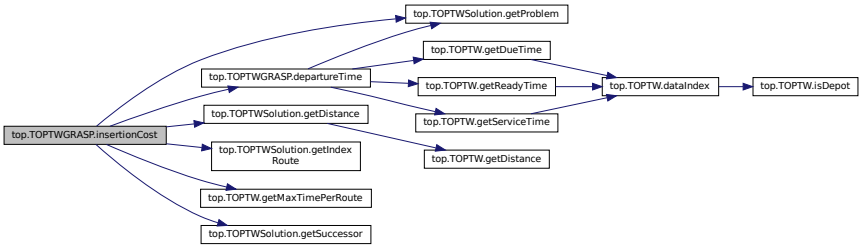


Gráfico de llamadas a esta función:



7.10.3.14. comprehensiveEvaluation()

```
List<double[]> top.TOPTWGRASP.comprehensiveEvaluation (
            List< Integer > customers,
            List< List< Double >> departureTimes )
```

Evalúa la mejor inserción factible de cada POI pendiente.

Para cada cliente recorre todas las posiciones de todas las rutas creadas y se queda con la de menor coste temporal.

Parámetros

customers	POI pendientes de insertar.
departureTimes	tiempos de salida por ruta y nodo.

Devuelve

lista de vectores {cliente, ruta, predecesor, coste, puntuación}, uno por cada cliente con al menos una inserción factible.

Definición en la línea 387 del archivo TOPTWGRASP.java.

Hace referencia a top.TOPTWGRASP.CANDIDATE_INFO_SIZE, top.TOPTWGRASP.COST, top.TOPTWGRASP.CUSTOMER, top.TOPTWSolution.getCreatedRoutes(), top.TOPTWSolution.getIndexRoute(), top.TOPTWSolution.getProblem(), top.TOPTW.getScore(), top.TOPTWSolution.getSuccessor(), top.TOPTWGRASP.insertionCost(), top.TOPTWGRASP.PREDECESSOR, top.TOPTWGRASP.ROUTE y top.TOPTWGRASP.SCORE.

Referenciado por top.TOPTWGRASP.sortedCandidates().

Gráfico de llamadas para esta función:

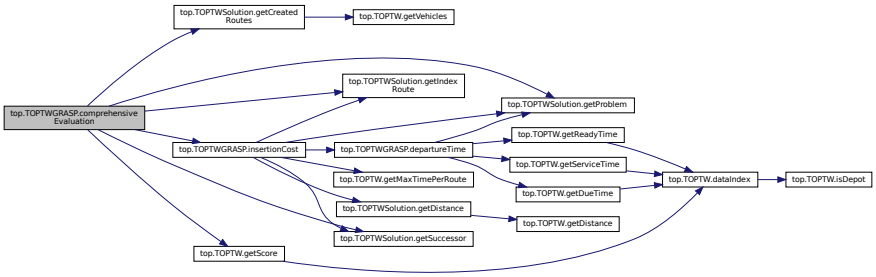


Gráfico de llamadas a esta función:



7.10.3.15. getSolution()

```
TOPTWSolution top.TOPTWGRASP.getSolution ( )
```

Solución de trabajo.

Devuelve

solución actual.

Definición en la línea 418 del archivo TOPTWGRASP.java.

Hace referencia a top.TOPTWGRASP.solution.

7.10.3.16. setSolution()

```
void top.TOPTWGRASP.setSolution (
            TOPTWSolution solution )
```

Sustituye la solución de trabajo.

Parámetros

solution	nueva solución.

Definición en la línea 426 del archivo TOPTWGRASP.java.

Hace referencia a top.TOPTWGRASP.solution.

7.10.3.17. getSolutionTime()

```
int top.TOPTWGRASP.getSolutionTime ( )
```

Tiempo de cómputo registrado.

Devuelve

tiempo de solución.

Definición en la línea 434 del archivo TOPTWGRASP.java.

Hace referencia a top.TOPTWGRASP.solutionTime.

7.10.3.18. setSolutionTime()

```
void top.TOPTWGRASP.setSolutionTime (
            int solutionTime )
```

Registra el tiempo de cómputo.

Parámetros

solutionTime	tiempo de solución.

Definición en la línea 442 del archivo TOPTWGRASP.java.

Hace referencia a top.TOPTWGRASP.solutionTime.

7.10.3.19. getStrategy()

```
SelectionStrategy top.TOPTWGRASP.getStrategy ( )
```

Estrategia de selección actual.

Devuelve

estrategia.

Definición en la línea 450 del archivo TOPTWGRASP.java.

Hace referencia a top.TOPTWGRASP.strategy.

7.10.3.20. setStrategy()

```
void top.TOPTWGRASP.setStrategy (
            SelectionStrategy strategy )
```

Cambia la estrategia de selección dentro de la RCL.

Parámetros

strategy	nueva estrategia.

Definición en la línea 458 del archivo TOPTWGRASP.java.

Hace referencia a top.TOPTWGRASP.strategy.

7.10.3.21. getAlpha()

```
double top.TOPTWGRASP.getAlpha ( )
```

Valor alfa del corte borroso.

Devuelve

alfa.

Definición en la línea 466 del archivo TOPTWGRASP.java.

Hace referencia a top.TOPTWGRASP.alpha.

7.10.3.22. setAlpha()

```
void top.TOPTWGRASP.setAlpha (
            double alpha )
```

Cambia el valor alfa del corte borroso.

Parámetros

alpha	nuevo valor en [0, 1].

Definición en la línea 474 del archivo TOPTWGRASP.java.

Hace referencia a top.TOPTWGRASP.alpha.

7.10.3.23. setOutput()

```
void top.TOPTWGRASP.setOutput (
             PrintStream out )
```

Cambia el flujo de salida de los informes.

Parámetros

out	flujo, o null para no escribir.

Definición en la línea 482 del archivo TOPTWGRASP.java.

Hace referencia a top.TOPTWGRASP.out.

7.10.3.24. getLastAverageFitness()

```
double top.TOPTWGRASP.getLastAverageFitness ( )
```

Puntuación media de la última ejecución.

Devuelve

media de las puntuaciones.

Definición en la línea 490 del archivo TOPTWGRASP.java.

Hace referencia a top.TOPTWGRASP.lastAverageFitness.

7.10.3.25. getLastBestFitness()

```
double top.TOPTWGRASP.getLastBestFitness ( )
```

Mejor puntuación de la última ejecución.

Devuelve

mejor puntuación.

Definición en la línea 498 del archivo TOPTWGRASP.java.

Hace referencia a top.TOPTWGRASP.lastBestFitness.

7.10.3.26. getMaxScore()

```
double top.TOPTWGRASP.getMaxScore ( )
```

Puntuación máxima entre todos los nodos de la instancia.

Devuelve

puntuación máxima.

Definición en la línea 506 del archivo TOPTWGRASP.java.

Hace referencia a top.TOPTWSolution.getProblem() y top.TOPTW.getScore().

Referenciado por top.TOPTWGRASP.fuzzySelectionAlphaCutRCL() y top.TOPTWGRASP.fuzzySelectionBest↩FDRCL().

Gráfico de llamadas para esta función:

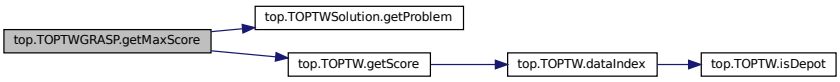


Gráfico de llamadas a esta función:



7.10.4. Documentación de los datos miembro

7.10.4.1. CUSTOMER

```
final int top.TOPTWGRASP.CUSTOMER = 0  [static], [package]
```

Posición del cliente en el vector de información de un candidato.

Definición en la línea 48 del archivo TOPTWGRASP.java.

Referenciado por top.TOPTWGRASP.comprehensiveEvaluation(), top.TOPTWGRASP.computeGreedySolution() y top.TOPTWGRASP.updateSolution().

7.10.4.2. ROUTE

```
final int top.TOPTWGRASP.ROUTE = 1  [static], [package]
```

Posición de la ruta en el vector de información de un candidato.

Definición en la línea 50 del archivo TOPTWGRASP.java.

Referenciado por top.TOPTWGRASP.comprehensiveEvaluation() y top.TOPTWGRASP.updateSolution().

7.10.4.3. PREDECESSOR

```
final int top.TOPTWGRASP.PREDECESSOR = 2  [static], [package]
```

Posición del predecesor en el vector de información de un candidato.

Definición en la línea 52 del archivo TOPTWGRASP.java.

Referenciado por top.TOPTWGRASP.comprehensiveEvaluation() y top.TOPTWGRASP.updateSolution().

7.10.4.4. COST

```
final int top.TOPTWGRASP.COST = 3  [static], [package]
```

Posición del coste temporal en el vector de información de un candidato.

Definición en la línea 54 del archivo TOPTWGRASP.java.

Referenciado por top.TOPTWGRASP.comprehensiveEvaluation() y top.TOPTWGRASP.sortedCandidates().

7.10.4.5. SCORE

```
final int top.TOPTWGRASP.SCORE = 4  [static], [package]
```

Posición de la puntuación en el vector de información de un candidato.

Definición en la línea 56 del archivo TOPTWGRASP.java.

Referenciado por top.TOPTWGRASP.comprehensiveEvaluation() y top.TOPTWGRASP.membership().

7.10.4.6. CANDIDATE_INFO_SIZE

```
final int top.TOPTWGRASP.CANDIDATE_INFO_SIZE = 5  [static], [private]
```

Tamaño del vector de información de un candidato.

Definición en la línea 58 del archivo TOPTWGRASP.java.

Referenciado por top.TOPTWGRASP.comprehensiveEvaluation().

7.10.4.7. DEFAULT_ALPHA

```
final double top.TOPTWGRASP.DEFAULT_ALPHA = 0.8  [static]
```

Valor alfa por defecto para el corte borroso.

Definición en la línea 60 del archivo TOPTWGRASP.java.

7.10.4.8. solution

```
TOPTWSolution top.TOPTWGRASP.solution  [private]
```

Solución sobre la que se construye.

Definición en la línea 63 del archivo TOPTWGRASP.java.

Referenciado por top.TOPTWGRASP.getSolution() y top.TOPTWGRASP.setSolution().

7.10.4.9. solutionTime

```
int top.TOPTWGRASP.solutionTime  [private]
```

Tiempo de cómputo de la solución (reservado).

Definición en la línea 65 del archivo TOPTWGRASP.java.

Referenciado por top.TOPTWGRASP.getSolutionTime() y top.TOPTWGRASP.setSolutionTime().

7.10.4.10. random

```
final Random top.TOPTWGRASP.random  [private]
```

Generador de números aleatorios (reutilizado en todas las selecciones).

Definición en la línea 67 del archivo TOPTWGRASP.java.

Referenciado por top.TOPTWGRASP.TOPTWGRASP().

7.10.4.11. strategy

```
SelectionStrategy top.TOPTWGRASP.strategy = SelectionStrategy.FUZZY_ALPHA_CUT  [private]
```

Estrategia de selección dentro de la RCL.

Definición en la línea 69 del archivo TOPTWGRASP.java.

Referenciado por top.TOPTWGRASP.getStrategy() y top.TOPTWGRASP.setStrategy().

7.10.4.12. alpha

```
double top.TOPTWGRASP.alpha = DEFAULT_ALPHA  [private]
```

Valor alfa para la estrategia FUZZY_ALPHA_CUT.

Definición en la línea 71 del archivo TOPTWGRASP.java.

Referenciado por top.TOPTWGRASP.fuzzySelectionAlphaCutRCL(), top.TOPTWGRASP.getAlpha() y top.↩ TOPTWGRASP.setAlpha().

7.10.4.13. out

```
PrintStream top.TOPTWGRASP.out  [private]
```

Flujo donde se escriben los informes; null para no escribir nada.

Definición en la línea 73 del archivo TOPTWGRASP.java.

Referenciado por top.TOPTWGRASP.setOutput() y top.TOPTWGRASP.TOPTWGRASP().

7.10.4.14. lastAverageFitness

```
double top.TOPTWGRASP.lastAverageFitness  [private]
```

Puntuación media de la última ejecución de grasp().

Definición en la línea 75 del archivo TOPTWGRASP.java.

Referenciado por top.TOPTWGRASP.getLastAverageFitness().

7.10.4.15. lastBestFitness

```
double top.TOPTWGRASP.lastBestFitness  [private]
```

Mejor puntuación de la última ejecución de grasp().

Definición en la línea 77 del archivo TOPTWGRASP.java.

Referenciado por top.TOPTWGRASP.getLastBestFitness().

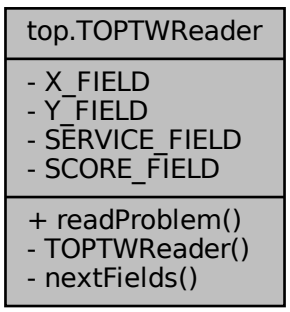
La documentación para esta clase fue generada a partir del siguiente fichero:

- top/TOPTWGRASP.java

7.11. Referencia de la Clase top.TOPTWReader

Lector de instancias TOPTW en el formato de Cordeau / Righini.

Diagrama de colaboración para top.TOPTWReader:



Métodos públicos estáticos

- static TOPTW readProblem (String filePath) throws IOException

  Lee una instancia desde un fichero.

Métodos privados

- TOPTWReader ()

  Clase de utilidad: no instanciable.

Métodos privados estáticos

- static String[ ] nextFields (BufferedReader reader, Path path) throws IOException

  Lee la siguiente línea y la divide en campos.

Atributos privados estáticos

- static final int X_FIELD = 1

  Posición del campo x.
- static final int Y_FIELD = 2

  Posición del campo y.
- static final int SERVICE_FIELD = 3

  Posición del campo de duración de la visita.
- static final int SCORE_FIELD = 4

  Posición del campo de puntuación.

7.11.1. Descripción detallada

Lector de instancias TOPTW en el formato de Cordeau / Righini.

Formato del fichero (ver Instances/TOPTW/formatInstances.txt):

- Línea 1: `k v N t`, donde v es el número de rutas y N el número de POI.
- Línea 2: `D Q` (no se usa).
- Una línea por nodo (empezando por el depósito, índice 0): `i x y d S f a lista O C`, donde d es la duración de la visita, S la puntuación, la lista tiene longitud variable (depende de a) y O/C son la apertura y el cierre de la ventana temporal.

Como la lista tiene longitud variable, la ventana temporal se toma siempre de los dos últimos campos de la línea.

Definición en la línea 26 del archivo TOPTWReader.java.

7.11.2. Documentación del constructor y destructor

7.11.2.1. TOPTWReader()

```
top.TOPTWReader.TOPTWReader ( )  [private]
```

Clase de utilidad: no instanciable.

Definición en la línea 40 del archivo TOPTWReader.java.

7.11.3. Documentación de las funciones miembro

7.11.3.1. readProblem()

```
static TOPTW top.TOPTWReader.readProblem (
            String filePath ) throws IOException  [static]
```

Lee una instancia desde un fichero.

Carga coordenadas, tiempos de servicio, puntuaciones y ventanas temporales, calcula la matriz de distancias y fija la duración máxima de ruta al cierre de la ventana del depósito.

Parámetros

filePath	ruta del fichero de la instancia.

Devuelve

instancia cargada.

Excepciones

IOException	si el fichero no existe, no se puede leer o está incompleto.

Definición en la línea 54 del archivo TOPTWReader.java.

Hace referencia a top.TOPTW.calculateDistanceMatrix(), top.TOPTW.getDueTime(), top.TOPTW.getPOIs(), top.↩ TOPTWReader.nextFields(), top.TOPTWReader.SCORE_FIELD, top.TOPTWReader.SERVICE_FIELD, top.↩ TOPTW.setDueTime(), top.TOPTW.setMaxTimePerRoute(), top.TOPTW.setReadyTime(), top.TOPTW.setScore(), top.TOPTW.setServiceTime(), top.TOPTW.setX(), top.TOPTW.setY(), top.TOPTWReader.X_FIELD y top.↩ TOPTWReader.Y_FIELD.

Referenciado por top.MainTOPTW.run().

Gráfico de llamadas para esta función:

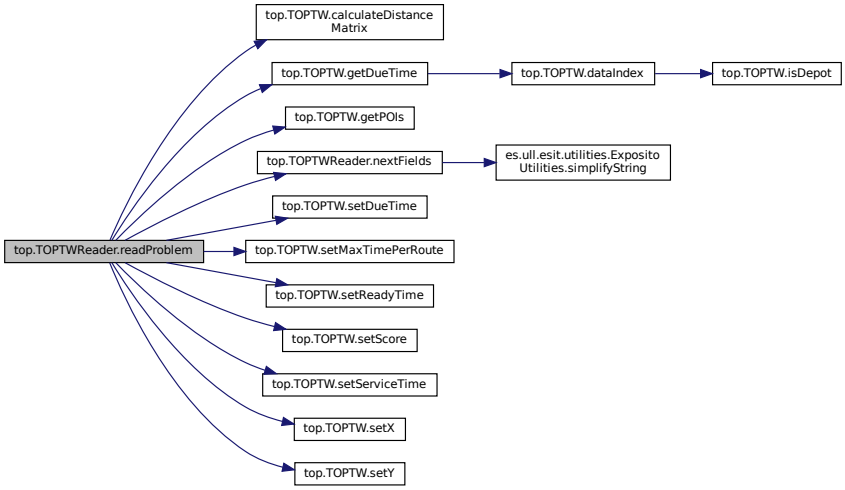
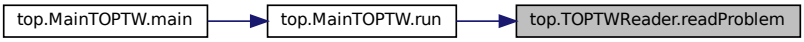


Gráfico de llamadas a esta función:



7.11.3.2. nextFields()

```
static String [] top.TOPTWReader.nextFields (
            BufferedReader reader,
            Path path ) throws IOException  [static], [private]
```

Lee la siguiente línea y la divide en campos.

Parámetros

reader	lector abierto.
path	fichero (para los mensajes de error).

Devuelve

campos de la línea normalizada.

Excepciones

IOException	si se alcanza el final del fichero.

Definición en la línea 85 del archivo TOPTWReader.java.

Hace referencia a es.ull.esit.utilities.ExpositoUtilities.simplifyString().

Referenciado por top.TOPTWReader.readProblem().

Gráfico de llamadas para esta función:

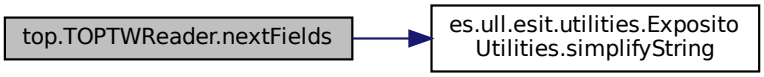


Gráfico de llamadas a esta función:



7.11.4. Documentación de los datos miembro

7.11.4.1. X_FIELD

```
final int top.TOPTWReader.X_FIELD = 1  [static], [private]
```

Posición del campo x.

Definición en la línea 29 del archivo TOPTWReader.java.

Referenciado por top.TOPTWReader.readProblem().

7.11.4.2. Y_FIELD

```
final int top.TOPTWReader.Y_FIELD = 2  [static], [private]
```

Posición del campo y.

Definición en la línea 31 del archivo TOPTWReader.java.

Referenciado por top.TOPTWReader.readProblem().

7.11.4.3. SERVICE_FIELD

```
final int top.TOPTWReader.SERVICE_FIELD = 3  [static], [private]
```

Posición del campo de duración de la visita.

Definición en la línea 33 del archivo TOPTWReader.java.

Referenciado por top.TOPTWReader.readProblem().

7.11.4.4. SCORE_FIELD

```
final int top.TOPTWReader.SCORE_FIELD = 4  [static], [private]
```

Posición del campo de puntuación.

Definición en la línea 35 del archivo TOPTWReader.java.

Referenciado por top.TOPTWReader.readProblem().

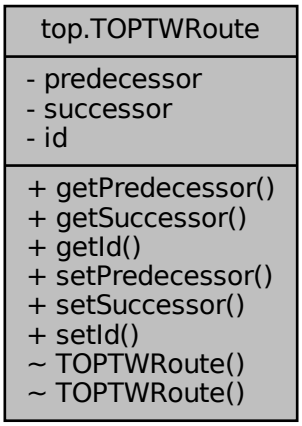
La documentación para esta clase fue generada a partir del siguiente fichero:

- top/TOPTWReader.java

7.12. Referencia de la Clase top.TOPTWRoute

Descriptor ligero de una ruta: identificador y nodos extremos.

Diagrama de colaboración para top.TOPTWRoute:



Métodos públicos

- int getPredecessor ()

  Predecesor de referencia.
- int getSuccessor ()

  Sucesor de referencia.
- int getId ()

  Identificador de la ruta.
- void setPredecessor (int pre)

  Establece el predecesor.
- void setSuccessor (int suc)

  Establece el sucesor.
- void setId (int id)

  Establece el identificador.

Funciones del 'package'

- TOPTWRoute ()

  Crea una ruta sin datos.
- TOPTWRoute (int pre, int succ, int id)

  Crea una ruta con sus datos.

Atributos privados

- int predecessor

  Nodo predecesor de referencia.
- int successor

  Nodo sucesor de referencia.
- int id

  Identificador de la ruta.

7.12.1. Descripción detallada

Descriptor ligero de una ruta: identificador y nodos extremos.

Estructura de datos auxiliar que agrupa el identificador de una ruta junto con un predecesor y un sucesor de referencia.

Definición en la línea 9 del archivo TOPTWRoute.java.

7.12.2. Documentación del constructor y destructor

7.12.2.1. TOPTWRoute() [1/2]

```
top.TOPTWRoute.TOPTWRoute ( )  [package]
```

Crea una ruta sin datos.

Definición en la línea 20 del archivo TOPTWRoute.java.

Hace referencia a top.TOPTWSolution.NO_INITIALIZED.

7.12.2.2. TOPTWRoute() [2/2]

```
top.TOPTWRoute.TOPTWRoute (
            int pre,
            int succ,
            int id )  [package]
```

Crea una ruta con sus datos.

Parámetros

pre	predecesor.
succ	sucesor.
id	identificador.

Definición en la línea 30 del archivo TOPTWRoute.java.

Hace referencia a top.TOPTWRoute.id.

7.12.3. Documentación de las funciones miembro

7.12.3.1. getPredecessor()

```
int top.TOPTWRoute.getPredecessor ( )
```

Predecesor de referencia.

Devuelve

predecesor.

Definición en la línea 40 del archivo TOPTWRoute.java.

Hace referencia a top.TOPTWRoute.predecessor.

7.12.3.2. getSuccessor()

```
int top.TOPTWRoute.getSuccessor ( )
```

Sucesor de referencia.

Devuelve

sucesor.

Definición en la línea 48 del archivo TOPTWRoute.java.

Hace referencia a top.TOPTWRoute.successor.

7.12.3.3. getId()

```
int top.TOPTWRoute.getId ( )
```

Identificador de la ruta.

Devuelve

identificador.

Definición en la línea 56 del archivo TOPTWRoute.java.

Hace referencia a top.TOPTWRoute.id.

7.12.3.4. setPredecessor()

```
void top.TOPTWRoute.setPredecessor (
            int pre )
```

Establece el predecesor.

Parámetros

pre	predecesor.

Definición en la línea 64 del archivo TOPTWRoute.java.

7.12.3.5. setSuccessor()

```
void top.TOPTWRoute.setSuccessor (
            int suc )
```

Establece el sucesor.

Parámetros

suc	sucesor.

Definición en la línea 72 del archivo TOPTWRoute.java.

7.12.3.6. setId()

```
void top.TOPTWRoute.setId (
            int id )
```

Establece el identificador.

Parámetros

id	identificador.

Definición en la línea 80 del archivo TOPTWRoute.java.

Hace referencia a top.TOPTWRoute.id.

7.12.4. Documentación de los datos miembro

7.12.4.1. predecessor

```
int top.TOPTWRoute.predecessor  [private]
```

Nodo predecesor de referencia.

Definición en la línea 11 del archivo TOPTWRoute.java.

Referenciado por top.TOPTWRoute.getPredecessor().

7.12.4.2. successor

```
int top.TOPTWRoute.successor  [private]
```

Nodo sucesor de referencia.

Definición en la línea 13 del archivo TOPTWRoute.java.

Referenciado por top.TOPTWRoute.getSuccessor().

7.12.4.3. id

```
int top.TOPTWRoute.id  [private]
```

Identificador de la ruta.

Definición en la línea 15 del archivo TOPTWRoute.java.

Referenciado por top.TOPTWRoute.getId(), top.TOPTWRoute.setId() y top.TOPTWRoute.TOPTWRoute().

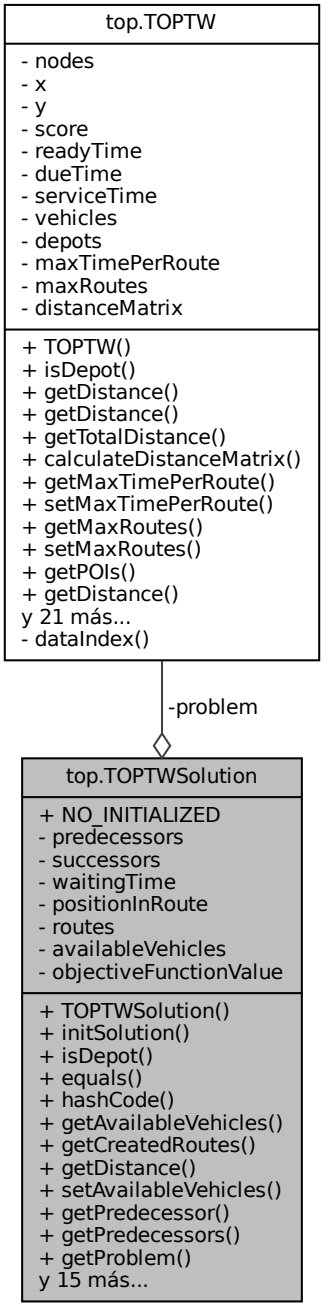
La documentación para esta clase fue generada a partir del siguiente fichero:

- top/TOPTWRoute.java

7.13. Referencia de la Clase top.TOPTWSolution

Solución del problema TOPTW representada mediante listas doblemente enlazadas.

Diagrama de colaboración para top.TOPTWSolution:



Métodos públicos

- TOPTWSolution (TOPTW problem)

  Crea una solución vacía asociada a una instancia.
- void initSolution ()

  Reinicia la solución dejando una única ruta vacía (depósito 0 → depósito 0).
- boolean isDepot (int c)

Indica si un nodo es el depósito de alguna ruta de la solución.

- boolean equals (Object o)

  Compara dos soluciones por su vector de predecesores.
- int hashCode ()

  Código hash coherente con equals().
- int getAvailableVehicles ()

  Vehículos todavía disponibles.
- int getCreatedRoutes ()

  Número de rutas creadas en la solución.
- double getDistance (int x, int y)

  Distancia entre dos nodos de la instancia.
- void setAvailableVehicles (int availableVehicles)

  Establece el número de vehículos disponibles.
- int getPredecessor (int customer)

  Predecesor de un nodo.
- int[ ] getPredecessors ()

  Copia del vector de predecesores.
- TOPTW getProblem ()

  Instancia del problema.
- double getObjectiveFunctionValue ()

  Valor de la función objetivo almacenado.
- int getPositionInRoute (int customer)

  Posición de un POI en su ruta.
- int getSuccessor (int customer)

  Sucesor de un nodo.
- int[ ] getSuccessors ()

  Copia del vector de sucesores.
- int getIndexRoute (int index)

  Depósito de la ruta k-ésima.
- double getWaitingTime (int customer)

  Tiempo de espera en un POI.
- void setObjectiveFunctionValue (double objectiveFunctionValue)

  Almacena el valor de la función objetivo.
- void setPositionInRoute (int customer, int position)

  Establece la posición de un POI en su ruta.
- void setPredecessor (int customer, int predecessor)

  Establece el predecesor de un nodo.
- void setSuccessor (int customer, int successor)

  Establece el sucesor de un nodo.
- void setWaitingTime (int customer, double waitingTime)

  Establece el tiempo de espera de un POI.
- String getInfoSolution ()

  Informe detallado de la solución.
- double evaluateFitness ()

  Calcula la puntuación total de la solución.
- int addRoute ()

  Abre una nueva ruta vacía con un depósito virtual propio.
- double printSolution (PrintStream out)

  Escribe las rutas de la solución y su puntuación.

Atributos públicos estáticos

- static final int NO_INITIALIZED = -1

  Valor centinela para posiciones aún no asignadas.

Atributos privados

- final TOPTW problem

  Instancia del problema a la que pertenece la solución.
- int[ ] predecessors

  Predecesor de cada nodo en su ruta.
- int[ ] successors

  Sucesor de cada nodo en su ruta.
- final double[ ] waitingTime

  Tiempo de espera en cada POI.
- final int[ ] positionInRoute

  Posición de cada POI dentro de su ruta.
- int[ ] routes

  Índice del depósito de cada ruta creada.
- int availableVehicles

  Vehículos todavía no utilizados.
- double objectiveFunctionValue

  Último valor de la función objetivo calculado por TOPTWEvaluator.

7.13.1. Descripción detallada

Solución del problema TOPTW representada mediante listas doblemente enlazadas.

Cada ruta comienza y termina en un depósito. El depósito de la primera ruta es el nodo 0; las rutas adicionales usan depósitos virtuales con índices POIs+1, POIs+2, ... Los vectores predecessors y successors almacenan, para cada nodo, su anterior y su siguiente en la ruta, de modo que recorrer una ruta consiste en seguir los sucesores desde su depósito hasta volver a él.

La calidad (fitness) de la solución es la suma de las puntuaciones de los POI visitados.

Definición en la línea 19 del archivo TOPTWSolution.java.

7.13.2. Documentación del constructor y destructor

7.13.2.1. TOPTWSolution()

```
top.TOPTWSolution.TOPTWSolution (
            TOPTW problem )
```

Crea una solución vacía asociada a una instancia.

Parámetros

problem	instancia del problema.

Definición en la línea 43 del archivo TOPTWSolution.java.

Hace referencia a top.TOPTW.getPOIs(), top.TOPTW.getVehicles(), top.TOPTWEvaluator.NO_EVALUATED, top.↩ TOPTWSolution.NO_INITIALIZED y top.TOPTWSolution.problem.

Referenciado por top.TOPTWSolution.equals().

Gráfico de llamadas para esta función:

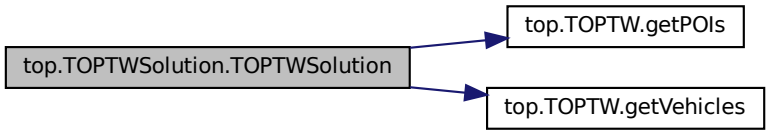
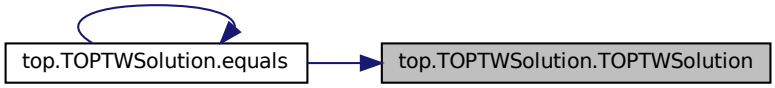


Gráfico de llamadas a esta función:



7.13.3. Documentación de las funciones miembro

7.13.3.1. initSolution()

```
void top.TOPTWSolution.initSolution ( )
```

Reinicia la solución dejando una única ruta vacía (depósito 0 → depósito 0).

Definición en la línea 61 del archivo TOPTWSolution.java.

Hace referencia a top.TOPTW.getPOIs(), top.TOPTW.getVehicles() y top.TOPTWSolution.NO_INITIALIZED.

Referenciado por top.TOPTWGRASP.computeGreedySolution().

Gráfico de llamadas para esta función:

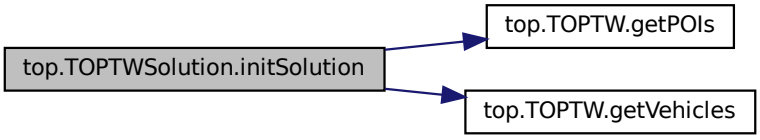


Gráfico de llamadas a esta función:



7.13.3.2. isDepot()

```
boolean top.TOPTWSolution.isDepot (
            int c )
```

Indica si un nodo es el depósito de alguna ruta de la solución.

Parámetros

c	índice del nodo.

Devuelve

true si c es el depósito de alguna ruta.

Definición en la línea 79 del archivo TOPTWSolution.java.

Referenciado por top.TOPTWGRASP.updateSolution().

Gráfico de llamadas a esta función:



7.13.3.3. equals()

```
boolean top.TOPTWSolution.equals (
            Object o )
```

Compara dos soluciones por su vector de predecesores.

Parámetros

o	objeto a comparar.

Devuelve

true si ambas soluciones tienen los mismos predecesores.

Definición en la línea 94 del archivo TOPTWSolution.java.

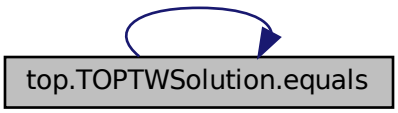
Hace referencia a top.TOPTWSolution.equals(), top.TOPTWSolution.predecessors y top.TOPTWSolution.↩ TOPTWSolution().

Referenciado por top.TOPTWSolution.equals().

Gráfico de llamadas para esta función:



Gráfico de llamadas a esta función:



7.13.3.4. hashCode()

```
int top.TOPTWSolution.hashCode ( )
```

Código hash coherente con equals().

Devuelve

hash del vector de predecesores.

Definición en la línea 110 del archivo TOPTWSolution.java.

7.13.3.5. getAvailableVehicles()

```
int top.TOPTWSolution.getAvailableVehicles ( )
```

Vehículos todavía disponibles.

Devuelve

número de vehículos sin ruta asignada.

Definición en la línea 118 del archivo TOPTWSolution.java.

Hace referencia a top.TOPTWSolution.availableVehicles.

7.13.3.6. getCreatedRoutes()

```
int top.TOPTWSolution.getCreatedRoutes ( )
```

Número de rutas creadas en la solución.

Devuelve

rutas abiertas.

Definición en la línea 126 del archivo TOPTWSolution.java.

Hace referencia a top.TOPTWSolution.availableVehicles y top.TOPTW.getVehicles().

Referenciado por top.TOPTWGRASP.comprehensiveEvaluation(), top.TOPTWGRASP.computeGreedySolution(), top.TOPTWSolution.evaluateFitness(), top.TOPTWSolution.getInfoSolution() y top.TOPTWSolution.printSolution().

Gráfico de llamadas para esta función:

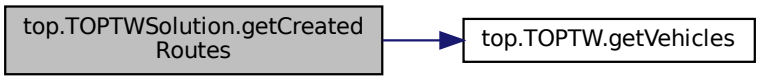
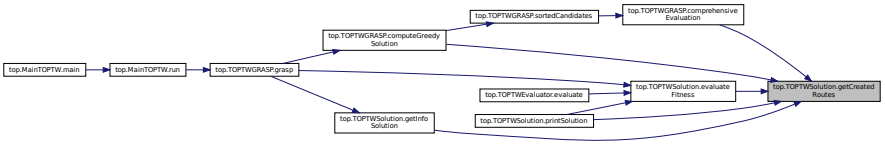


Gráfico de llamadas a esta función:



7.13.3.7. getDistance()

```
double top.TOPTWSolution.getDistance (
            int x,
            int y )
```

Distancia entre dos nodos de la instancia.

Parámetros

x	nodo origen.
y	nodo destino.

Devuelve

distancia entre ambos.

Definición en la línea 136 del archivo TOPTWSolution.java.

Hace referencia a top.TOPTW.getDistance().

Referenciado por top.TOPTWSolution.getInfoSolution(), top.TOPTWGRASP.insertionCost() y top.TOPTWGRASP.↩ updateSolution().

Gráfico de llamadas para esta función:

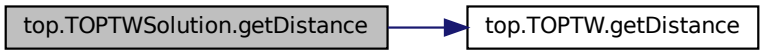
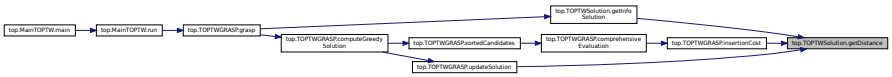


Gráfico de llamadas a esta función:



7.13.3.8. setAvailableVehicles()

```
void top.TOPTWSolution.setAvailableVehicles (
             int availableVehicles )
```

Establece el número de vehículos disponibles.

Parámetros

availableVehicles	vehículos disponibles.

Definición en la línea 144 del archivo TOPTWSolution.java.

Hace referencia a top.TOPTWSolution.availableVehicles.

7.13.3.9. getPredecessor()

```
int top.TOPTWSolution.getPredecessor (
             int customer )
```

Predecesor de un nodo.

Parámetros

customer	nodo.

Devuelve

predecesor en su ruta.

Definición en la línea 153 del archivo TOPTWSolution.java.

7.13.3.10. getPredecessors()

```
int [] top.TOPTWSolution.getPredecessors ( )
```

Copia del vector de predecesores.

Devuelve

predecesores indexados por nodo.

Definición en la línea 161 del archivo TOPTWSolution.java.

7.13.3.11. getProblem()

```
TOPTW top.TOPTWSolution.getProblem ( )
```

Instancia del problema.

Devuelve

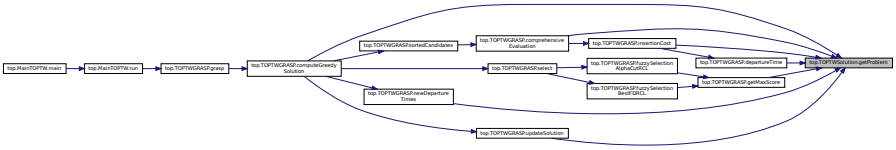
problema asociado.

Definición en la línea 169 del archivo TOPTWSolution.java.

Hace referencia a top.TOPTWSolution.problem.

Referenciado por top.TOPTWGRASP.comprehensiveEvaluation(), top.TOPTWGRASP.computeGreedySolution(), top.TOPTWGRASP.departureTime(), top.TOPTWGRASP.getMaxScore(), top.TOPTWGRASP.insertionCost(), top.TOPTWGRASP.newDepartureTimes() y top.TOPTWGRASP.updateSolution().

Gráfico de llamadas a esta función:



7.13.3.12. getObjectiveFunctionValue()

```
double top.TOPTWSolution.getObjectiveFunctionValue ( )
```

Valor de la función objetivo almacenado.

Devuelve

valor objetivo, o TOPTWEvaluator::NO_EVALUATED si no se ha evaluado.

Definición en la línea 177 del archivo TOPTWSolution.java.

Hace referencia a top.TOPTWSolution.objectiveFunctionValue.

7.13.3.13. getPositionInRoute()

```
int top.TOPTWSolution.getPositionInRoute (
            int customer )
```

Posición de un POI en su ruta.

Parámetros

customer	POI.

Devuelve

posición dentro de la ruta.

Definición en la línea 186 del archivo TOPTWSolution.java.

7.13.3.14. getSuccessor()

```
int top.TOPTWSolution.getSuccessor (
            int customer )
```

Sucesor de un nodo.

Parámetros

customer	nodo.

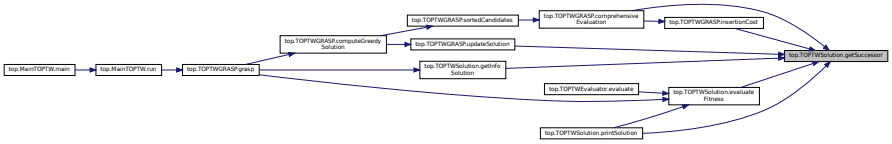
Devuelve

sucesor en su ruta.

Definición en la línea 195 del archivo TOPTWSolution.java.

Referenciado por top.TOPTWGRASP.comprehensiveEvaluation(), top.TOPTWSolution.evaluateFitness(), top.↩ TOPTWSolution.getInfoSolution(), top.TOPTWGRASP.insertionCost(), top.TOPTWSolution.printSolution() y top.↩ TOPTWGRASP.updateSolution().

Gráfico de llamadas a esta función:



7.13.3.15. getSuccessors()

```
int [] top.TOPTWSolution.getSuccessors ( )
```

Copia del vector de sucesores.

Devuelve

sucesores indexados por nodo.

Definición en la línea 203 del archivo TOPTWSolution.java.

7.13.3.16. getIndexRoute()

```
int top.TOPTWSolution.getIndexRoute (
            int index )
```

Depósito de la ruta k-ésima.

Parámetros

index	número de ruta.

Devuelve

índice del depósito que abre y cierra esa ruta.

Definición en la línea 212 del archivo TOPTWSolution.java.

Referenciado por top.TOPTWGRASP.comprehensiveEvaluation(), top.TOPTWSolution.evaluateFitness(), top.↩ TOPTWSolution.getInfoSolution(), top.TOPTWGRASP.insertionCost(), top.TOPTWSolution.printSolution() y top.↩ TOPTWGRASP.updateSolution().

Gráfico de llamadas a esta función:



7.13.3.17. getWaitingTime()

```
double top.TOPTWSolution.getWaitingTime (
            int customer )
```

Tiempo de espera en un POI.

Parámetros

customer	POI.

Devuelve

tiempo de espera.

Definición en la línea 221 del archivo TOPTWSolution.java.

7.13.3.18. setObjectiveFunctionValue()

```
void top.TOPTWSolution.setObjectiveFunctionValue (
            double objectiveFunctionValue )
```

Almacena el valor de la función objetivo.

Parámetros

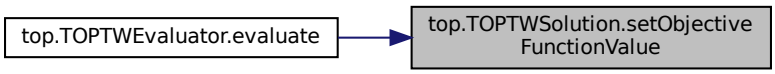
objectiveFunctionValue	valor calculado.

Definición en la línea 229 del archivo TOPTWSolution.java.

Hace referencia a top.TOPTWSolution.objectiveFunctionValue.

Referenciado por top.TOPTWEvaluator.evaluate().

Gráfico de llamadas a esta función:



7.13.3.19. setPositionInRoute()

```
void top.TOPTWSolution.setPositionInRoute (
            int customer,
            int position )
```

Establece la posición de un POI en su ruta.

Parámetros

customer	POI.
position	posición.

Definición en la línea 238 del archivo TOPTWSolution.java.

7.13.3.20. setPredecessor()

```
void top.TOPTWSolution.setPredecessor (
            int customer,
            int predecessor )
```

Establece el predecesor de un nodo.

Parámetros

customer	nodo.
predecessor	predecesor.

Definición en la línea 247 del archivo TOPTWSolution.java.

Referenciado por top.TOPTWGRASP.updateSolution().

Gráfico de llamadas a esta función:



7.13.3.21. setSuccessor()

```
void top.TOPTWSolution.setSuccessor (
            int customer,
            int successor )
```

Establece el sucesor de un nodo.

Parámetros

customer	nodo.
successor	sucesor.

Definición en la línea 256 del archivo TOPTWSolution.java.

Referenciado por top.TOPTWGRASP.updateSolution().

Gráfico de llamadas a esta función:



7.13.3.22. setWaitingTime()

```
void top.TOPTWSolution.setWaitingTime (
            int customer,
            double waitingTime )
```

Establece el tiempo de espera de un POI.

Parámetros

customer	POI.
waitingTime	tiempo de espera.

Definición en la línea 265 del archivo TOPTWSolution.java.

Hace referencia a top.TOPTWSolution.waitingTime.

7.13.3.23. getInfoSolution()

```
String top.TOPTWSolution.getInfoSolution ( )
```

Informe detallado de la solución.

Incluye la secuencia de nodos de cada ruta, si la solución es factible, la puntuación total, el coste temporal y una tabla por ruta con los tiempos de llegada y salida de cada nodo.

Devuelve

texto con el informe.

Definición en la línea 278 del archivo TOPTWSolution.java.

Hace referencia a top.TOPTWSolution.getCreatedRoutes(), top.TOPTWSolution.getDistance(), top.TOPTW.getDueTime(), es.ull.esit.utilities.ExpositoUtilities.getFormat(), top.TOPTWSolution.getIndexRoute(), top.TOPTW.getMaxRoutes(), top.TOPTW.getMaxTimePerRoute(), top.TOPTW.getPOIs(), top.TOPTW.getReadyTime(), top.TOPTW.getScore(), top.TOPTW.getServiceTime(), top.TOPTWSolution.getSuccessor(), top.TOPTW.getX() y top.TOPTW.getY().

Referenciado por top.TOPTWGRASP.grasp().

Gráfico de llamadas para esta función:

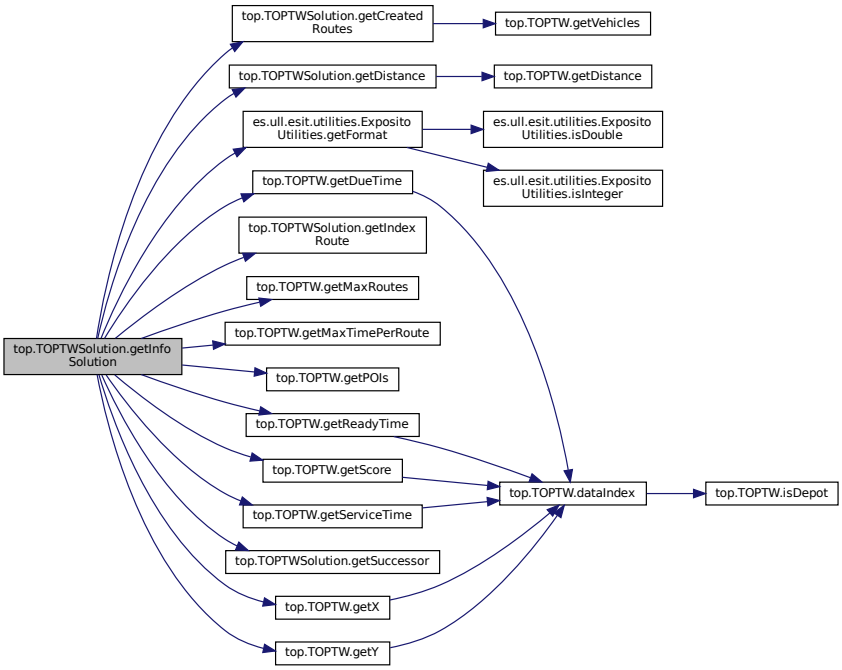


Gráfico de llamadas a esta función:



7.13.3.24. evaluateFitness()

```
double top.TOPTWSolution.evaluateFitness ( )
```

Calcula la puntuación total de la solución.

Recorre todas las rutas creadas sumando la puntuación de cada nodo visitado (los depósitos tienen puntuación 0).

Devuelve

suma de puntuaciones de los POI visitados.

Definición en la línea 357 del archivo TOPTWSolution.java.

Hace referencia a top.TOPTWSolution.getCreatedRoutes(), top.TOPTWSolution.getIndexRoute(), top.TOPTW.↩ getScore() y top.TOPTWSolution.getSuccessor().

Referenciado por top.TOPTWEvaluator.evaluate(), top.TOPTWGRASP.grasp() y top.TOPTWSolution.print↩ Solution().

Gráfico de llamadas para esta función:

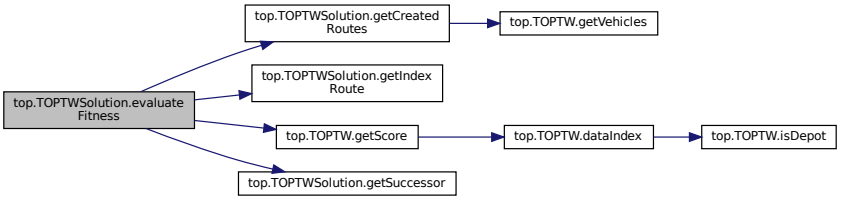
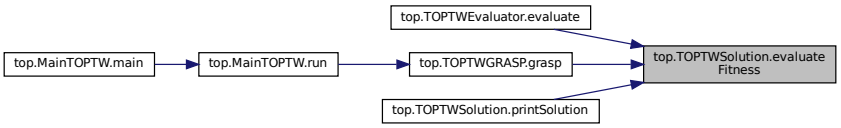


Gráfico de llamadas a esta función:



7.13.3.25. addRoute()

```
int top.TOPTWSolution.addRoute ( )
```

Abre una nueva ruta vacía con un depósito virtual propio.

El nuevo depósito recibe el índice siguiente al último depósito virtual creado (o POIs + 1 si es el primero) y se enlaza consigo mismo.

Devuelve

índice del depósito de la nueva ruta.

Definición en la línea 380 del archivo TOPTWSolution.java.

Hace referencia a top.TOPTW.addNodeDepot(), top.TOPTW.getPOIs() y top.TOPTWSolution.NO_INITIALIZED.

Referenciado por top.TOPTWGRASP.computeGreedySolution().

Gráfico de llamadas para esta función:

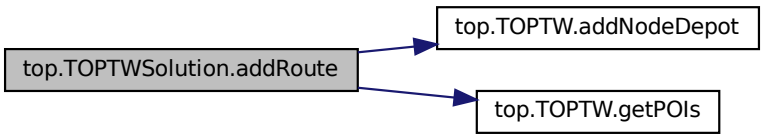


Gráfico de llamadas a esta función:



7.13.3.26. printSolution()

```
double top.TOPTWSolution.printSolution (
            PrintStream out )
```

Escribe las rutas de la solución y su puntuación.

Parámetros

out	flujo de salida.

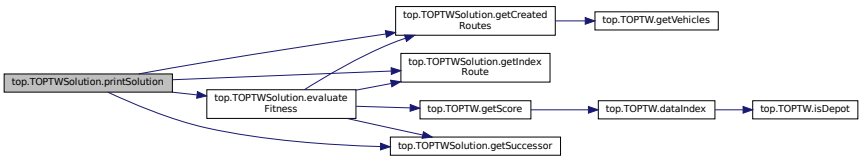
Devuelve

puntuación de la solución.

Definición en la línea 402 del archivo TOPTWSolution.java.

Hace referencia a top.TOPTWSolution.evaluateFitness(), top.TOPTWSolution.getCreatedRoutes(), top.↩ TOPTWSolution.getIndexRoute() y top.TOPTWSolution.getSuccessor().

Gráfico de llamadas para esta función:



7.13.4. Documentación de los datos miembro

7.13.4.1. NO_INITIALIZED

```
final int top.TOPTWSolution.NO_INITIALIZED = -1  [static]
```

Valor centinela para posiciones aún no asignadas.

Definición en la línea 21 del archivo TOPTWSolution.java.

Referenciado por top.TOPTWSolution.addRoute(), top.TOPTWSolution.initSolution(), top.TOPTWRoute.↩ TOPTWRoute() y top.TOPTWSolution.TOPTWSolution().

7.13.4.2. problem

```
final TOPTW top.TOPTWSolution.problem  [private]
```

Instancia del problema a la que pertenece la solución.

Definición en la línea 23 del archivo TOPTWSolution.java.

Referenciado por top.TOPTWSolution.getProblem() y top.TOPTWSolution.TOPTWSolution().

7.13.4.3. predecessors

```
int [] top.TOPTWSolution.predecessors  [private]
```

Predecesor de cada nodo en su ruta.

Definición en la línea 25 del archivo TOPTWSolution.java.

Referenciado por top.TOPTWSolution.equals().

7.13.4.4. successors

```
int [] top.TOPTWSolution.successors  [private]
```

Sucesor de cada nodo en su ruta.

Definición en la línea 27 del archivo TOPTWSolution.java.

7.13.4.5. waitingTime

```
final double [] top.TOPTWSolution.waitingTime  [private]
```

Tiempo de espera en cada POI.

Definición en la línea 29 del archivo TOPTWSolution.java.

Referenciado por top.TOPTWSolution.setWaitingTime().

7.13.4.6. positionInRoute

```
final int [] top.TOPTWSolution.positionInRoute  [private]
```

Posición de cada POI dentro de su ruta.

Definición en la línea 31 del archivo TOPTWSolution.java.

7.13.4.7. routes

```
int [] top.TOPTWSolution.routes  [private]
```

Índice del depósito de cada ruta creada.

Definición en la línea 33 del archivo TOPTWSolution.java.

7.13.4.8. availableVehicles

```
int top.TOPTWSolution.availableVehicles  [private]
```

Vehículos todavía no utilizados.

Definición en la línea 35 del archivo TOPTWSolution.java.

Referenciado por top.TOPTWSolution.getAvailableVehicles(), top.TOPTWSolution.getCreatedRoutes() y top.↩ TOPTWSolution.setAvailableVehicles().

7.13.4.9. objectiveFunctionValue

```
double top.TOPTWSolution.objectiveFunctionValue  [private]
```

Último valor de la función objetivo calculado por TOPTWEvaluator.

Definición en la línea 37 del archivo TOPTWSolution.java.

Referenciado por top.TOPTWSolution.getObjectiveFunctionValue() y top.TOPTWSolution.setObjectiveFunction↩ Value().

La documentación para esta clase fue generada a partir del siguiente fichero:

- top/TOPTWSolution.java

Capítulo 8

Documentación de archivos

8.1. Referencia del Archivo /home/andrea/Descargas/ExpositoTOP/doxygen/mainpage.md

8.2. Referencia del Archivo es/ull/esit/utilities/BellmanFord.java

Clases

- class es.ull.esit.utilities.BellmanFord

  Algoritmo de Bellman-Ford sobre una matriz de distancias.

Paquetes

- package es.ull.esit.utilities

8.3. Referencia del Archivo es/ull/esit/utilities/ExpositoUtilities.java

Clases

- class es.ull.esit.utilities.ExpositoUtilities

  Funciones de utilidad: formato de texto tabular, ficheros, matrices y grafos.

Paquetes

- package es.ull.esit.utilities

8.4. Referencia del Archivo es/ull/esit/utilities/PowerSet.java

Clases

- class es.ull.esit.utilities.PowerSet< E >

  Conjunto potencia (todos los subconjuntos) de un conjunto, recorrible con for-each.
- class es.ull.esit.utilities.PowerSet< E >.PowerSetIterator

  Iterador sobre los subconjuntos, basado en un contador binario.

Paquetes

- package es.ull.esit.utilities

8.5. Referencia del Archivo es/ull/esit/utils/Pair.java

Paquetes

- package es.ull.esit.utils

Funciones

- record es.ull.esit.utils.Pair< F, S > (F first, S second)

  Par inmutable de valores.

8.6. Referencia del Archivo top/MainTOPTW.java

Clases

- class top.MainTOPTW

  Punto de entrada del recomendador turístico RTVRPTW / TOPTW.
- class top.MainTOPTW.Options

  Opciones de ejecución obtenidas de la línea de órdenes.

Paquetes

- package top

8.7. Referencia del Archivo top/TOPTW.java

Clases

- class top.TOPTW

  Modelo de una instancia del problema TOPTW (Team Orienteering Problem with Time Windows).

Paquetes

- package top

8.8. Referencia del Archivo top/TOPTWEvaluator.java

Clases

- class top.TOPTWEvaluator

  Evaluador de la función objetivo del TOPTW.

Paquetes

- package top

8.9. Referencia del Archivo top/TOPTWGRASP.java

Clases

- class top.TOPTWGRASP

  Metaheurística GRASP (Greedy Randomized Adaptive Search Procedure) para el TOPTW.
- enum top.TOPTWGRASP.SelectionStrategy

  Estrategias de selección de un candidato dentro de la RCL.

Paquetes

- package top

8.10. Referencia del Archivo top/TOPTWReader.java

Clases

- class top.TOPTWReader

  Lector de instancias TOPTW en el formato de Cordeau / Righini.

Paquetes

- package top

8.11. Referencia del Archivo top/TOPTWRoute.java

Clases

- class top.TOPTWRoute

  Descriptor ligero de una ruta: identificador y nodos extremos.

Paquetes

- package top

8.12. Referencia del Archivo top/TOPTWSolution.java

Clases

- class top.TOPTWSolution

  Solución del problema TOPTW representada mediante listas doblemente enlazadas.

Paquetes

- package top

Índice alfabético

/home/andrea/Descargas/ExpositoTOP/doxygen/mainpage.md, 139

addNode
    top.TOPTW, 75
addNodeDepot
    top.TOPTW, 75
addRoute
    top.TOPTWSolution, 133
aleatorySelectionRCL
    top.TOPTWGRASP, 88
ALIGNMENT_LEFT
    es.ull.esit.utilities.ExpositoUtilities, 35
ALIGNMENT_RIGHT
    es.ull.esit.utilities.ExpositoUtilities, 35
alpha
    top.TOPTWGRASP, 105
arr
    es.ull.esit.utilities.PowerSet< E >, 48
availableVehicles
    top.TOPTWSolution, 136

BellmanFord
    es.ull.esit.utilities.BellmanFord, 15
bset
    es.ull.esit.utilities.PowerSet< E >.PowerSetIterator, 51

calculateDistanceMatrix
    top.TOPTW, 60
calculateEdges
    es.ull.esit.utilities.BellmanFord, 16
CANDIDATE_INFO_SIZE
    top.TOPTWGRASP, 103
comprehensiveEvaluation
    top.TOPTWGRASP, 97
computeGreedySolution
    top.TOPTWGRASP, 93
COST
    top.TOPTWGRASP, 103
CUSTOMER
    top.TOPTWGRASP, 102

dataIndex
    top.TOPTW, 57
DEFAULT_ALPHA
    top.TOPTWGRASP, 104
DEFAULT_COLUMN_WIDTH
    es.ull.esit.utilities.ExpositoUtilities, 35
DEFAULT_DIR
    top.MainTOPTW, 42
DEFAULT_INSTANCES
    top.MainTOPTW, 43
DEFAULT_ITERATIONS
    top.MainTOPTW, 42
departureTime
    top.TOPTWGRASP, 95
depots
    top.TOPTW, 78
dir
    top.MainTOPTW.Options, 45
distanceMatrix
    es.ull.esit.utilities.BellmanFord, 18
    top.TOPTW, 79
distances
    es.ull.esit.utilities.BellmanFord, 19
dueTime
    top.TOPTW, 77

edges1
    es.ull.esit.utilities.BellmanFord, 18
edges2
    es.ull.esit.utilities.BellmanFord, 18
equals
    top.TOPTWSolution, 121
es.ull.esit.utilities, 11
es.ull.esit.utilities.BellmanFord, 13
    BellmanFord, 15
    calculateEdges, 16
    distanceMatrix, 18
    distances, 19
    edges1, 18
    edges2, 18
    getDistances, 16
    getValue, 17
    INFINITY, 17
    nodes, 18
    path, 18
    solve, 17
    value, 19
es.ull.esit.utilities.ExpositoUtilities, 20
    ALIGNMENT_LEFT, 35
    ALIGNMENT_RIGHT, 35
    DEFAULT_COLUMN_WIDTH, 35
    ExpositoUtilities, 22
    getFormat, 24–31
    getFormatIntegers, 28
    hasArc, 34
    isAcyclic, 33

isDouble, 32
isInteger, 32
multiplyMatrices, 23
printFile, 22
simplifyString, 22
thereIsPath, 33
writeTextToFile, 23
es.ull.esit.utilities.PowerSet< E >, 46
arr, 48
iterator, 48
PowerSet, 47
es.ull.esit.utilities.PowerSet< E >.PowerSetIterator, 49
bset, 51
hasNext, 50
next, 50
es.ull.esit.utils, 11
Pair< F, S >, 11
es/ull/esit/utilities/BellmanFord.java, 139
es/ull/esit/utilities/ExpositoUtilities.java, 139
es/ull/esit/utilities/PowerSet.java, 139
es/ull/esit/utils/Pair.java, 140
evaluate
top.TOPTWEvaluator, 80
evaluateFitness
top.TOPTWSolution, 132
ExpositoUtilities
es.ull.esit.utilities.ExpositoUtilities, 22

FUZZY_ALPHA_CUT
top.TOPTWGRASP.SelectionStrategy, 52
FUZZY_BEST
top.TOPTWGRASP.SelectionStrategy, 52
fuzzySelectionAlphaCutRCL
top.TOPTWGRASP, 90
fuzzySelectionBestFDRCL
top.TOPTWGRASP, 89

getAlpha
top.TOPTWGRASP, 100
getAvailableVehicles
top.TOPTWSolution, 123
getCreatedRoutes
top.TOPTWSolution, 123
getDepots
top.TOPTW, 74
getDistance
top.TOPTW, 58, 62
top.TOPTWSolution, 124
getDistances
es.ull.esit.utilities.BellmanFord, 16
getDueTime
top.TOPTW, 71
getFormat
es.ull.esit.utilities.ExpositoUtilities, 24–31
getFormatIntegers
es.ull.esit.utilities.ExpositoUtilities, 28
getId
top.TOPTWRoute, 114
getIndexRoute
top.TOPTWSolution, 128
getInfoSolution
top.TOPTWSolution, 132
getLastAverageFitness
top.TOPTWGRASP, 101
getLastBestFitness
top.TOPTWGRASP, 101
getMaxRoutes
top.TOPTW, 61
getMaxScore
top.TOPTWGRASP, 101
getMaxTimePerRoute
top.TOPTW, 60
getNodes
top.TOPTW, 64
getObjectiveFunctionValue
top.TOPTWSolution, 126
getPOIs
top.TOPTW, 62
getPositionInRoute
top.TOPTWSolution, 127
getPredecessor
top.TOPTWRoute, 113
top.TOPTWSolution, 125
getPredecessors
top.TOPTWSolution, 126
getProblem
top.TOPTWSolution, 126
getReadyTime
top.TOPTW, 70
getScore
top.TOPTW, 68, 69
getServiceTime
top.TOPTW, 72
getSolution
top.TOPTWGRASP, 98
getSolutionTime
top.TOPTWGRASP, 99
getStrategy
top.TOPTWGRASP, 99
getSuccessor
top.TOPTWRoute, 113
top.TOPTWSolution, 127
getSuccessors
top.TOPTWSolution, 128
getTime
top.TOPTW, 63
getTotalDistance
top.TOPTW, 59
getValue
es.ull.esit.utilities.BellmanFord, 17
getVehicles
top.TOPTW, 74
getWaitingTime
top.TOPTWSolution, 129
getX
top.TOPTW, 64
getY

top.TOPTW, 67
grasp
top.TOPTWGRASP, 86

hasArc
es.ull.esit.utilities.ExpositoUtilities, 34
hashCode
top.TOPTWSolution, 122
hasNext
es.ull.esit.utilities.PowerSet< E >.PowerSetIterator, 50

id
top.TOPTWRoute, 115
INFINITY
es.ull.esit.utilities.BellmanFord, 17
initSolution
top.TOPTWSolution, 120
insertionCost
top.TOPTWGRASP, 96
instances
top.MainTOPTW.Options, 45
isAcyclic
es.ull.esit.utilities.ExpositoUtilities, 33
isDepot
top.TOPTW, 56
top.TOPTWSolution, 121
isDouble
es.ull.esit.utilities.ExpositoUtilities, 32
isInteger
es.ull.esit.utilities.ExpositoUtilities, 32
iterations
top.MainTOPTW.Options, 45
iterator
es.ull.esit.utilities.PowerSet< E >, 48

lastAverageFitness
top.TOPTWGRASP, 105
lastBestFitness
top.TOPTWGRASP, 106
LOGGER
top.MainTOPTW, 43

main
top.MainTOPTW, 41
MainTOPTW
top.MainTOPTW, 38
maxRoutes
top.TOPTW, 78
maxTimePerRoute
top.TOPTW, 78
membership
top.TOPTWGRASP, 88
multiplyMatrices
es.ull.esit.utilities.ExpositoUtilities, 23

newDepartureTimes
top.TOPTWGRASP, 91
next
es.ull.esit.utilities.PowerSet< E >.PowerSetIterator, 50
nextFields
top.TOPTWReader, 109
NO_EVALUATED
top.TOPTWEvaluator, 81
NO_INITIALIZED
top.TOPTWSolution, 135
nodes
es.ull.esit.utilities.BellmanFord, 18
top.TOPTW, 76

objectiveFunctionValue
top.TOPTWSolution, 137
optionValue
top.MainTOPTW, 39
out
top.TOPTWGRASP, 105

Pair< F, S >
es.ull.esit.utils, 11
parseArgs
top.MainTOPTW, 38
path
es.ull.esit.utilities.BellmanFord, 18
positionInRoute
top.TOPTWSolution, 136
PowerSet
es.ull.esit.utilities.PowerSet< E >, 47
PREDECESSOR
top.TOPTWGRASP, 103
predecessor
top.TOPTWRoute, 115
predecessors
top.TOPTWSolution, 135
print
top.TOPTWGRASP, 87
printFile
es.ull.esit.utilities.ExpositoUtilities, 22
printSolution
top.TOPTWSolution, 134
problem
top.TOPTWSolution, 135

RANDOM
top.TOPTWGRASP.SelectionStrategy, 52
random
top.TOPTWGRASP, 104
RCL_SIZES
top.MainTOPTW, 42
readProblem
top.TOPTWReader, 108
readyTime
top.TOPTW, 77
ROUTE
top.TOPTWGRASP, 102
routes
top.TOPTWSolution, 136
run

top.MainTOPTW, 40

SCORE
top.TOPTWGRASP, 103
score
top.TOPTW, 77
SCORE_FIELD
top.TOPTWReader, 111
seed
top.MainTOPTW.Options, 45
select
top.TOPTWGRASP, 90
SERVICE_FIELD
top.TOPTWReader, 110
serviceTime
top.TOPTW, 77
setAlpha
top.TOPTWGRASP, 100
setAvailableVehicles
top.TOPTWSolution, 125
setDueTime
top.TOPTW, 72
setId
top.TOPTWRoute, 115
setMaxRoutes
top.TOPTW, 61
setMaxTimePerRoute
top.TOPTW, 60
setNodes
top.TOPTW, 64
setObjectiveFunctionValue
top.TOPTWSolution, 129
setOutput
top.TOPTWGRASP, 101
setPositionInRoute
top.TOPTWSolution, 130
setPredecessor
top.TOPTWRoute, 114
top.TOPTWSolution, 130
setReadyTime
top.TOPTW, 70
setScore
top.TOPTW, 69
setServiceTime
top.TOPTW, 73
setSolution
top.TOPTWGRASP, 98
setSolutionTime
top.TOPTWGRASP, 99
setStrategy
top.TOPTWGRASP, 100
setSuccessor
top.TOPTWRoute, 114
top.TOPTWSolution, 131
setWaitingTime
top.TOPTWSolution, 131
setX
top.TOPTW, 66
setY
top.TOPTW, 67
simplifyString
es.ull.esit.utilities.ExpositoUtilities, 22
solution
top.TOPTWGRASP, 104
solutionTime
top.TOPTWGRASP, 104
solve
es.ull.esit.utilities.BellmanFord, 17
sortedCandidates
top.TOPTWGRASP, 92
strategy
top.TOPTWGRASP, 105
successor
top.TOPTWRoute, 115
successors
top.TOPTWSolution, 136

thereIsPath
es.ull.esit.utilities.ExpositoUtilities, 33
top, 12
top.MainTOPTW, 36
DEFAULT_DIR, 42
DEFAULT_INSTANCES, 43
DEFAULT_ITERATIONS, 42
LOGGER, 43
main, 41
MainTOPTW, 38
optionValue, 39
parseArgs, 38
RCL_SIZES, 42
run, 40
top.MainTOPTW.Options, 44
dir, 45
instances, 45
iterations, 45
seed, 45
top.TOPTW, 53
addNode, 75
addNodeDepot, 75
calculateDistanceMatrix, 60
dataIndex, 57
depots, 78
distanceMatrix, 79
dueTime, 77
getDepots, 74
getDistance, 58, 62
getDueTime, 71
getMaxRoutes, 61
getMaxTimePerRoute, 60
getNodes, 64
getPOIs, 62
getReadyTime, 70
getScore, 68, 69
getServiceTime, 72
getTime, 63
getTotalDistance, 59
getVehicles, 74
getX, 64

getY, 67
isDepot, 56
maxRoutes, 78
maxTimePerRoute, 78
nodes, 76
readyTime, 77
score, 77
serviceTime, 77
setDueTime, 72
setMaxRoutes, 61
setMaxTimePerRoute, 60
setNodes, 64
setReadyTime, 70
setScore, 69
setServiceTime, 73
setX, 66
setY, 67
TOPTW, 56
toString, 74
vehicles, 78
x, 76
y, 76
top.TOPTWEvaluator, 79
evaluate, 80
NO_EVALUATED, 81
top.TOPTWGRASP, 81
aleatorySelectionRCL, 88
alpha, 105
CANDIDATE_INFO_SIZE, 103
comprehensiveEvaluation, 97
computeGreedySolution, 93
COST, 103
CUSTOMER, 102
DEFAULT_ALPHA, 104
departureTime, 95
fuzzySelectionAlphaCutRCL, 90
fuzzySelectionBestFDRCL, 89
getAlpha, 100
getLastAverageFitness, 101
getLastBestFitness, 101
getMaxScore, 101
getSolution, 98
getSolutionTime, 99
getStrategy, 99
grasp, 86
insertionCost, 96
lastAverageFitness, 105
lastBestFitness, 106
membership, 88
newDepartureTimes, 91
out, 105
PREDECESSOR, 103
print, 87
random, 104
ROUTE, 102
SCORE, 103
select, 90
setAlpha, 100
setOutput, 101
setSolution, 98
setSolutionTime, 99
setStrategy, 100
solution, 104
solutionTime, 104
sortedCandidates, 92
strategy, 105
TOPTWGRASP, 85
updateSolution, 94
top.TOPTWGRASP.SelectionStrategy, 51
FUZZY_ALPHA_CUT, 52
FUZZY_BEST, 52
RANDOM, 52
top.TOPTWReader, 106
nextFields, 109
readProblem, 108
SCORE_FIELD, 111
SERVICE_FIELD, 110
TOPTWReader, 107
X_FIELD, 110
Y_FIELD, 110
top.TOPTWRoute, 111
getId, 114
getPredecessor, 113
getSuccessor, 113
id, 115
predecessor, 115
setId, 115
setPredecessor, 114
setSuccessor, 114
successor, 115
TOPTWRoute, 112, 113
top.TOPTWSolution, 116
addRoute, 133
availableVehicles, 136
equals, 121
evaluateFitness, 132
getAvailableVehicles, 123
getCreatedRoutes, 123
getDistance, 124
getIndexRoute, 128
getInfoSolution, 132
getObjectiveFunctionValue, 126
getPositionInRoute, 127
getPredecessor, 125
getPredecessors, 126
getProblem, 126
getSuccessor, 127
getSuccessors, 128
getWaitingTime, 129
hashCode, 122
initSolution, 120
isDepot, 121
NO_INITIALIZED, 135
objectiveFunctionValue, 137
positionInRoute, 136
predecessors, 135

printSolution, 134
problem, 135
routes, 136
setAvailableVehicles, 125
setObjectiveFunctionValue, 129
setPositionInRoute, 130
setPredecessor, 130
setSuccessor, 131
setWaitingTime, 131
successors, 136
TOPTWSolution, 119
waitingTime, 136
top/MainTOPTW.java, 140
top/TOPTW.java, 140
top/TOPTWEvaluator.java, 140
top/TOPTWGRASP.java, 141
top/TOPTWReader.java, 141
top/TOPTWRoute.java, 141
top/TOPTWSolution.java, 141
TOPTW
top.TOPTW, 56
TOPTWGRASP
top.TOPTWGRASP, 85
TOPTWReader
top.TOPTWReader, 107
TOPTWRoute
top.TOPTWRoute, 112, 113
TOPTWSolution
top.TOPTWSolution, 119
toString
top.TOPTW, 74

updateSolution
top.TOPTWGRASP, 94

value
es.ull.esit.utilities.BellmanFord, 19
vehicles
top.TOPTW, 78

waitingTime
top.TOPTWSolution, 136
writeTextToFile
es.ull.esit.utilities.ExpositoUtilities, 23

x
top.TOPTW, 76
X_FIELD
top.TOPTWReader, 110

y
top.TOPTW, 76
Y_FIELD
top.TOPTWReader, 110